FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Implicit-Explicit Runge-Kutta Time Integration Schemes for Cilia Driven Flows

Dinis Gomes Sousa

THIS THESIS WAS DEVELOPED AT THE KU LEUVEN UNIVERSITY AS PART OF THE ERASMUS+ PROGRAM AND SUBMITTED TO THE UNIVERSITY OF PORTO FOR THE DEGREE OF:

Master in Mechanical Engineering - Fluids and Energy

Home University Supervisor: Célio Bruno Pinto Fernandes

Host University Supervisors: Johan Meyers
Benjamin Gorissen

Mentor: Nick Janssens

April 21, 2026

Resumo

Os escoamentos em regime de Stokes, também conhecidos como escoamentos lentos, caracterizam-se pela sua estabilidade, ordem e baixo número de Reynolds, sendo predominantes em ambientes microscópicos onde as forças viscosas dominam. No entanto, essa estabilidade representa um desafio significativo na Mecânica dos Fluidos Computacional pois impõe restrições temporais rigorosas, determinadas pela condição de Courant-Friedrichs-Lewy (CFL) viscosa, criando desafios numéricos compleos, para escoamentos tão simples. Este trabalho investiga a implementação de uma família de esquemas temporais IMEX (Implícito-Explícito) Lineares Multi-estágios Runge-Kutta para mitigar essa limitação e permitir simulações numéricas eficientes.

O foco principal deste trabalho é o desenvolvimento de um procedimento numérico para modelar escoamentos induzidos por cílios. Para tal, o código interno da KU Leuven, SP-Wind, originalmente desenvolvido para simulações de parques eólicos, é adaptado para simular escoamento em canal limitado entre duas placas paralelas fixas. Mais ainda, um cilindro estático, representando um único cílio, é fixado na parede inferior.

Antes de sua implementação no SP-Wind, o esquema IMEX foi formulado, com análise das suas propriedades de estabilidade. Além disso, os métodos foram validados usando dois casos de referência: a equação de Burgers unidimensional e um escoamento bidimensional em cavidade conduzida por tampa. Os resultados foram comparados com dados de referência estabelecidos e uma análise formal de ordem foi realizada para o escoamento na cavidade. A metodologia numérica emprega diversos esquemas de discretização espacial juntamente com três métodos de calculo da pressão, cada um acoplado a um *solver* específico: os métodos de gradiente conjugado e multi-malha algébrica para os casos de referência, e um *solver* baseado em decomposição LU para a aplicação dos cílios.

Foram desenvolvidos dois códigos capazes de implementar o esquema IMEX, cujos resultados foram considerados satisfatórios. O comportamento físico dos esquemas IMEX também foi verificado usando o caso de teste da cavidade conduzida por tampa, e o aumento admissível do passo temporal mostrou-se adequado. Infelizmente, a implementação no SP-Wind não teve sucesso, apesar de uma versão final ter sido codificada, devido a problemas relacionados às condições de fronteira apresentando resultados com caráter não-físico.

Palavras-chave: IMEX, Runge-Kutta, Cílios, Regime de Stokes, Equações de Navier-Stokes, SP-Wind, Cavidade conduzida por tampa, Equação de Burgers.

Abstract

Stokes flows, also known as creeping flows, are characterized by their stability, orderliness, and low Reynolds number regime, dominant in microscale environments where viscous forces prevail. However, this stability poses a significant challenge in Computational Fluid Dynamics (CFD), as it introduces stringent temporal constraints governed by the viscous Courant-Friedrichs-Lewy (CFL) condition, giving rise to complex numerical challenges. It is striking how such seemingly simple flows can give rise to complex numerical challenges. This work investigates the implementation of an Implicit-Explicit (IMEX) Linear Multistep Runge-Kutta family of temporal schemes to mitigate this limitation and enable efficient numerical simulation.

The primary focus of this work is the development of a numerical framework for modeling cilia-induced flows. To this end, the in-house KU Leuven code, SP-Wind, originally developed for wind farm simulations, is adapted to simulate channel flow bounded by two fixed parallel plates. In addition, a static cylinder, representing a single cilium, is attached to the bottom wall.

Before its implementation in SP-Wind, the IMEX scheme is formulated with a look at its stability properties. Furthermore, the numerical methods are validated using two benchmark cases: the one-dimensional Burgers equation and a two-dimensional lid-driven cavity flow. The results are compared with established benchmark data, and a formal order-of-accuracy analysis is performed for the cavity flow. The numerical methodology employs various spatial discretization schemes alongside three pressure calculation methods, each coupled with a specialized solver: the conjugate gradient and algebraic multi-grid methods for the benchmark cases, and an LU decomposition-based solver for the cilia application.

Two codes capable of implementing the IMEX scheme were developed, and their results were found to be satisfactory. The physical behavior of the IMEX schemes was also verified using the lid-driven cavity case study, and the admissible time-step increase was also found to be satisfactory. Unfortunately, the implementation in SP-Wind was not successful, despite a final version having been encoded, due to issues related to boundary conditions, the results showed nonphysical characteristics.

Key Words: IMEX, Runge-Kutta, Cilia, Stokes Flow, Navier-Stokes Equations, SP-Wind, Lid-driven Cavity, Burgers Equation.

Agradecimentos

Quero começar por agradecer ao Prof. Dr. Célio Bruno Pinto Fernandes por todo o apoio, dedicação e acompanhamento que teve comigo. Agradecer também todo o conhecimento que o mesmo me transmitiu e me instruiu, do excelente ambiente proporcionado e pelos bons momentos tidos ao longo destes quase 6 meses. Em momento algum senti falta de interesse de sua parte, mesmo o projeto não ter sido proposto por si. Agradeço também todas as recomendações de sua parte e por em junho me ter cartolado.

Do mesmo modo, agradecer ao Prof. Dr. Johan Meyers por me ter proposto o tema da minha dissertação, mesmo não me conhecendo de lugar algum. Agradeço por me ter acolhido como seu estudante e pela disponibilidade de me atribuir um lugar nos escritórios do departamento de mecânica. Sinto que este pequeno gesto tornou o foco na tese algo mais sério. Agradeço também toda a disponibilidade para burocracias relativas ao programa ERASMUS+ bem como para fins de recomendação. Quero também referir que reconheço todos os valiosos *insights* que apresentou nas reuniões.

Não obstante, um sincero e digno obrigado ao Dr. Nick Janssens por desempenhar um papel muito presente no meu acompanhamento ao longo desta dissertação. Agradeço todo o compromisso e ampla disponibilidade ao projeto, de principio ao fim. Embora no papel de mentor, considero-o devidamente como um orientador presente.

A estes dois, agradeço também o bom ambiente proporcionado em todas as reuniões e por todo o conhecimento que me transmitiram direta ou indiretamente. Quero reforçar que não tenho nada de negativo a apontar nos quatro meses que estive em Leuven e que me senti verdadeiramente integrado.

Quero também agradecer a dois Professores que indiretamente traçaram o caminho desta tese. Agradeço ao Sr. Prof. Dr. Fernando Manuel Pinho e Sr. Prof. Dr. José Laginha Palma. Obrigado por me despertarem um grande interesse na área de Mecânica dos Fluidos e por verdadeiramente me ensinarem sobre a área. O meu grande medo ao entrar no ensino superior insidia em não encontrar algo que gostasse verdadeiramente de fazer no futuro. Hoje digo com convicção que CFD está no meu caminho e que quero um dia poder estar ao vosso nível. Obrigado também por toda a disponibilidade para fins de recomendação e por me ajudarem a encontrar boas oportunidades para o futuro.

De seguida, quero agradecer aos serviços de Outgoing da Faculdade de Engenharia da Universidade do Porto, pela prontidão nas respostas a e-mails bem como pelos esclarecimentos ao processo de candidatura para a bolsa ERASMUS+. Tenho todo o reconhecimento pela Dra. Leonor, por mais de uma vez me apontar na direção correta. Igualmente agradeço ao Prof. Dr. José Machado não só pelos *insights* sobre o programa ERASMUS+ bem como pela carta de recomendação que me escreveu no final da tese.

Aos meus amigos da velha guarda, Álvaro, Sousa, Artur, Ian e Johnny, um obrigado não chega. Obrigado por serem sempre os mais sinceros comigo, por me ajudarem quando mais preciso e

por me proporcionarem momentos incríveis que só vocês conseguem. Embora tenhamos todos seguido direções diferentes, a nossa amizade pervalece e pervalecerá.

Aos mais recentes, começando pelo assunto, obrigado Campos, António Guedes, Kiko, Ruben, Manel, Fred e Jota. Obrigado por estes incríveis 5 anos de FEUP Caffés, Queimas, Noites de Enegenharia, Noites da Católica, Francesinhas, estudo, cinemas e Karts. Obrigado por me integrarem tão bem na universidade e por serem pessoas espetaculares, como diria o Varga. Obrigado por todas as noites que me deixaram dormir nas vossas casas, por toda a ramboia que por aqui se viveu. Definitivamente vocês são o Porto.

Aos do Mestrado, um forte obrigado a ti José Morais por partilhares dos mesmos interesses que eu, pelas longas conversas sobre tudo e pelos teus comentários tão próprios. De igual modo, quero te agradecer Alcobia por seres um verdadeiro motor humano. Sem o teu carácter único não teria vivido o que vivi na tese. Espero que tenhas o maior sucesso porque pessoas como tu não existem. Quero também agradecer a duas lendas, Sr. Eng. Francisco Calhindro e Tomás (Bodi) Rocha, por fazerem parte de forma muito presente no meu mestrado. São vocês todos que fazem a faculdade valer a pena.

Por fim, a Ti agradeço pela companhia, pelos bons momentos e pela coragem de embarcares nesta aventura comigo. Sei que nem tudo foi fácil mas ajudaste-me a desenvolver como Engenheiro e sobretudo como Pessoa. Guardo todos os momentos com amor e saudade e sei que tudo isto ficará para sempre na minha memória. As pessoas são como as marés, vão e vêm.

Espero embarcar numa nova aventura nos próximos meses e anseio saber qual vai ser. Se há coisa que notei é que a vida vai passando cada vez mais rápido. Por isso, agradeço a ti vida por me proporcionares tudo isto. De um pequeno e sincero rapaz da ilha para o mundo.

Declaration of AI use

The following chapter is an honest and open declaration regarding the use of Artificial Intelligence (AI). I, the author of this report, state that only free AI tools were used in this thesis. Their applications were purely auxiliary, namely: (i) to make the writing clearer and easier to read, (ii) to assist in the debugging process, and (iii) to help identify relevant references.

Regarding point (i), having dyslexia makes the writing process slow and challenging. I tend to make grammatical errors and produce poorly structured sentences, especially under time constraints. AI served as an optimizer, making the process faster and more accurate. However, at no point did I explicitly ask any AI to write text on a given topic for me. My process was to first write the sentence or paragraph myself, then paste it into the AI with the request: "Correct any grammar errors and improve the structure. Use proper language for a thesis report, be as direct as possible, and avoid excessive use of adjectives".

At the beginning of this thesis, I inquire my Mentor, Nick, whether I could use AI to help me write the code. My concern was that they might not allow pasting the source code of *SP-Wind* into any AI tool, since it is confidential. They gave me permission to do so, and I mainly used *DeepSeek* and *ChatGPT* for debugging purposes. These tools also assisted me in constructing the Burgers' equation and the lid-driven cavity test cases, mainly in the post-processing field. I emphasize that I never asked the AI systems to do the work for me, such as writing the code itself.

Lastly, I declare that I used an extension of *ChatGPT* of name *Consensus*, as a research tool for documents such as papers and books. With that I could easily navigate through web, finding great benchmark studies for the constructed test cases as well as fundamental works that support almost every section of this report.

AI is a powerful tool that, whether we like it or not, will be part of an engineer's life from now on. It has shown that it can coexist with the human mind and that certain tasks can be simplified with its assistance. It is up to authors to remain aware of its use and to be open to its declaration of use.

"We must use time as a tool, not as a couch."

— John F. Kennedy (1917-1963)

Contents

List of Figures

List of Tables

Abbreviations and Symbols

C_D	Drag coefficient of a the cylinder
Co	Courant number
Eu	Euler number
Fr	Froude number
Re	Reynolds number
St	Strouhal number
$\forall$	Volume
ε^r	Relative error
Ω	Computational domain
γ_g	Euler–Mascheroni, $\gamma_g \approx 0.57721$
λ	Eigenvalue
Λ	Laplacian Operator for the velocity
ν	Kinematic viscosity
∇_n^2	Laplacian Operator for the pressure
ρ	Density
σ	Number of explicit stages
ϕ	Potential quantity
A	Runge-Kutta Matrix
AMG	Algebraic Multigrid method
ARC	Asymptotic Rate of Convergence
ASC	Ascher method
BC	Boundary condition operator
b	Weights vectors
CG	Conjugate Gradient method
CFL	Courant–Friedrichs–Lewy condition
c	Node vectors
D_{cil}	Cilia diameter
$\overline{E}_k$	Mean kinetic energy
E_1	Fractional error
FE	Forward Euler methods
f	Characteristic frequency
f_D	Body force component
G_i	Grid
GCI	Grid Convergence Index
G	Gaussian filter
g	Gravitational acceleration
H	Sum of the explicit terms of the numerical applied scheme
h_{cil}	Cilia height

IMEX	Implicit-Explicit
K	Slope variable
L_c	Characteristic length
L_i	Total length in direction i
N_{elm}	Total number of elements
N_i	Number of grid points in direction i
P	Total pressure contribution at the IMEX stages
p	Total pressure
p_0	Reference pressure
p_∞	Surroundings pressure
$\hat{p}$	Order-of-convergence
RHS	Right-hand side operator
RK4	Classical Runge-Kutaa methods, 4 stages 4^{th} order
r	Refinement ration
S_i	Grid set
s	Number of implicit stages
T_i	Temporal set
t	Time
U_i	Stage velocity
U_{lid}	Lid velocity
u	Velocity component in direction x
u_{min}	Minimum $x-$velocity component at the $x-$centerline plane
V_c	Characteristic Velocity
v	Velocity component in direction y
v_{min}	Minimum $y-$velocity component at the $y-$centerline plane
v_{max}	Minimum $y-$velocity component at the $y-$centerline plane
w	Velocity component in direction z
x_i	Spatial coordinate
$\mathscr{E}$	Explicit terms
$\mathscr{I}$	Implicit terms
Δt^+	Time step increase
t^-_{wall}	Total wall time reduction

Nomenclature

This document adheres to a consistent notational convention to ensure clarity and prevent ambiguity. The following rules define the nomenclature used throughout this work.

Mathematical Typography

- **Vectors** are denoted by boldface Roman letters (e.g., velocity $\mathbf{V}$, position $\mathbf{x}$).
- **Matrices** are denoted by boldface letters (Roman or Greek) enclosed in square brackets (e.g., $[\mathbf{A}]$, $[\alpha]$).
- **Scalars** are represented by italicized letters (e.g., time t, density ρ).

Superscripts and Special Symbols

- The superscript $^{(n)}$ denotes a quantity at a specific time step n (e.g., $\mathbf{V}^{(n)}$ is the velocity at time $t^{(n)} = t^{(0)} + \Delta t_n$).
- A superscript asterisk $(^*)$ denotes a non-dimensional quantity (e.g., p^*). Exceptions are the well establish dimensionless numbers, such as the Reynolds number (Re).
- A superscript number sign $\left(^{\#}\right)$ denotes a non-conservative field.

Specific Symbol Definitions

- The symbol for volume $(\forbidden)$ is adopted from Munson et al. [1]. The uppercase V is reserved for the velocity field $\mathbf{V}$ of components u, v and w, respectively in x, y and z directions.

Chapter 1

Introduction

Fluid transport due to cilia movement is one of the most important phenomena in many physical and biological problems [2]. Cilia are slender, hair-like organelles that protrude from the plasma membrane of most eukaryotic cells and are built around a conserved "9 + 2" axonemal core of nine outer doublet microtubules surrounding a central pair, or "9 + 0" in primary cilia, together with dynein motor arms, radial spokes, and nexin links that generate sliding between adjacent doublets (see Luxmi and King [3] for a more complete definition). While cilia are generally involved in the transport of proteins, nutrients, or dust inside larger organisms, their functional importance extends across diverse biological processes. Figure 1.1 shows a schematic representation of the cilia given by Luxmi and King [3].

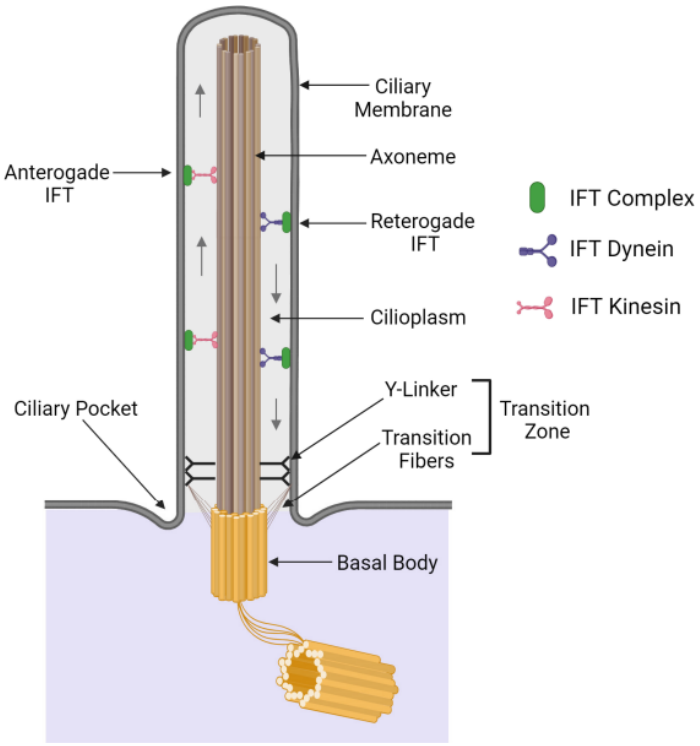

Figure 1.1: Generic schematic of 9+0 ciliary organization from [3].

The coordinated activity of these dynein arms produces a periodic, asymmetric beat that drives fluid transport at low Reynolds numbers, a mechanism essential for mucus clearance in the respiratory tract, cerebrospinal fluid circulation in the brain, and left–right patterning in embryos [4]. This cycle can be decomposed into two principal phases: an effective forward (power) stroke and a backward (recovery) stroke, depicted by Figure 1.2. During the forward stroke, the cilium extends nearly fully and sweeps stiffly in a directional arc, maximizing drag to propel the surrounding fluid. This is followed by the backward stroke, where the cilium bends sharply at its base and returns to its starting position close to the cell surface, minimizing retrograde flow by reducing its perpendicular profile. This asymmetry in form and drag between the two phases generates the net fluid transport. The precise coordination, or metachronal rhythm, between neighboring cilia — where the beat of each cilium is slightly phase-shifted relative to its neighbor — further enhances flow efficiency by creating traveling waves that propel fluid continuously along the epithelial surface. Numerical studies of the fluid–structure interaction between cilia and their surrounding fluid have been used to model their movement, as shown in the works of Chen et al. [5] and Decoene et al. [6].

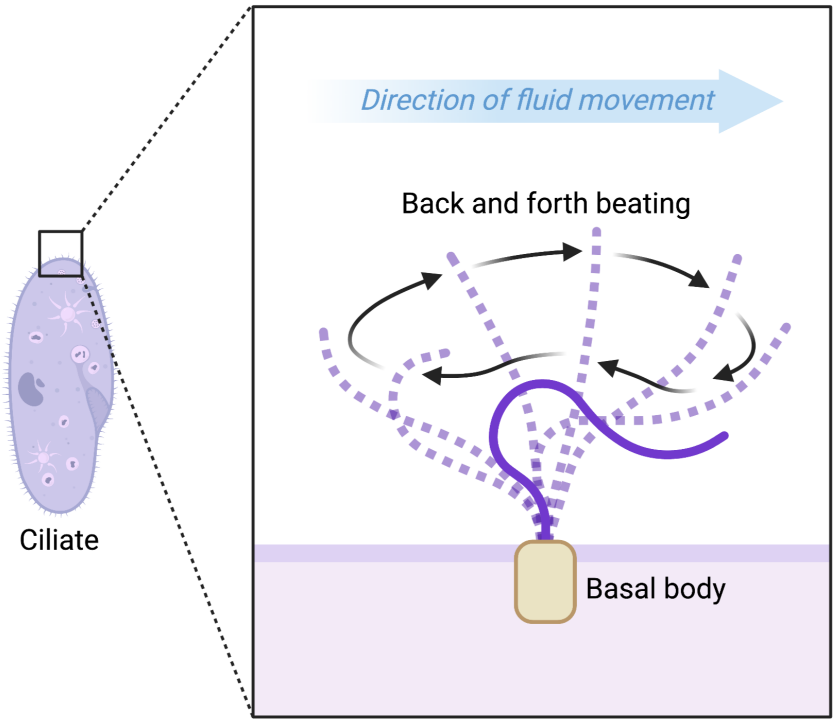

Figure 1.2: 1D scheme illustrating ciliary motion, which drives the overall movement of the organism. The cilia is fixed to the bottom plane, and the effective (forward) and recovery (backward) strokes are shown, from [7].

[]

1.1 Previous Studies

The study of ciliary flows has direct relevance to health. Yoshida et al. [8] studied the effect of ciliary disorders as a cause of hydrocephalus. They developed a cilia motility numerical model based on experimental data and applied it to a geometric model of the human ventricles, defined by medical imaging data. Their results demonstrated that reduced cilia motility does not significantly alter intraventricular pressure, however it severely impairs cerebrospinal fluid (CSF) circulation, preventing new CSF from reaching the anterior and inferior horns of the lateral ventricles. This indicates that ciliary motility disorders likely cause delayed CSF exchange in these specific ventricular regions.

In the same field, Salman et al. [9] studied how the tilt angle, quantity, and phase relationship of cilia affect CSF flow patterns in the brain ventricles of zebrafish embryos and defined the most suitable cilia configuration for transport efficiency. Their results showed that the cilium angle significantly alters the generated flow velocity and mass flow rates and that as the cilium angle gets closer to the wall, higher flow velocities are observed. In summary, the simulations revealed that the most efficient method for cilia-driven fluid transport relies on the alignment of multiple cilia beating with a phase difference.

Wuttanachamsri and Schreyer [10] in turn focused on modeling the cilia found in the human respiratory system. They set a benchmark study for the movement of fluid phases, based on a mixed finite element method of Taylor-Hood type. The results are validated in a simple case by comparison with an exact solution with good agreement.

Engineering applications exploit this biological principle in micro-fluidic devices, where artificial cilia fabricated from magnetic or polymeric materials mimic the natural beat to pump, mix, or manipulate small quantities of fluids in lab-on-a-chip platforms, for complete and detailed examples see [11].

1.2 Numerical Simulation: Low *Re* Flows

Because the characteristic length scales are too small for conventional experimental investigation, with Satir and Christensen [12] and Flaherty et al. [13] defining the cilia's height and diameter typically of order $\sim 10-15\,\mu$m and $\sim 0.25\,\mu$m, respectively, numerical simulations have become the primary tool for investigating cilia-driven flows [14]. The world of microorganisms is the world of low Reynolds number, a regime where inertia plays little role and viscous damping is paramount. In this context, there are several ways to define the Reynolds number, as stated by [15]. Since our case will only describe fixed cilia normal to the ground, the definition that suits the most is the classical one, where:

$$Re_{cil} = \frac{U_{cil} L_{cil}}{\nu}, \tag{1.1}$$

with U_{cil} the mean cilia velocity, L_{cil} the cilia height, and ν the kinematic viscosity of the surrounding fluid around. Depending on the type of cilia and its behavior, Re_{cil} can range from $\sim 10^{-5}$ to $\sim 10^{-1}$ [15].

In computational fluid dynamics (CFD), simulating such low Reynolds number flows presents a significant challenge. The primary constraint is the strict time-step limitation imposed by the viscous Courant-Friedrichs-Lewy (CFL) condition, which can generate impractically small time steps, rendering explicit time-integration schemes computationally infeasible. Consequently, a temporal scheme that is not restricted by this viscous stability limit is required.

Fully implicit methods eliminate time-step restrictions entirely; however, their implementation is complex due to the non-linear systems that must be solved at each step. Recently, Implicit-Explicit (IMEX) schemes have been applied in this context [16–18], offering a simpler yet effective alternative that balances computational efficiency with stability.

1.3 SP-Wind

The numerical simulation of cilia is performed using the in-house code *SP-Wind*, developed at KU Leuven. The code solves the three-dimensional, incompressible Navier–Stokes equations, presented in Section 2.1, and supports both Large Eddy Simulations (LES) and Direct Numerical Simulations (DNS). SP-Wind employs a pseudo-spectral discretization, which is well suited for turbulent flows in simple geometries and delivers high performance in simulations of wind farms within the atmospheric boundary layer, despite its limited flexibility with complex geometries [19]. The code applies a fourth-order spatial integration scheme and an explicit fourth-order Runge–Kutta (RK4) method for time integration, see [20] and Section 2.2.1 respectively.

A central challenge in airflow simulations is solving the Poisson equation for pressure. General-purpose codes often encounter significant performance issues when solving this equation on large-scale supercomputers, since it requires solving a large system of equations across many processors. SP-Wind addresses this efficiently by using Fast Fourier Transforms (FFT), significantly accelerating the computations. Additionally, it employs an LU decomposition method (described in Section 2.2.7), which provides an efficient strategy for solving linear systems.

From a technical perspective, SP-Wind is written in Fortran 90 and is parallelized using a three-dimensional domain decomposition of the computational grid. Inter-processor communication is handled through MPI with pencil decomposition for Fourier transforms as well as fringe-region forcing. Turbine models are integrated into the framework. The code depends on HDF5 and FFTW libraries, with input provided primarily through namelists and output stored in HDF5 format.

SP-Wind serves both as a research and production tool. As a research code, it is under continuous development, with frequent extensions and experimental implementations. Consequently, it is provided without guarantees, and the source code itself remains the ultimate reference. As a high-performance tool for production runs, however, it has been benchmarked and profiled, providing a level of assurance regarding its efficiency and correctness.

The code has been used for the optimal control of wind-farms using the adjoint-based optimization method [21, 22], channel flow simulation [23, 24] and many other applications. In this thesis, a new case study is implemented in *SP-Wind* by modifying the boundary conditions and interactive elements. The analysis shows that the viscous CFL condition dominates, imposing a strict time-step restriction that prevents low-cost simulations. To address this, a code is developed that applies an IMEX scheme as the time integrator, based on the formulation presented in Section 2.2.3.

Lastly, due to confidentiality reasons we cannot show any piece of *SP-Wind*'s code.

1.4 Thesis Objectives

The overarching goal of this thesis is to develop a numerical model in which a fixed and statical vertical cilia is placed between two fixed parallel plates and anchored to the bottom plate in a controlled environment. The work focuses on (i) implementing a simplified model for the cilia geometry in the *SP-Wind* code base, and (ii) designing, implementing, and validating a numerical code for time integration using an IMEX scheme. The objective is to assess whether increasing the time-step size through IMEX methods can compensate for the additional computational cost per step, which arises from solving an extra linear system. For the method to be viable, the gain in time-step size must be significant enough to reduce the overall wall-time and make simulations computationally feasible.

1.5 Thesis Outline

The thesis is organized into five chapters.

Chapter 2 – Methodology: This chapter presents the governing equations for Navier–Stokes and Stokes flows in both dimensional and non-dimensional form, adapted to the *SP-Wind* framework. The body-force term is introduced as the driving mechanism for approximating cilia geometry. Numerical integration methods, including Runge–Kutta schemes, are described with emphasis on their stability properties. Particular attention is given to the pressure projection method, as it constitutes a critical point in the simulations, together with the linear solvers required for both velocity and pressure fields. Fully implicit and IMEX schemes are introduced, motivating the shift away from explicit schemes limited by CFL constraints.

Chapter 3 – Numerical Validation: This chapter presents benchmark tests, including the 1D Burgers equation and the lid-driven cavity, used to validate the IMEX scheme. Results are compared with classical Runge–Kutta solutions. Convergence rates, physical accuracy, and time-step sensitivity are analyzed to assess the effectiveness and limitations of IMEX methods.

Chapter 4 – Application to 3D Cilia Flows: The validated IMEX approach is applied to three-dimensional cilia-driven flow. The case setup is shown to be consistent and to provide satisfactory solutions, but the classical Runge–Kutta scheme performs more robustly as a temporal integrator under current constraints. The challenges and computational trade-offs of applying IMEX schemes in 3D are discussed.

Chapter 5 – Conclusions and Future Work: The thesis concludes by summarizing the main findings and evaluating the potential of IMEX schemes for simulating cilia-driven low Reynolds number flows. Possible extensions and recommendations for future research are outlined.

Chapter 2

Methodology

This chapter presents the methodologies employed to model and simulate cilia-driven flows. We begin by defining the governing equations, the Navier–Stokes equations, with particular emphasis on the low-Reynolds-number regime, where the Stokes flow approximation becomes especially relevant.

The chapter then addresses the numerical procedures used for temporal integration. The family of Runge–Kutta methods is introduced in Section 2.2.1, while fully implicit methods are discussed in Section 2.2.2 as a theoretically optimal solution to time-step restrictions, although their non-linear character makes them computationally challenging to implement. IMEX schemes are presented in Section 2.2.3 as a practical compromise, where viscous terms are treated implicitly and non-linear terms explicitly. The stability analysis of these temporal schemes is divided into two parts: Section 2.2.4 examines the stability regions of each method and Section 2.2.5 derives the corresponding time-step restrictions, including the influence of the viscous CFL condition.

Finally, Section 2.2.6 examines the incompressibility constraint through the pressure projection step, which represents a critical component of the simulations. Both direct and iterative linear solvers applied to the velocity and pressure fields are presented in Section 2.2.7, with a discussion of their efficiency and scalability for large, sparse systems.

2.1 Governing Equations

In engineering, it is of interest to describe how fluids behave under various conditions to better understand and predict their flow. For this purpose, a set of mathematical equations, known as Navier-Stokes equations, arise as the backbone of fluid dynamics, combining conservation of mass and momentum.

In general, the incompressible Navier-Stokes equations for Newtonian fluids are defined as

[25]:

$$\frac{\partial \mathbf{V}}{\partial t} + \mathbf{V} \cdot \nabla \mathbf{V} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{V} + \mathbf{g} + \mathbf{f_D}\,, \tag{2.1}$$

$$\nabla \cdot \mathbf{V} = 0\,. \tag{2.2}$$

where Equations (2.1) and (2.2) describe the momentum and mass balance, respectively. There, $\mathbf{V}$ and p are respectively the velocity and pressure fields, ρ the density, $\mathbf{g}$ the gravitational acceleration vector, and $\mathbf{f_D}$ the body force exerted by the cilia motion, see Section 2.1.2.

Often, there is an interest in non-dimensionalizing the Navier-Stokes equations to allow for the comparison of the relative magnitudes of different terms within the equations. To achieve this, it is first necessary to define appropriate scaling parameters, as listed in Table 2.1.

Table 2.1: Scaling parameters used to non-dimensionalize the momentum and continuity equations, together with their primary dimensions. {M} denotes mass, {L} length, and {T} time; together they form the primary dimensional group {MLT}.

Scaling Parameter	Description	Primary Dimensions
L_c	Characteristic length	$\{L\}$
V_c	Characteristic speed	$\{LT^{-1}\}$
f	Characteristic frequency	$\{T^{-1}\}$
$p_0 - p_\infty$	Reference pressure difference	$\{ML^{-1}T^{-2}\}$
g	Gravitational acceleration	$\{LT^{-2}\}$

Then, we define the dimensionless variables as follows,

$$t^* = ft; \quad \mathbf{x}^* = \frac{\mathbf{x}}{L_c}; \quad \mathbf{V}^* = \frac{\mathbf{V}}{V_c}; \tag{2.3}$$

$$p^* = \frac{p - p_\infty}{p_0 - p_\infty}; \quad \mathbf{g}^* = \frac{\mathbf{g}}{g}; \quad \nabla^* = L_c \nabla. \tag{2.4}$$

The non-dimensional Navier-Stokes equations are then written as follows,

$$[St] \frac{\partial \mathbf{V}^*}{\partial t^*} + (\mathbf{V}^* \cdot \nabla^*) \mathbf{V}^* = -[Eu] \nabla^* p^* + \left[\frac{1}{Fr^2}\right] \mathbf{g}^* + \left[\frac{1}{Re}\right] \nabla^{*2} \mathbf{V}^* + \mathbf{f_D}^*\,. \tag{2.5}$$

The dimensionless numbers are defined as follows [1]:

1. Strouhal number: This dimensionless parameter quantifies the ratio of inertial forces arising from flow unsteadiness (local acceleration) to those associated with velocity variations in the flow field (convective acceleration). It is defined as

$$[St] = \frac{f L_c}{V_c}\,. \tag{2.6}$$

2. Euler number: This dimensionless number quantifies the ratio between pressure forces and inertia forces. It is given by

$$[Eu] = \frac{p_0 - p_\infty}{\rho V_c^2}. \tag{2.7}$$

3. Froude number: This dimensionless number quantifies the ratio of inertial forces to gravitational forces. It is defined as

$$[Fr] = \frac{V_c}{\sqrt{gL_c}}. \tag{2.8}$$

4. Reynolds number: This dimensionless number quantifies the ratio between inertial forces and viscous forces. It is defined as

$$[Re] = \frac{V_c L_c}{\nu}. \tag{2.9}$$

To remain consistent with the in-house simulation code SP-Wind used in this thesis (see Section 1.3), the density is set to unity ($\rho = 1$). The reference velocity and length scales, (V_c) and (L_c), are defined such that $\nu = 1/Re$. The remaining characteristic quantities are chosen so that the Strouhal, Euler, and Froude numbers do not appear in the equations. The resulting form of the Navier–Stokes equations applied in this thesis is

$$\frac{\partial \mathbf{V}^*}{\partial t^*} + \mathbf{V}^* \cdot \nabla \mathbf{V}^* = -\nabla p^* + \left[\frac{1}{Re}\right] \nabla^2 \mathbf{V}^* + \mathbf{g}^* + \mathbf{f}_\mathbf{D}^*, \tag{2.10}$$

$$\nabla \cdot \mathbf{V}^* = 0. \tag{2.11}$$

2.1.1 Stokes Flow Formulation

Besides Navier-Stokes, this thesis also focuses on Stokes flow, also referred to as creeping flow, which is a subset of laminar flows occuring at very low Reynolds numbers ($Re \ll 1$). In practical terms, this corresponds to either a flow with relatively low fluid density (ρ), relatively low characteristic velocity (V_c), or relatively small characteristic length scale (L_c), or alternatively, from relative high viscosity, or any combination of these factors [26]. This regime was first investigated by Stokes [27], who demonstrated that under such conditions, viscous forces dominate inertial effects. As a result, the convective acceleration term in the Navier–Stokes equations, represented by $\mathbf{V} \cdot \nabla \mathbf{V}$, becomes negligible. This simplification yields the governing form of the Stokes flow equation:

$$\frac{\partial \mathbf{V}}{\partial t} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{V} + \mathbf{f}_\mathbf{D}, \tag{2.12}$$

$$\nabla \cdot \mathbf{V} = 0. \tag{2.13}$$

In this limit, inertial forces become negligible compared to viscous forces, leading to a flow that is entirely governed by viscosity. This results in a predictable motion without flow separation

or turbulence, a hallmark of creeping flow. These features are illustrated in Figure 2.1, which depicts a representative example of a creeping flow around a cylindrical object, highlighting the smooth and symmetric streamlines that typical occur in this regime. To further understand the

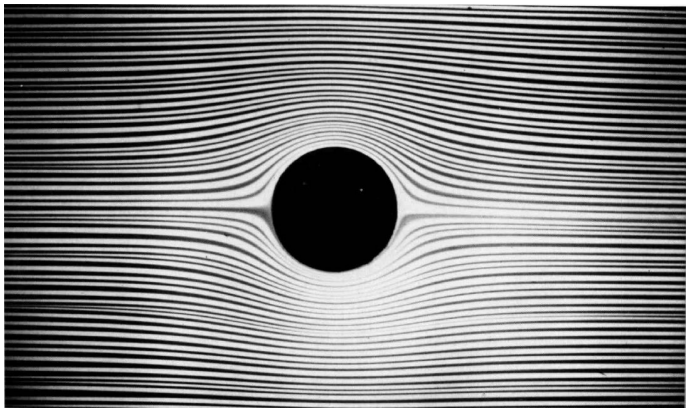

Figure 2.1: Visualization of Stokes flow past a circular cylinder confined between glass plates. The dye traces the streamlines in water flowing at 1 mm/s. Photograph by D. H. Peregrine (1982).

physics of Stokes flow, an additional non-dimensional analysis is conducted. However, this time, as considered by [28], the pressure cannot be scaled with density as this last parameter is considered to be zero in the Stokes flow approximation, leading to a different choice of characteristic groups. In [28] the following non-dimensional quantities are proposed:

$$t^* = \frac{tV_c}{L_c}, \qquad \mathbf{x}^* = \frac{\mathbf{x}}{L_c}, \qquad \mathbf{V}^* = \frac{\mathbf{V}}{V_c}, \qquad p^* = \frac{p - p_\infty}{\mu V_c / L_c}, \qquad \nabla^* = L_c \nabla. \tag{2.14}$$

Substituting these non-dimensional variables into the Stokes equations leads to the dimensionless form of the unsteady Stokes equation:

$$[Re]\,\frac{\partial \mathbf{V}^*}{\partial t^*} = -\nabla p^* + \nabla^{*2}\mathbf{V}^*. \tag{2.15}$$

As the Reynolds number approaches zero, the inertial term on the left-hand side vanishes entirely, yielding a steady-state form of the momentum equation known as the Stokes condition:

$$Re \to 0: \quad \nabla p^* = \nabla^{*2}\mathbf{V}^*. \tag{2.16}$$

For small but finite Reynolds numbers, this approximation remains valid, leading to the general assumption used for creeping flow:

$$Re \ll 1: \quad \nabla p^* \approx \nabla^{*2}\mathbf{V}^*. \tag{2.17}$$

Another particular characteristic of the creeping flow assumption is that its application is not restricted to Reynolds numbers less than unity. Although its definition arose from this procedure, in practice, the consideration of Stokes regime can always be done when the advective term can be

neglected, just as in the case of a fully developed duct flow [28].

The cilia are modeled as cylinders for simplicity. In the equations, the associated geometric parameters are expressed in dimensionless form. The non-dimensional cilia height h^*_{cil} and diameter D^*_{cil} are defined as the respective dimensional quantities normalized by L_c

2.1.2 Cilia Modeling: Stokes Flow Past an Immersed Cylinder

The drag force acting on the cilia, $\mathbf{f}^*_{\mathbf{D}}$, is modeled as a non-dimensional source acting on the fluid, meaning that no explicit no-slip condition is enforced on the cilia boundary; this will therefore be referred to as a body force. To represent the cilia geometry, we approximate it as a cylinder, an assumption commonly adopted in the literature [9, 29, 30].

Moreover, its the bodyforce is based on the approach from Meyers and Meneveau [31]. The total non-dimensional body force is obtained by integrating the non-dimensional force per unit volume over the height of the cilia, as follows:

$$\mathbf{f}^*_{\mathbf{D}} = \int_0^{h^*_{cil}} \frac{C_D\left(\overline{Re}_D\right) A^*_p}{2\forall^*} \left|\overline{\mathbf{U}}^*\right|^2 \hat{\mathbf{e}}_{\mathbf{D}}\, dz^*. \tag{2.18}$$

Here, C_D is the drag coefficient, a function of the Reynolds number based on the cilia's diameter and the mean non-dimensional velocity magnitude $\left|\overline{\mathbf{U}}^*\right|$. The term $\hat{\mathbf{e}}_{\mathbf{D}}$ is a unit vector that ensures that the drag force acts in the direction opposite to the flow, i.e. $\mathbf{U}^*_{\mathbf{i}}/\left|\mathbf{U}^*_{\mathbf{i}}\right|$. In addition, the projected area is set as $D^*_{cil}\, z^*$ and the volume is defined as $Ap^*\, dx$.

For the numerical simulation, this continuous formulation is projected onto the computational grid. The non-dimensional body force is calculated for each segment of the cilia, z_i, as:

$$\mathbf{f}^*_{\mathbf{D}} = \sum_{k=1}^{N_{z,cil}} \mathbf{f}_{\mathbf{D},k}^* = \sum_{i=1}^{N_{z,cil}} \frac{1}{2} C_D\left(\overline{Re}_{D,k}\right) \left|\overline{\mathbf{U}}^*_k\right|^2 \hat{\mathbf{e}}_{\mathbf{D}}. \tag{2.19}$$

The mean velocity $\overline{\mathbf{U}}^*_i$ is calculated by applying a Gaussian filter, $\mathrm{G}\left(\mathbf{x}_j\right)$, to the local fluid velocity, $\mathbf{V}^*_j$, at every cell in a horizontal plane at location z_i:

$$\overline{\mathbf{U}}^*_k = \sum_{i=1}^{N_{elm,x}} \sum_{j=1}^{N_{elm,y}} \mathbf{V}^*_{i,j,k}\, \mathrm{G}\left(\mathbf{x}_{i,j,k}\right), \quad k = 1, ..., N_{z,cil}\,, \tag{2.20}$$

where $N_{z,cil}$ denotes the number of elements defined along the cilium height.

Extensive experimental data on the drag coefficient of immersed cylinders exist across a wide range of Reynolds numbers [32]. Based on these data, a reference curve for smooth cylinders was established by Munson et al. [1]. To provide mathematical descriptions of the dependence of C_D on Re, several empirical formulas were also proposed, each valid within specific Reynolds number ranges.

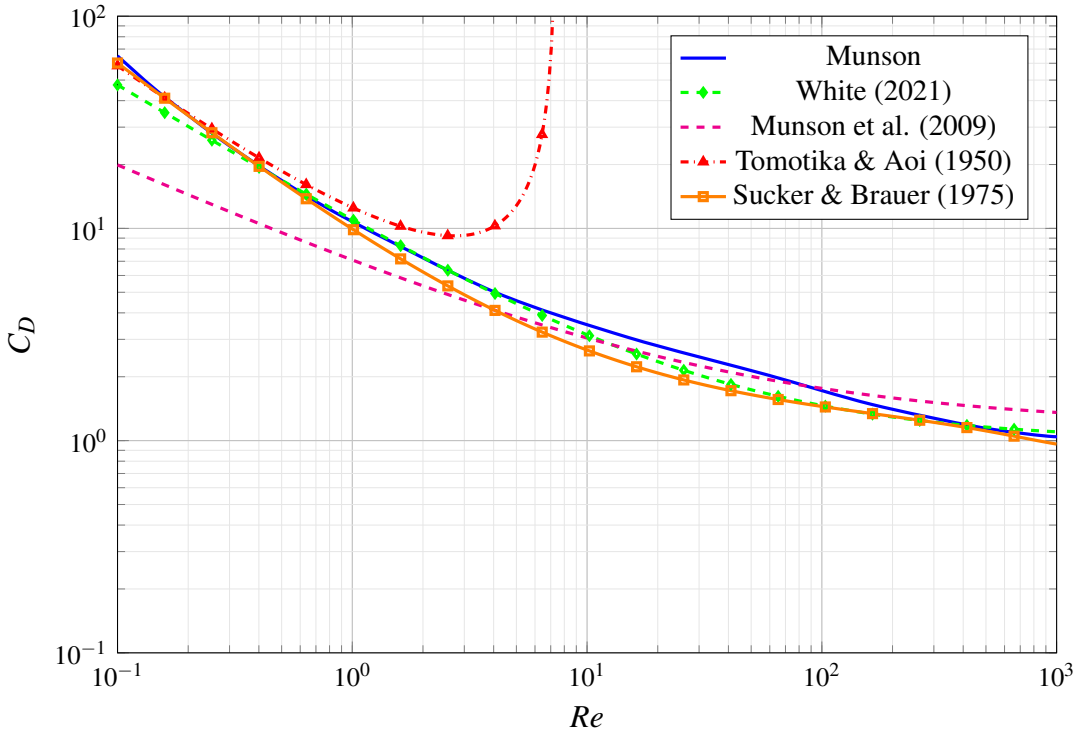

Figure 2.2: Drag coefficient, C_D, as a function of Reynolds number $\left(\overline{Re}_D = \overline{U}D/\nu\right)$ for a smooth cylinder.

The most commonly used drag relations are:

- White [28] obtained a curve fit formula, based on the data from Tritton [33] and Wieselsberger [34], which is reasonable up to $Re < 2.5 \times 10^5$:

$$C_D = 1 + \frac{10}{Re_D^{2/3}}. \tag{2.21}$$

- Munson et al. [1] suggests a formula which assumes the flow's boundary layer separation to occur at the angle of 130°, but the results are considered rough:

$$C_D = 1.17 + \frac{5.93}{\sqrt{Re_D}}. \tag{2.22}$$

- Tomotika and Aoi [35] show an asymptotic expression for low Re_D, that diverges when Re_D tends to 1:

$$C_D = \frac{8\pi}{Re_D\left(0.5 - \gamma_e + \ln\left(8/Re_D\right)\right)} \tag{2.23}$$

  . Here, γ_e is the Euler–Mascheroni constant approximately equal to 0.57721.

- Sucker and Brauer [36] obtained a curve fit formula that shows good accuracy for $10^{-4} < Re_D < 10^5$:

$$C_D = 1.18 + \frac{6.8}{Re_D^{0.89}} + \frac{1.96}{\sqrt{Re_D}} - \frac{4 \times 10^{-4} \times Re_D}{1 + 3.63 \times 10^{-7} \times Re_D^2}. \tag{2.24}$$

Based on the curves shown in Figure 2.2, the Sucker and Brauer [36] model shows the highest reliability compared to the others; therefore, their formulation will be adopted in the remainder of this work.

2.2 Numerical Procedure

Solving both the Navier-Stokes and Stokes equations analytically can be humanly challenging or even impossible, depending on the flow type and conditions. A common approach is to compute a numerical solution, by discretizing the governing equations over the spatial domain, and advancing them in time step by step, thereby simulating the flow. However, performing those steps can be computationally demanding, mainly due to the time-step restriction (see Section2.2.5).

To that end, we developed an IMEX scheme to mitigate the CFL restriction. First, Section 2.2.1 presents the family of explicit Runge-Kutta (ERK) schemes, which are the basis for the selected solution approach and provide a comparative benchmark for the applied method. Next, Section 2.2.2 addresses implicit Runge-Kutta (IRK) schemes. This section outlines their properties as a desired solution class, due to its unrestricted stable time-step, and explains the rationale for not employing them in this work. Subsequently, in Section 2.2.3 we enter the core of this thesis, where implicit-explicit linear multi-step Runge-Kutta schemes (IMEX) are presented and justified as the selected solution. The discussion begins with a relevant historical background before detailing their formulation applied to a fluid mechanics problem. Furthermore, Butcher tables are presented for second- and third-order IMEX methods. Although these methods are lower-order than the classical fourth-order Runge-Kutta (RK4) method, they offer significantly enhanced stability with respect to the viscous time-step restriction, a fact demonstrated by an analysis of their stability regions (see Section 2.2.4). In addition, particular attention is given to the pressure projection step, as its implementation can be non-trivial (see Section 2.2.6). Lastly, Section 2.2.7 describes the set of suitable linear solvers employed in this work to solve the discretized governing equations.

2.2.1 Explicit Runge-Kutta Schemes (ERK)

In the context of operator splitting, an $s-$stage method proceeds through s successive Euler steps, each associated with a time increment Δt_i and a specific directional estimate or slope of the field's $\mathbf{y}$ rate of change, denoted as $\mathbf{K}_i$. Butcher [37] proposed a notation that simplifies the representation of multi-step methods. A general RK method is composed of three parameters, commonly called the Runge-Kutta matrix $[\mathbf{A}]$, the weights $\mathbf{b}$, and the node vectors $\mathbf{c}$. Their values are stored in a mnemonic device, known as a Butcher tableau, of type:

Table 2.2: General form for a Butcher Tableau

$\mathbf{c}$	$[\mathbf{A}]$
	$\mathbf{b}$

Definition 2.2.1 (Explicit Runge-Kutta Method (ERK))**.** *Let $s \in \mathbb{N}$, $a_{i,j} \in \mathbb{R}$, $1 \leq j \leq i \leq s$, $b_i \in \mathbb{R}$ for $1 \leq i \leq s$, and $c_i = \sum_{j=1}^{i-1} a_{i,j}$. Consider the problem,*

$$\frac{\partial \mathbf{y}}{\partial t} = \mathscr{F}(t, \mathbf{y}), \tag{2.25}$$

with initial condition,

$$\mathbf{y}\left(t^{(0)}, \mathbf{y}^{(0)}\right), \tag{2.26}$$

The scheme,

$$\mathbf{K}_1 = \mathscr{F}\left(t^{(n)} + \Delta t c_1, \mathbf{y}^{(n)}\right), \tag{2.27}$$

$$\mathbf{K}_i = \mathscr{F}\left(t^{(n)} + \Delta t\, c_i,\, \mathbf{y}^{(n)} + \Delta t \sum_{j=1}^{i-1} a_{i,j}\mathbf{K}_j\right), \quad \text{for } i = 2, ..., s, \tag{2.28}$$

$$\mathbf{y}^{(n+1)} = \mathbf{y}^{(n)} + \Delta t \sum_{i=1}^{s} b_i \mathbf{K}_i, \tag{2.29}$$

has the name of s-stage explicit Runge-Kutta method.

Therefore, there are two constraints that dictate both the viability and the order of these types of methods.

Definition 2.2.2. *Let $s \in \mathbb{N}$, $a_{i,j} \in \mathbb{R}$, $1 \leq j \leq i \leq s$, $b_i \in \mathbb{R}$. Applying Taylor series expansions to Equation (2.25), a Runge–Kutta method is consistent if and only if,*

$$\sum_{i=1}^{s-1} b_i = 1. \tag{2.30}$$

Definition 2.2.3. *Let $s \in \mathbb{N}$, $a_{i,j} \in \mathbb{R}$, $1 \leq j \leq i \leq s$ and $b_i \in \mathbb{R}$ for $1 \leq j \leq s$. For a certain method to have an order $\hat{p} \in \mathbb{N}$ and local truncation error $\mathscr{O}\left(\Delta t^{\hat{p}+1}\right)$,*

$$\sum_{j=1}^{i-1} a_{ij} = c_i, \quad \text{for } i = 1, 2, ..., s. \tag{2.31}$$

This last definition alone is neither sufficient nor necessary for consistency [38]. Furthermore, Butcher [39] proven that, in general, for an explicit s-stage Runge-Kutta method to achieve order $\hat{p}$, the number of stages must satisfy $s \geq \hat{p}$, and if $\hat{p} \geq 5$, then $s \geq \hat{p} + 1$. However, it is not known whether these bounds are sharp in all cases. In some instances, it has been proven that the bounds cannot be reached. For example, Butcher showed that for $\hat{p} > 6$, there is no explicit RK method with $s = \hat{p} + 1$, and for $\hat{p} > 7$, there is no explicit RK method exists with $s = \hat{p} + 2$. In general, the precise minimum number of stages s required for an explicit Runge–Kutta method to achieve order $\hat{p}$ remains an open problem.

In most engineering applications, an ERK method of $4-$ stages ($4^{th}-$ order) is used. This method is also known by the name Classical Runge-Kutta or RK4, and is the time integration method employed in *SP-Wind*. Table 2.3 shows the Butcher tableau for RK4.

Table 2.3: Butcher tableau for the Classical Runge-Kutta

0	0	0	0	0
1/2	1/2	0	0	0
1/2	0	1/2	0	0
1	0	0	1	0
	1/6	1/3	1/3	1/6

Since its Butcher tableau has all the elements below the lower sub-diagonal equal to zero, the method is said to be low-storage. In other words, the computation of the i^{th} slope variable does not require storing the previously computed slopes.

The Runge-Kutta time integration method is traditionally formulated to solve systems of Ordinary Differential Equations (ODEs). In the context of Navier-Stokes equations, special care should be taken in formulating the pressure correction (see Section 2.2.6). In the following, we explain how the pressure correction step is included in the ERK defined above. The description is consistent with the implementation in both *SP-Wind* and the cavity case study, which will be presented later in this work. To accommodate the ERK scheme in this context, the momentum equation is first recast as:

$$\frac{\partial \mathbf{V}^*}{\partial t} = \mathrm{RHS}\left(\mathbf{V}^*\right) - \nabla p^*, \tag{2.32}$$

where $\mathrm{RHS}\left(\mathbf{V}^*\right)$ contains all the terms on the right-hand side of the momentum equation (2.10), except the pressure gradient. Two distinct approaches were followed to implement the pressure projection in Runge-Kutta methods, each corresponding to the cavity case and to *SP-Wind*.

1. **Single Projection Method:** This formulation computes the velocity and corresponding slope variable $\mathbf{K}_i^*$ at each stage while excluding the pressure term. In this approach, $\mathbf{K}_i^*$ expresses only the right-hand side operator, $\mathrm{RHS}\{\mathbf{V}^*\}$, with the pressure projection performed exclusively at the advancing step (see Equation (2.29)). This methodology offers computational efficiency by minimizing the number of pressure solves required and is the one applied in the cavity case study;

2. **Stage-wise Projection Method:** This approach applies pressure projection at every intermediate stage to enforce continuous incompressibility. Here, $\mathbf{K}_i^*$ incorporates the complete right-hand side of the Navier-Stokes equations, including pressure gradient terms. This method is only practical when employing a direct solver for the pressure-Poisson equation. Consequently, the final solution $\mathbf{V}^{*(n+1)}$, which is a linear combination of divergence-free fields, inherently satisfies the incompressibility constraint without requiring a final projection in the advancing step. This approach was followed in *SP-Wind*.

Furthermore, the stage-wise projection method is unsuitable for iterative solvers. When applying this method, we must correct the stages velocities, rather than the slope variables. The motivation behind this statement is that the inherent tolerance of iterative solvers introduces small but cumulative errors in the divergence constraint for the velocities if the pressure-Poisson equation is solved in its closed form,

$$\nabla^{*2} p_i^* = \nabla^* \cdot \mathbf{K}_i^{*\#}, \tag{2.33}$$

with $\mathbf{K}_i^{*\#}$ expressing the non-conservative slope variable, so that

$$\mathbf{K}_i^* = \mathbf{K}_i^{*\#} - \nabla^* p_i^* . \tag{2.34}$$

Therefore, $\mathbf{K}_i^*$ would be considered numerically divergence-free, but

$$\mathbf{U}_\mathbf{i}^* = \mathbf{V}^{*(\mathbf{n})} + \Delta t^* \mathbf{K}_i^* , \tag{2.35}$$

will diffuse the tolerance error in

$$\mathbf{V}^{*(n+1)} = \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \mathbf{K}_i^* , \tag{2.36}$$

making the divergence of the final velocity field, past some iterations, orders of magnitude higher this same tolerance. This behavior is worst with grid refinement, as we saw in the cavity case study. To illustrate this behavior, Figure 2.3 shows the influence of the pressure projection method on the velocity divergence field, when applied to the $2D$ cavity case (the numerical setup for this case will be discussed later in Section 4.2). In the figure on the left, the stage-wise projection method is performed leading to a divergence field error of order one. In the figure on the right, a single projection method is employed, leading to a numerical divergence-free velocity field.

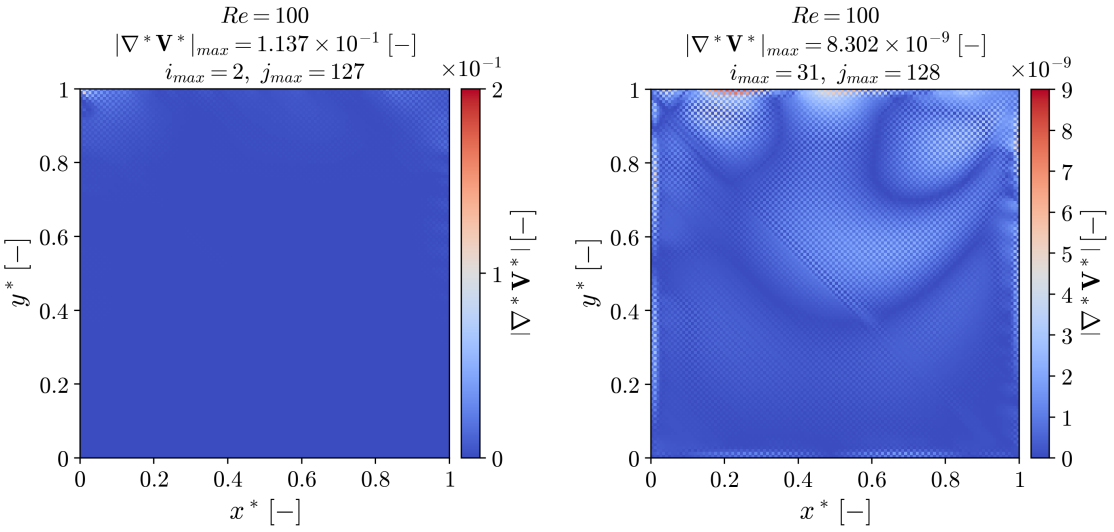

Figure 2.3: Influence of the pressure projection correction method on the divergence of the velocity field.

Both pressure projection methods discussed above are implemented algorithmically as outlined in Algorithm 1, with the distinction between iterative and direct solvers handled through conditional branching.

Algorithm 1 Explicit Low Storage $s-$stage Runge-Kutta

procedure RK4($\mathbf{V}^{*(n)}, p^{*(n)}, t^*$)
  if ProjectionMethod == Iterative **then**
    $\mathbf{K}_1^{*\#} \leftarrow \text{RHS}(\mathbf{V}^{*(n)})$
    $\mathbf{U}_2^{*\#} \leftarrow \mathbf{V}^{*(n)} + \frac{\Delta t^*}{2}\mathbf{K}_1^*$
    for $i = 2$ to s **do**
      $\mathbf{K}_i^{*\#} \leftarrow \text{RHS}(\mathbf{U}_i^{*\#})$
      $\mathbf{U}_{i+1}^{*\#} \leftarrow \mathbf{V}^{*(n)} + a_{i,i-1}\Delta t^*\mathbf{K}_i^*$
    end for
    $\mathbf{V}^{*\#} \leftarrow \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \mathbf{K}_1^{*\#}$
    $p^{*(n+1)} \leftarrow solve\left[\left[\nabla_n^{*2}\right] p^{*(n+1)} = \frac{\nabla^* \cdot \mathbf{V}^{*\#}}{\Delta t^*}\right]$
    $\mathbf{V}^{*(n+1)} \leftarrow \mathbf{V}^{*\#} - \Delta t^* \nabla^* p^{*(n+1)}$
  else if ProjectionMethod == Direct **then**
    $\mathbf{K}_1^{*\#} \leftarrow \text{RHS}\left(\mathbf{V}^{*(n)}\right)$
    $p_1^* \leftarrow solve\left[\left[\nabla_n^{*2}\right] p_1^* = \nabla^* \cdot \mathbf{K}_1^{*\#}\right]$
    $\mathbf{K}_1^* \leftarrow \mathbf{K}_1^{*\#} - \nabla^* p_1^*$
    $\mathbf{U}_2^* \leftarrow \mathbf{V}^{*(n)} + a_{i,i-1}\mathbf{K}_1^*$
    for $i = 2$ to s **do**
      $\mathbf{K}_i^{*\#} \leftarrow \text{RHS}\left(\mathbf{U}_i^*\right)$
      $p_i^* \leftarrow solve\left[\left[\nabla_n^{*2}\right] p_i^* = \nabla^* \cdot \mathbf{K}_i^{*\#}\right]$
      $\mathbf{K}_i^* \leftarrow \mathbf{K}_i^{*\#} - \nabla^* p_i^*$
      $\mathbf{U}_{i+1}^* \leftarrow \mathbf{V}^{*(n)} + a_{i,i-1}\mathbf{K}_i^*$
    end for
    $\mathbf{V}^{*(n+1)} \leftarrow \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \mathbf{K}_1^*$
  end if
end procedure

2.2.2 Implicit Runge-Kutta (IRK)

The implicit Runge-Kutta (IRK) scheme is based on the same fundamentals as the explicit formulation; however, it differs in the way it defines the slope parameters. In the implicit case, the summation domain is extended from $i-1$ to s, where s denotes the total number of RK stages.

Definition 2.2.4 (Implicit Runge-Kutta Method (IRK))**.** *Let $s \in \mathbb{N}$, $a_{i,j} \in \mathbb{R}$, $1 \leq j \leq i \leq s$, $b_i \in \mathbb{R}$ for $1 \leq j \leq s$, and $c_i = \sum_{j=1}^{s} a_{i,j}$. Consider the problem,*

$$\frac{\partial \mathbf{y}}{\partial t} = \mathscr{F}(t, \mathbf{y}), \tag{2.37}$$

with initial condition,

$$\mathbf{y}\left(t^{(0)}, y^{(0)}\right), \tag{2.38}$$

where $\mathscr{F}(t,y)$ represents the right-hand side of the differential equation (i.e., the time derivative of y as a function of t and y) The scheme,

$$K_i = \mathscr{F}\left(t + \Delta t\, c_i,\ \mathbf{y}^{(n)} + \Delta t \sum_{j=1}^{s} a_{ij} \cdot K_j\right), \quad \text{for } i = 2, \ldots, s, \tag{2.39}$$

$$\mathbf{y}^{(n+1)} = \mathbf{y}^{(n)} + \Delta t \sum_{i=1}^{s} b_i K_i, \quad \text{for } i = 1, \ldots, s. \tag{2.40}$$

has the name of s-stage implicit Runge-Kutta scheme.

In this case, $\mathbf{K}_i$ is a function of all other stages $\mathbf{K}_j$, $j = 1, \ldots, s$, so all $\mathbf{K}_i$ must be solved simultaneously as a heavy system. This formulation reflects how the future intermediate stage is influenced not only by its current value but also by an integrated history of its response to governing forces over the course of a full time step and the further future intermediate stages. The implicit nature of the scheme allows for greater numerical stability, disabling the dependence on a time-step restriction.

There are multiple possibilities for the composition of both the matrix $[\mathbf{A}]$ and the weight vector $\mathbf{b}$. The simplest case is to consider a matrix and weight vector consisting of a single element, both equal to one, leading to the Backward Euler method, a first-order implicit integration scheme. Additionally, several other well-known implicit time integration schemes can be derived using the general Runge-Kutta formulation. Some examples are the Crank-Nicolson (see Table 2.4, second-order accurate) and Gauss–Legendre (see Table 2.5, fourth-order accurate) schemes.

Table 2.4: Butcher tableau for the Crank-Nicolson scheme

0	0	0
1	1/2	1/2
	1/2	1/2

Table 2.5: Butcher tableau for the Gauss–Legendre scheme

$\frac{1}{2}-\frac{\sqrt{3}}{6}$	$\frac{1}{4}$	$\frac{1}{4}-\frac{\sqrt{3}}{6}$
$\frac{1}{2}+\frac{\sqrt{3}}{6}$	$\frac{1}{4}+\frac{\sqrt{3}}{6}$	$\frac{1}{4}$
	$\frac{1}{2}$	$\frac{1}{2}$

Furthermore, Diagonally Implicit Runge–Kutta (DIRK) methods are widely employed for numerical integration of stiff initial value problems [40]. A key advantage of this class of methods lies in the structure of the coefficient matrix $[\mathbf{A}]$, which is lower triangular. This property allows the system of equations to be solved sequentially rather than simultaneously, which means that each K_i depends only on itself and the stage values previously computed K_r for $r < i$. This significantly reduces computational complexity and makes the scheme more feasible for numerical implementation, especially in large-scale or stiff systems. Examples of these schemes are Crouzeix's three-stage scheme with $\alpha = \frac{2}{\sqrt{3}}\cos\frac{\pi}{18}$ (see Table 2.6, a 4th order Diagonally Implicit Runge–Kutta scheme) and the Nørsett's three-stage scheme (see Table 2.7, a 4th order Diagonally Implicit Runge–Kutta scheme) where x is one of the three roots of the cubic equation,

$$x^3 - \frac{3x^2}{2} + \frac{x}{2} - \frac{1}{24} = 0. \tag{2.41}$$

Table 2.6: Butcher tableau for the Crouzeix's three-stage scheme

$\frac{1+\alpha}{2}$	$\frac{1+\alpha}{2}$	0	0
$\frac{1}{2}$	$-\frac{\alpha}{2}$	$\frac{1+\alpha}{2}$	0
$\frac{1-\alpha}{2}$	$1+\alpha$	$-(1+2\alpha)$	$\frac{1+\alpha}{2}$
	$\frac{1}{6\alpha^2}$	$1-\frac{1}{3\alpha^2}$	$\frac{1}{6\alpha^2}$

Table 2.7: Butcher tableau for the Nørsett's three-stage scheme

x	x	0	0
$1/2$	$1/2-x$	x	0
$1-x$	$2x$	$1-4x$	x
	$\frac{1}{6(1-2x)^2}$	$\frac{3(1-2x)^2-1}{3(1-2x)^2}$	$\frac{1}{6(1-2x)^2}$

Fully implicit methods, even in their diagonally implicit form, require solving a non-linear system at each time step. Given the time constraints of this thesis, an alternative approach is considered, presented in Section 2.2.3.

2.2.3 Implicit-Explicit Linear Multi-step Runge-Kutta (IMEX-RK)

To overcome the challenges for IRK methods outlined above, a Diagonally Implicit-Explicit (DIMEX) linear multi-step time-discretization scheme is employed. This approach treats the non-linear terms explicitly while maintaining the linear viscous and pressure terms implicitly, thereby reducing computational complexity without sacrificing stability.

In practical terms, the discretized Navier–Stokes equations are decomposed into a discrete implicit Stokes operator, viscous plus pressure terms, and an explicit contribution related to the advection term and potential body forces (e.g., those associated with the cilia motion).

Moreover, IMEX schemes have been extensively used in the literature, particularly alongside spectral methods [41]. These schemes have demonstrated successful application not only to the incompressible Navier–Stokes equations [42], but also to their compressible counterparts [43].

A significant contribution to the development and analysis of IMEX schemes comes from the group of Prof. Uri M. Ascher. In particular, the study presented in [44] offers a comprehensive analysis of multistep IMEX methods, highlighting their stability behavior and performance in convection-diffusion and reaction-diffusion problems. The authors propose improved schemes that mitigate issues related to high-frequency error propagation and aliasing, particularly when spectral collocation methods or multigrid solvers are used, by selecting schemes with stronger damping properties. The findings show that, for this class of problems, the use of weak damping schemes, such as the popular Crank-Nicolson and second-order Adams-Bashforth combination, is discouraged, and more robust alternatives are proposed. Moreover, the study explores a wide variety of combinations for the implicit-explicit partitioning, resulting in IMEX schemes of the second to fourth order, depending on the chosen formulation.

Based on these insights, the same authors further advanced the field in a subsequent work [45], where they introduced multi-step (Runge-Kutta-based) IMEX schemes with enhanced stability properties. Unlike traditional IMEX schemes, which can suffer from restrictive time-step limitations unless diffusion dominates, the Runge-Kutta formulations presented in their study offer broader stability regions across a wider range of problem parameters.

Having the referred works as a starting point, the governing equations can then be decomposed into implicit and explicit blocks, respectively $\mathscr{I}$ and $\mathscr{E}$, as follows:

$$\frac{\partial \mathbf{V}^*}{\partial t} = \mathscr{I}\left(\mathbf{V}^*, p^*\right) + \mathscr{E}\left(\mathbf{V}^*, \mathbf{f}_\mathbf{D}^*\right), \tag{2.42}$$

where

$$\mathscr{E} = -\mathbf{V}^* \cdot \nabla \mathbf{V}^* + \mathbf{f}_\mathbf{D}^*\left(\mathbf{V}^*\right), \tag{2.43}$$

and

$$\mathscr{I} = \nu \nabla^{*2} \mathbf{V}^* - \nabla p^*. \tag{2.44}$$

Furthermore, to integrate the implicit term $\mathscr{I}$, Ascher et al. [45] proposes an s-stage Diagonally Implicit Runge–Kutta (DIRK) scheme. This scheme is defined by the Butcher tableau represented by the matrix $[\mathbf{A}] \in \mathbb{R}^{s \times s}$ and the time and weight vectors, respectively, $\mathbf{c}, \mathbf{b} \in \mathbb{R}^s$. In

turn, the same reference considers for the explicit block $\mathscr{E}$ a Diagonally Explicit Runge–Kutta method with $\sigma = s+1$ stages. This method is described by abscissae $\check{\mathbf{c}}^T = (0, \mathbf{c})$, coefficient matrix $\left[\check{\mathbf{A}}\right] \in \mathbb{R}^{\sigma\times\sigma}$, and weights $\check{\mathbf{b}} \in \mathbb{R}^{\sigma}$.

The time integration over a single step from $t^{*(n)}$ to $t^{*(n+1)} = t^{*(n)} + \Delta t^*$, starting from the initial velocity field $\mathbf{V}^{*(n)}$, begins by computing the first explicit slope variable,

$$\check{\mathbf{K}}_1^* = \mathscr{E}\left(\mathbf{V}^{*(\mathbf{n})}\right). \tag{2.45}$$

Then, for each stage $i = 1, \ldots, s$, the corresponding stage velocity field $\mathbf{U}_i^*$ is calculated as

$$\mathbf{U}_i^* = \mathbf{V}^{*(n)} + \Delta t^* \sum_{j=1}^{i} a_{i,j}\mathbf{K}_j^* + \Delta t^* \sum_{j=1}^{i} \check{a}_{i+1,j}, \check{\mathbf{K}}_j^*, \tag{2.46}$$

with

$$\mathbf{K}_i^* = \mathscr{I}\left(\mathbf{U}_i^*, p_i^*\right). \tag{2.47}$$

An alternative formulation, proposed by Guesmi et al. [46], introduces a pressure-splitting term, $\mathbf{P}_i^*$, which groups the pressure contributions of different stages:

$$\mathbf{P}_i^* = a_{dd}\nabla^* p_i^* + \sum_{k=1}^{i-1} a_{i,k}\nabla^* p_k^*. \tag{2.48}$$

This leads to the reformulated stage velocity:

$$\mathbf{U}_i^* = \mathbf{V}^{*(n)} + \nu\Delta t^* \sum_{j=1}^{i} a_{i,j}\nabla^{*2}\mathbf{U}_j^* + \Delta t^* \sum_{j=1}^{i} \check{a}_{i+1,j}, \check{\mathbf{K}}_j^* + \Delta t^*\mathbf{P}_i^*, \tag{2.49}$$

which simplifies the implementation by removing the need to explicitly store the stage-level pressures. Consequently, solving Equation (2.49) requires s linear systems for the s stage velocities, together with s pressure projection steps. By excluding $\tilde{\mathbf{P}}_i$ and introducing the non-conservative stage velocity field $\mathbf{U}_i^{\#}$, we obtain

$$\left([\mathbf{I}] - \nu\Delta t^* a_{dd}\,[\Lambda]\right)\mathbf{U}_i^{*\#} = \mathbf{V}^{*(n)} + \Delta t^*\check{a}_{i+1,i}\check{\mathbf{K}}_i^* + \nu\Delta t^* \sum_{j=1}^{i-1} a_{i,j}\mathbf{K}_j^* + \Delta t^* \sum_{j=1}^{i-1} \check{a}_{i+1,j}\check{\mathbf{K}}_j^*. \tag{2.50}$$

Note that $[\Lambda]$ denotes the Laplacian matrix applied to the velocity field. This distinction is necessary because the operator differs when applied to velocity or pressure due to the imposed boundary conditions; for pressure, the Laplacian is represented as ∇_n^2. Although the mathematical form is the same, the actual values differ in practice. In addition, we set a_{dd} as the diagonal element of the implicit Butcher tableau, since its value does not change for all rows. Therefore, the solution of this system, which involves numerical inversion of the operator

$$[\mathbf{M}]_i = [\mathbf{I}] - \nu\Delta t^* a_{dd}\,[\Lambda]\,, \tag{2.51}$$

constitutes the computational bottleneck of the method. The intermediate conservative velocity fields are then written as

$$\mathbf{U}_i^* = \mathbf{U}_i^{*\#} - \Delta t^* \mathbf{P}_i^* \,, \tag{2.52}$$

with the respective velocity-Poisson equation,

$$\left[\nabla_n^{*2}\right] \mathbf{P}_i^* = \frac{\nabla_n^* \cdot \mathbf{U}_i^{*\#}}{\Delta t^*} \,. \tag{2.53}$$

Finally, the next time step solution is calculated using Equation (2.54), considering, one more time, a pressure projection step.

$$\mathbf{V}^{*(n+1)} = \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \mathbf{K}_i^* + \Delta t^* \sum_{i=1}^{s+1} \check{b}_i \check{\mathbf{K}}_i^* - \nabla_n^* p^{*(n+1)} \,. \tag{2.54}$$

All steps are summarized in the following algorithm.

Algorithm 2 IMEX Runge-Kutta method with pressure projection

procedure IMEX_RK($\mathbf{V}^{(n)}, t$)
  $\check{\mathbf{K}}_1^* \leftarrow \mathscr{E}\left(\mathbf{V}^{*(n)}\right)$
  for $i = 1$ to s **do**
    $\mathbf{U}_i^{*\#} \leftarrow \text{solve}\left[\left(\left[\mathbf{I}\right] - \nu\Delta t^* a_{dd}\left[\Lambda\right]\right)\mathbf{U}_i^{*\#} = \mathbf{V}^{*(n)} + \Delta t \check{a}_{i+1,i}\check{\mathbf{K}}_i^* + \Delta t^* \sum_{j=1}^{i-1} a_{i,j}\mathbf{K}_j^* + \check{a}_{i+1,j}\check{\mathbf{K}}_j^*\right]$
    $\mathbf{P}_i^* \leftarrow \text{solve}\left[\left[\nabla_n^{*2}\right]\mathbf{P}_i^* = \frac{\nabla_n^* \cdot \mathbf{U}_i^{*\#\,*}}{\Delta t}\right]$
    $\mathbf{U}_i^* \leftarrow \mathbf{U}_i^{*\#} - \Delta t^* \mathbf{P}_i^*$
    $\mathbf{K}_i^{*\#} \leftarrow \mathscr{I}\left(\mathbf{U}_i^*\right)$
    if $i < s$ **then**
      $\check{\mathbf{K}}_{i+1}^* \leftarrow \mathscr{E}\left(\mathbf{U}_i^*, \mathbf{f}_\mathbf{D}^*\right)$
    end if
  end for
  $\check{\mathbf{K}}_{s+1}^* \leftarrow \mathscr{E}\left(\mathbf{U}^{*(s)}\right)$
  $\mathbf{V}^{*\#} \leftarrow \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \mathbf{K}_i^* + \Delta t^* \sum_{i=1}^{s+1} \check{b}_i \check{\mathbf{K}}_i^*$
  $p^{*(n+1)} \leftarrow solve\left[\left[\nabla_n^2\right] p^{*(n+1)} = \frac{\nabla_n^* \cdot \mathbf{V}^{*\#}}{\Delta t^*}\right]$
  $\mathbf{V}^{(n+1)} \leftarrow \mathbf{V}^{(n+1)} - \Delta t \nabla \tilde{\mathbf{p}}^{(n+1)}$
end procedure

Moreover, it is important to properly define the combination of Butcher tableaus to be used. Following once again the work of Ascher et al. [45], two special cases for constructing the Butcher tableau for $\mathscr{E}$ are of particular interest. First, some methods modify the Butcher tableau for $\mathscr{I}$ by adding a row of zeros at the top and a column of zeros at the beginning, in order to construct the Butcher tableau for $\mathscr{E}$. This results in:

Table 2.8: Implicit Butcher tableau padded with zeros at the left column and top line

0	0	0	0	...	0
c_1	0	a_{11}	a_{12}	...	a_{1s}
c_2	0	a_{21}	a_{22}	...	a_{2s}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_s	0	a_{s1}	a_{s2}	...	a_{ss}
	0	b_1	b_2	...	b_s

This approach will lead to the simplification of Equation (2.54) as follows:

$$\mathbf{V}^{*(n+1)} = \mathbf{V}^{*(n)} + \Delta t^* \sum_{i=1}^{s} b_i \left(\mathbf{K}_i^* + \check{\mathbf{K}}_i^*\right) - \Delta t^* \nabla^* p^{*(n+1)}. \tag{2.55}$$

Second, some methods set the last explicit weight, $\check{b}\sigma$, to zero, thus eliminating the need to evaluate $\check{K}\sigma^*$, since it will not be included in Equation 2.54. Additionally, the implicit scheme is defined as stiffly accurate with $c_s = 1$:

$$b_j = a_{s,j},\ \check{b}_j = \check{a}_{\sigma,j}, \quad j = 1, ..., s, \tag{2.56}$$

leading to

$$\mathbf{V}^{(n+1)} = \mathbf{U}_s^{\#} - \Delta t \tilde{\mathbf{p}}^{(n+1)}. \tag{2.57}$$

This last formulation has proven particularly advantageous in stiff regimes, as shown by Ascher et al. [45].

The works of Ascher [44, 45] present a set of Butcher tableaus defining schemes with accuracies ranging from first to third order. In the present work, first-order schemes were not considered because of their insufficient accuracy for the intended objectives. Furthermore, the schemes will be referred to the initials of Ascher, since they were created in his work, followed by the triplet $(s, \sigma, \hat{p})$, where s is the number of implicit stages, σ the number of explicit stages ($\sigma = s+1$ for case (1), defined by Equation (2.55), and $\sigma = s$ for case (2), defined by Equations (2.56) and (2.57)) and $\hat{p}$ is the combined order of the scheme.

Beginning, with the second-order schemes, Ascher consider an L-stable, two-stage, second-order DIRK ASC(2,2,2), which is stiffly accurate and has the following Butcher tableaux

Table 2.9: ASC(2,2,2) implicit block butcher tableau

γ	γ	0
1	1-γ	γ
	1-γ	γ

Table 2.10: ASC(2,2,2) explicit block butcher tableau

0	0	0	0
γ	γ	0	0
1	δ	1-δ	0
	δ	1-δ	0

where, $\gamma = \frac{2-\sqrt{2}}{2}$ and $\delta = 1 - \frac{1}{2\gamma}$.

The third-order methods proposed by Ascher require four stages, since no pair consisting of a three-stage, L-stable DIRK and a four-stage ERK satisfying condition (2.56) with a combined third-order accuracy could be found [45]. They set $c_1 = c_3 = b_4 = \check{a}_{43} = \frac{1}{2}$, as well as the diagonal elements of the implicit BT, so that the final method, ASC(4,4,3) is, third-order and L-stable. The respective Butcher tableau are:

Table 2.11: ASC(4,4,3) implicit block butcher tableau

$\frac{1}{2}$	$\frac{1}{2}$	0	0	0
$\frac{2}{3}$	$\frac{1}{6}$	$\frac{1}{2}$	0	0
$\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	0	$\frac{1}{2}$
1	$\frac{3}{2}$	$-\frac{3}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
	$\frac{3}{2}$	$-\frac{3}{2}$	$\frac{1}{2}$	$\frac{1}{2}$

Table 2.12: ASC(4,4,3) explicit block butcher tableau

0	0	0	0	0	0
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0	0
$\frac{2}{3}$	$\frac{11}{18}$	$\frac{1}{18}$	0	0	0
$\frac{1}{2}$	$\frac{5}{6}$	$-\frac{5}{6}$	$\frac{1}{2}$	0	0
1	$\frac{1}{4}$	$\frac{7}{4}$	$\frac{3}{4}$	$-\frac{7}{4}$	0
	$\frac{1}{4}$	$\frac{7}{4}$	$\frac{3}{4}$	$-\frac{7}{4}$	0

2.2.4 Stability

The key motivation for this analysis is to evaluate whether IMEX schemes provide a wider stability range compared to classical explicit methods, as this would indicate their potential to overcome the strict time-step limitations imposed by viscous CFL conditions. To investigate this, we compare the stability properties of IMEX schemes with those of the classical RK4 scheme by plotting their stability regions. These regions provide a visual characterization of the areas in the complex plane where the numerical solution of the linear test equation, Equation (2.59), remains bounded.

This analysis is particularly relevant for IMEX schemes because their implicit and explicit components interact in a coupled manner that strongly influences overall stability. By contrasting their stability regions with that of RK4, we assess the extent to which IMEX schemes are better suited for handling stiff terms while retaining efficiency for non-stiff dynamics.

In this context, the desired outcome for IMEX schemes is to exhibit unconditional stability along the negative real axis, reflecting their ability to handle diffusive terms without restriction from the CFL condition.

Definition 2.2.5 (Stability Function)**.** *Let* $[\mathbf{A}] \in \mathbb{R}^{s\times s}$ *and* $\mathbf{b} \in \mathbb{R}^s$ *be the coefficients of an implicit Runge-Kutta method and* $z \in \mathbb{C}$*. The function*

$$R(z) = \frac{det\left([\mathbf{I}] - z[\mathbf{A}] + z[\mathbf{1}]\,\mathbf{b}^T\right)}{det\left([\mathbf{I}] - z[\mathbf{A}]\right)}, \tag{2.58}$$

is the stability function of the method, where $[\mathbf{1}] = (1,...,1)^T$. *It corresponds to the numerical solution applied to the test equation:*

$$\frac{dy}{dt} = \lambda y, \quad y^{(n)} = 1\,, \tag{2.59}$$

where $\lambda \in \mathbb{C}$ *is the eigenvalue governing the solution* $y(t) = e^{\lambda t}$, *and* $z = \Delta t\lambda$, *for step size* $\Delta t \in \mathbb{R}$.

Definition 2.2.6 (Stability Domain)**.** *The* ***stability domain*** S, *also referred to as the region of absolute stability of a numerical method, is defined as the set of all complex numbers* $z \in \mathbb{C}$ *for which the stability function* $R(z)$ *satisfies:*

$$\mathrm{S} = (z \in \mathbb{C} : |R(z)| \leq 1)\,. \tag{2.60}$$

The method is called ***A-stable*** *if its stability domain contains the entire left-half complex plane, i.e., when:*

$$\mathrm{S} \subset \mathbb{C}^-, \quad \textit{where} \quad \mathbb{C}^- = \{z \in \mathbb{C} : \mathrm{Re}(z) \leq 0\}. \tag{2.61}$$

Definition 2.2.7 (L-stable)**.** *Let* $z \in \mathbb{C}$. *A Runge-Kutta scheme is* ***L-stable*** *if and only if it is A-stable, and it additionally holds that*

$$\lim_{z\to\infty} R(z) = 0. \tag{2.62}$$

Proposition 2.2.1. *Let* $[\mathbf{A}] \in \mathbb{R}^{s\times s}$ *and* $\mathbf{b} \in \mathbb{R}^s$, *with* $s \in \mathbb{N}$, *be an s-stage implicit A-stable Runge-Kutta scheme such that* $[\mathbf{A}]$ *is non-singular. If* $[\mathbf{A}]$ *and* $\mathbf{b}$ *satisfy at least one of the following conditions:*

$$b_j = a_{sj}, \quad for\ j = 1,...,s, \tag{2.63}$$

$$b_1 = a_{i1}, \quad for\ i = 1,...,s, \tag{2.64}$$

the scheme is called L-stable.

We will now analyze the stability properties for the numerical schemes considered in this work, in particular RK4 from Section 2.2.1 and the IMEXs from Section 2.2.3. Rather than analyzing the stability regions of the implicit and explicit components separately, the stability region of the combined scheme is considered. Calvo et al. [47] defined the stability function of an IMEX Runge–Kutta scheme as:

$$R(x+iy) = \frac{det\left([\mathbf{I}]\,d - x\,[\mathbf{A}] - iy\left[\check{\mathbf{A}}\right] + z\,[\mathbf{1}]\,\mathbf{b}^T + iy\,[\mathbf{1}]\,\check{\mathbf{b}}^T\right)}{det\left([\mathbf{I}]\,d - x\,[\mathbf{A}] - iy\left[\check{\mathbf{A}}\right]\right)}. \tag{2.65}$$

In this context, $x = \Delta t\alpha$ and $y = \Delta t\beta$, where $\alpha \in \mathbb{R}$ and $\beta \in \mathbb{R}$ represent the eigenvalue of the implicit and explicit blocks, respectively, resulting from a scalar test equation:

$$\frac{dx}{dt} = \alpha x + i\beta x, \tag{2.66}$$

where $\alpha \leq 0$, $\beta > 0$ and $i = \sqrt{-1}$. The stability functions for the considered schemes are:

1. **Classical Runge-Kutta** [39]

$$R(x+iy) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}. \tag{2.67}$$

2. **ARS(2,2,2)** [48]

$$R(x+iy) = \frac{1 + (1-2\gamma)x + \gamma(\delta-1)y^2 + i\left[y + \gamma(1-\delta-\gamma)xy\right]}{(1-\gamma x)^2}. \tag{2.68}$$

3. **ARS(4,4,3)** [48]

$$R(x+iy) = \frac{48\left(6-6x+x^3\right) - 144(1-x)y^2 + 3x^2y^2 - 7y^4}{18\left(x-2\right)^4} + \tag{2.69}$$

$$+ \frac{i\left[48\left(6-6x-y^2\right)y + 29x^3y + 57xy^3\right]}{18\left(x-2\right)^4} \tag{2.70}$$

Figure 2.4 shows the stability region for several time-integration schemes.

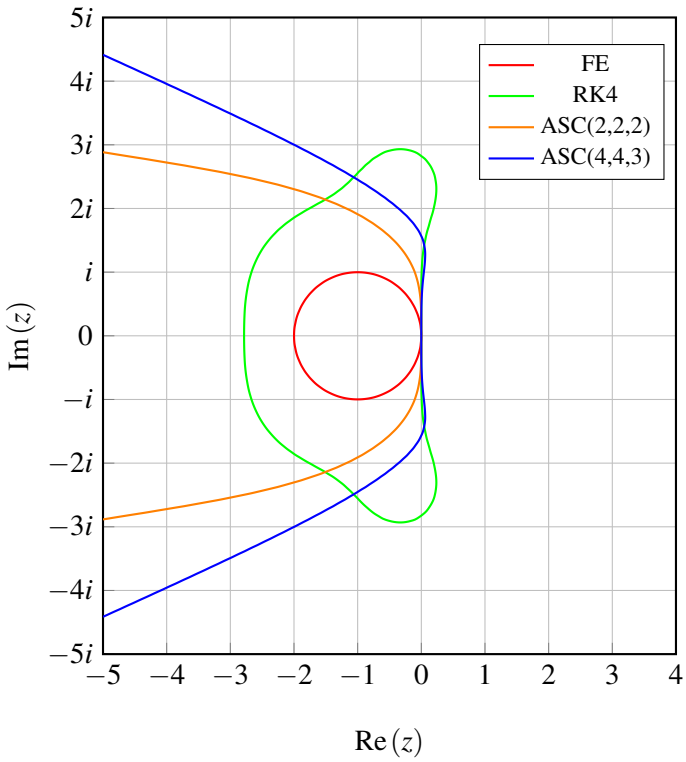

Figure 2.4: Stability regions comparison in the complex plane for several time-integration schemes. The interior of each curve represents the stable region where $|R(z)| \leq 1$.

These stability regions were obtained through a numerical process (detailed in Appendix E) since directly solving the inequality in Equation (2.60) is analytically intractable. The numerical

approach evaluates the stability function $R(z)$ across the complex plane and identifies the boundary where $|R(z)| < 1$.

Table 2.13 summarizes the most important characteristics of each stability region.

Table 2.13: Characteristics of each stability region

	Re_{min}	Re_{max}	Im_{min}	Im_{max}	P_{Re}
FE	-2.00	0	$-i$	i	$(0,0)$
RK4	-2.79	0.24	$-2.93i$	$-2.93i$	$(0,+-2.93)$
ARS(2,2,2)	$-\infty$	0.00055	$-\infty$	$+\infty$	$(0,0)$
ARS(4,4,4)	$-\infty$	0.061	$-\infty$	$+\infty$	$(0,+-0.33)$

As seen, the results demonstrate stability properties consistent with theoretical expectations:

1. Explicit Schemes Comparison

   The fourth-order Runge-Kutta scheme (RK4) exhibits a significantly larger stability region compared to the Forward Euler (FE) method, with its boundary fully encompassing FE's stability domain. The RK4's expanded stability region (39.5% times larger along the imaginary axis than FE and 39.5% times larger in the real axis) confirms its superior tolerance to higher-frequency modes;

2. Implicit Schemes Characteristics

   In contrast, the implicit ARS schemes demonstrate open stability regions extending infinitely along the negative real axis, meaning an unconditionally stable character for the diffusive term. Between the two IMEX formulations, ARS(4,4,3) shows greater stability coverage than ARS(2,2,2);

3. Comparative Stability Performance

   Notably, the IMEX schemes unbounded real axis character displaces a superior role when applied to stiff regime problems, although their orders are lower than the RK4. Both ARS(4,4,3) and RK4 exhibit stability regions that extend to the positive real axis, with RK4 demonstrating a significantly larger stable area in this regime.

All things considered, giving the bigger stability region for the IMEX, potentially implies a greater admissible time-step, as explained in the next Subsection.

2.2.5 Time Step Restriction

Both the FE and RK4 schemes have closed stability regions. This property of explicit schemes leads to two time-step restrictions, known as CFL conditions [49], corresponding to the convective and diffusive terms. These conditions limit the admissible time step to ensure numerical stability.

Moukalled et al. [49] defines the general $N_{dim}-$dimentional convective CFL condition as

$$\Delta t_u \leq \frac{Co_u^{max}}{\sum_{i=1}^{N_{dim}} \frac{V_i}{\Delta x_i}}, \tag{2.71}$$

and the diffusive CFL condition as

$$\Delta t_\nu \leq \frac{Co_\nu^{max}}{\nu \sum_{i=1}^{N_{dim}} \frac{1}{\Delta x_i^2}}, \tag{2.72}$$

where V_i is the $i-$th velocity component. The stable Δt corresponds to the minimum value satisfying these constraints across all grid cells. The standard limits $Co_u^{\max} = 1$ and $Co_\nu^{\max} = 0.5$ are derived for the FE scheme and are considered by default in *SP-Wind*. However, higher-order time integrators are stable for larger Courant numbers.

In the cavity case study, we use the stability analysis from Saad and Karam [50], which provides precise $Co_u^{\max}$ and $Co_\nu^{\max}$ values for high-order Runge-Kutta schemes. This allows for a larger, stable time step compared to the conventional FE-based criterion.

For IMEX schemes, a general CFL condition remains undefined [48]. Their open stability region along the negative real axis exhibits no intersection point, rendering these schemes effectively unconditionally stable with respect to viscous time-step restrictions.

2.2.6 Pressure Projection Method

As seen previously, the pressure projection method constitutes a critical step not only in terms of computational time but also in terms of accuracy. The reason is that this stage requires solving Equations (2.52) and/or (2.54), which constitutes a large, sparse linear system, which can be computationally expensive, therefore dominating the cost of the overall simulation. Moreover, the projection directly determines the accuracy with which the divergence-free condition is enforced in the velocity field. Any inaccuracies in this step propagate throughout the solution, numerical instabilities, or loss of mass conservation. For these reasons, we find it important to clearly describe this step in detail. Consider that any time-integrator can be written as,

$$[\mathbf{M}]\,\mathbf{V}^{*(n+1)} = \mathbf{H}^{*(n)} - \Delta t^* \cdot \nabla_n^* p^{*(n+1)}, \tag{2.73}$$

where $\mathbf{H}^{*(n)} = \mathbf{V}^{*(n)} + \Delta t^* \cdot \mathrm{RHS}\left(\mathbf{V}^{*(n)}\right)$ expresses the sum of the explicit terms of the numerical scheme applied. Recall that $\mathrm{RHS}\left(\mathbf{V}^{*(n)}\right)$ accounts for all terms on the right side of the momentum equation, except the pressure. Furthermore, $[\mathbf{M}]$ is set as the mass matrix, expressing all the interaction coefficients between cells and, in fact, $[\mathbf{M}]$ determines whether the numerical method is implicit or explicit; in the latter case, it reduces to the identity matrix, $[\mathbf{I}]$.

The principle behind this method is the fundamental theorem of vector calculus, known as Helmholtz-Hodge decomposition, which states that any sufficiently smooth, rapidly decaying vector field can be resolved into the sum of an irrotational vector field (curl-free) and a solenoidal vector field (divergence-free) [51, 52]. Applied to the velocity field, we can write

$$\mathbf{V}^{*\#} = \mathbf{V}^*_{\perp} + \mathbf{V}^*_{\parallel}, \tag{2.74}$$

where $\mathbf{V}^*_{\parallel}$ and $\mathbf{V}^*_{\perp}$ are the irrotational and solenoidal velocity components, respectively, i.e.,

$$\nabla^* \cdot \mathbf{V}^*_{\perp} = 0, \tag{2.75}$$

$$\nabla^* \times \mathbf{V}^*_{\parallel} = 0. \tag{2.76}$$

In practice, the form,

$$\mathbf{V}^*_{\perp} = \mathbf{V}^{*\#} - \mathbf{V}^*_{\parallel}, \tag{2.77}$$

is what we want to solve for, the incompressible field. Since the irrotational component $\mathbf{V}^*_{\parallel}$ is curl-free, it can be expressed as the gradient of a scalar potential ϕ^*:

$$\mathbf{V}^*_{\parallel} = \nabla^* \phi^*, \tag{2.78}$$

thus leading to,

$$\mathbf{V}^*_{\perp} = \mathbf{V}^{*\#} - \nabla^* \phi^*. \tag{2.79}$$

In the context of the incompressible Navier-Stokes equations, the scalar potential ϕ^* is identified with the pressure gradient, scaled by the time step. Hence, the final velocity field that satisfies the incompressibility constraint is recovered as:

$$\mathbf{V}^{*(n+1)} = \mathbf{V}^{*\#} - \Delta t^* \nabla^* p^*. \tag{2.80}$$

All things considered, the workflow of the pressure correction step begins by considering the discrete Navier–Stokes equations without the pressure term and computing a provisional, non-conservative velocity field $\mathbf{V}^{*\#}$. This intermediate velocity is subsequently corrected to enforce incompressibility through the introduction of the pressure term by solving a Poisson equation for the pressure. If we consider an iterative solver, just as we did in the cavity case study (see Section 4.2), we solve for:

$$\nabla^*_n \cdot \nabla^*_n p^* = \frac{1}{\Delta t^*} \nabla^*_n \cdot \mathbf{H}^{*(n)}. \tag{2.81}$$

Hence, when applying a direct solver, just as in *SP-Wind*, we could solve for the closed form of (2.81):

$$\nabla^*_n \cdot \nabla^*_n p^* = \nabla^*_n \cdot \mathrm{RHS}\left(\mathbf{V}^{*(n)}\right). \tag{2.82}$$

Although there is a natural tendency to simplify the left-hand side of this equation from $\nabla^*_n \cdot \nabla^*_n p^*$ to $\left[\nabla^{*2}_n\right] p^*$ - a transformation that is always valid in a continuous domain - its assumption may

introduce a potential caveat. Depending on the selected discretization method and the grid arrangement, this statement may not always be valid. A well-known example arises from the Ladyzhenskaya–Babuška–Brezzi (LBB) condition [53, 54] – also referred to as the divergence stability condition – in finite element analysis. This condition states that using a finite difference (FD) scheme on a collocated grid leads to an even–odd decoupling of the pressure field, resulting in nonphysical solutions for both pressure and velocity. To address this issue, one may employ staggered grids, adopt different interpolation orders for pressure and velocity, or modify the continuity equation to introduce artificial compressibility [55]. This consideration supports the choice of employing the finite-volume method (FVM) in the cavity case study and the option of solving the w-velocity component on a staggered grid in *SP-Wind*. Thus, Equations (2.81) and (2.82) can then be written as well-known Poisson equations, respectively,

$$\left[\nabla_n^{*2}\right] p^* = \frac{1}{\Delta t} \nabla_n^* \cdot \mathbf{H}^{*(n)}, \tag{2.83}$$

$$\left[\nabla_n^{*2}\right] p^* = \nabla_n^* \cdot \mathrm{RHS}\left(\mathbf{V}^{*(n)}\right). \tag{2.84}$$

The Poisson equation essentially boils down to solving a linear system. In general, the associated computational cost scales according to N^3 (where N refers to the dimension of the system), although more efficient implementations can be obtained by exploiting the sparsity of the matrices or through a smart choice for discretization. However, it is still considered one of the most time-consuming processes in common CFD codes [56, 57]. On the one hand, when selecting a direct solver, the "exact" solution is found, affected only by round-off errors; however, they scale severely in time with the number of elements. On the other hand, in response to this behavior, iterative solvers emerge, consisting, as the name indicates, of an iterative process that approximates, up to a certain desired tolerance, the system solution. As this tolerance increases in demand, so does the time and accuracy. The stability of the linear solvers can be consulted in the Appendix [A].

2.2.7 Linear Solvers

In this section, we present the linear solvers under consideration throughout the test cases. They express ways of solving systems of type $[\mathbf{A}]\,\mathbf{x} = \mathbf{b}$.

- **LU Decomposition (SP-Wind)**: LU decomposition factorizes the matrix $[\mathbf{A}]$ into a lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$, i.e.,

$$[\mathbf{A}] = [\mathbf{L}]\,[\mathbf{U}]. \tag{2.85}$$

  Once $[\mathbf{L}]$ and $[\mathbf{U}]$ are computed, solving $[\mathbf{A}]\,\mathbf{x} = \mathbf{b}$ reduces to simple forward and backward substitutions. This approach is efficient when the iteration matrix remains constant and only the right-hand side $\mathbf{b}$ changes. In this work, the LU solver is applied using the implementation available in *SP-Wind*.

- **Conjugate Gradient Method (CG)**: The Conjugate Gradient method is an iterative technique for solving linear systems where the matrix is symmetric and positive definite, i.e.,

$$\mathbf{A} = \mathbf{A}^{\mathrm{T}} \quad \text{and} \quad \mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} > 0 \quad \forall \mathbf{x} \neq 0. \tag{2.86}$$

  It can be derived by minimizing the quadratic objective function

$$Q(\mathbf{x}) = \frac{1}{2}\mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} - \mathbf{b}^{\mathrm{T}}\mathbf{x} + c\,, \tag{2.87}$$

  whose gradient

$$\nabla Q(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}, \tag{2.88}$$

  vanishes at the solution $\mathbf{x}^*$, satisfying

$$\mathbf{A}\mathbf{x}^* = \mathbf{b}. \tag{2.89}$$

  The method generates a sequence of conjugate search directions to converge efficiently to $\mathbf{x}^*$. In practice, we will use `scipy.sparse.linalg.cg` [58] to apply this method, as can be seen in Appendix D.

- **Algebraic Multigrid (AMG)**: AMG is an iterative solver for large, sparse linear systems that accelerates convergence by operating across multiple levels of coarsened grids. The key idea is that a low-frequency error on a fine grid appears as a high-frequency error on a coarser grid, which is easier to smooth. When convergence slows, the solver switches to a coarser grid.

  AMG automatically constructs coarse representations of the system, smoothing errors on fine levels and correcting them on coarse levels. For an in-depth knowledge, see [59]. In the cavity case study, the AMG solver is implemented using `pyAMG.ruge_stuben_solver` [60], also seen in Appendix D.

Chapter 3

Numerical Setups and Case Description

This chapter describes three numerical case studies designed to validate the IMEX schemes (ARS (2,2,2) and ARS (4,4,3)), see Section 2.2.3. The case studies were chosen to follow a logical progression in complexity, gradually increasing both the dimensionality of the problem and the numerical operations involved, such as the addition of the pressure projection method.

The first test, described in Section 3.1, focuses on a one-dimensional convection–diffusion problem. By simplifying the governing equations, this case served as the initial step in the code development. Its main purpose was to examine the sensitivity of the methods to periodic boundary conditions and to provide a controlled environment to validate the basic implementation.

The second test case, presented in Section 3.2, extends the formulation to two dimensions through the classical Lid-Driven Cavity problem, where the full Navier–Stokes equations are solved. This represents a key step forward, as it introduces the pressure term and the projection step. The goal here is twofold: to better understand the pressure projection method and to assess the physical reliability of the IMEX solutions. This case also serves as the basis for a convergence study analyzing the error behavior under spatial and temporal refinement.

Finally, a three-dimensional case, discussed in Section 3.3, is considered by adapting the *SP-Wind* code, originally developed for wind farm simulations, to a cilia-driven flow configuration. Despite considerable effort, the IMEX schemes could not be successfully integrated into this framework. Although the code was rewritten twice, issues related to boundary condition enforcement and inter-processor communication prevented the simulations from yielding physically meaningful results.

3.1 Burgers' Equation (1D)

Burgers' equation is a fundamental one-dimensional model that captures the interplay of nonlinear convection and viscous diffusion. Due to its simplicity and analytical tractability, it is widely used as a reference for testing new time integration schemes in fluid dynamic codes.

Early numerical studies applied pseudospectral, finite-element and finite-difference discretizations together with explicit or implicit, single-step, time integration schemes [61, 62]. More recently, a study by Mukundan and Awasthi [63] proposed a novel numerical scheme based on the method of lines (MOL) to solve the nonlinear time-dependent Burgers' equation. This approach utilized second-order finite differences for spatial discretization and an implicit linearized scheme for time integration. The resulting system was then integrated using an implicit scheme, which allowed for the treatment of stiff terms in the equation. Related to the goal of this thesis, Rottmann-Matthes [64] introduced an IMEX-RK scheme to capture similarity solutions in the multidimensional Burgers' equation. The scheme was based on a method of lines (semi-)discretization and proved second-order convergence for smooth solutions. Numerical experiments demonstrated the method's effectiveness across various viscosity regimes, suggesting its suitability for capturing similarity solutions in general hyperbolic-parabolic partial differential equations.

To facilitate the comparison, we choose the work of Javidi [65], which presents a numerical solution of Burgers' equation using a modified extended backward differentiation formula (MF-MEBDF) scheme. This approach combined the method of lines with a matrix-free implementation of the MF-MEBDF, providing an efficient and accurate solution to Burgers' equation. As a spatial discretisation scheme, it applies the $4^{th}-$order finite difference method on a cell-centered grid (see Darvishi and Javidi [66]). The results obtained demonstrated the effectiveness of the method, with numerical solutions that closely matched exact solutions for several viscosity values. The Burgers' equation is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \tag{3.1}$$

with initial condition,

$$u(0,x) = u_{exact}(0,x), \tag{3.2}$$

and boundary conditions,

$$BC_1^{(n)} = u_{exact}(t_n, 0), \tag{3.3}$$

$$BC_2^{(n)} = u_{exact}(t_n, N_x). \tag{3.4}$$

The exact solution is defined as,

$$u(t,x) = \frac{0.1e^{-A} + 0.5e^{-B} + e^{-C}}{e^{-A} + e^{-B} + e^{-C}}, \tag{3.5}$$

where,

$$A = \frac{0.05}{\nu}(x - 0.5 + 4.95t), \tag{3.6}$$

$$B = \frac{0.25}{\nu}(x - 0.5 + 0.75t), \tag{3.7}$$

$$C = \frac{0.5}{\nu}(x - 0.5 + 0.375t). \tag{3.8}$$

For the IMEX scheme, the conjugate gradient method was used to solve the momentum equations (Equation (2.50)), with a tolerance of 10^{-6}. This value was chosen based on trial-and-error, and is coherent with the tolerance defined at the cavity case. The code for this test case is shown in Appendix [F].

3.2 Lid-Driven Cavity Flow (2D)

Next, we turn our attention to the Lid-Driven Cavity flow in a two-dimensional domain. This benchmark problem is widely recognized in CFD as a standard test for the implementation of numerical schemes. In the present work, it also marks the first application of the implemented IMEX schemes to the full Navier–Stokes equations (see (2.10) and (2.11)). Unlike the previous one-dimensional case, the lid-driven cavity introduces additional physical meaning through the pressure–velocity coupling and projection step, providing a meaningful validation of the solver.

Starting with the most relevant work in the field, Ghia et al. [67] study introduced a coupled-strongly-implicit multigrid (CSI-MG) algorithm to solve the two-dimensional incompressible Navier–Stokes equations for the lid-driven cavity flow, with a constant lid velocity at Reynolds numbers up to 10,000 on uniform meshes (fine mesh as 257×257 grid points). Using a vorticity stream function formulation and a full multigrid (FMG) cycle, the authors achieved convergence rates that were far superior to conventional Gauss-Seidel iterations, reducing the CPU time for a grid 129×129 at $Re = 1000$ to only 1.5 min on an AMDAHL 470 V/6 computer. Furthermore, in their study, benchmark results for the u and v velocity components along the vertical centerline, the stream-function and vorticity fields, and the locations and strengths of the primary and secondary vortices were reported for a cavity with an aspect ratio equal to 1. In addition, they also reported the mean kinetic energy for several Re numbers. These quantities were compared with previously published data for $Re = 100$, 400 and 1000, and the agreement of the velocity profiles and streamline patterns served as the primary validation of their CSI-MG solver.

Later, Benjamin and Denny [68] obtained high-Re solutions ($Re = 10,000$) on a non-uniform 151×151 grid, but required computational times of an hour or more. Agarwal [69] used a uniform 121×121 mesh with a third-order upwind scheme to reach $Re = 7500$, again at substantial cost. Together, these works form a comprehensive theoretical and numerical baseline for evaluating IMEX schemes on the lid-driven cavity flow problem. Recently, the work by Erturk [70] provided high-accuracy reference solutions for $Re \leq 100$, using second-order central differencing and fine uniform grids to eliminate discretization error in the viscous-dominated limit. Then Khorasanizade and Joao Sousa [71] studied the same problem at moderate Reynolds numbers using the incompressible Smoothed Particle Hydrodynamics (SPH) method. The authors developed and tested a new treatment for no-slip boundary conditions that improves the accuracy of wall shear stress calculations. They also incorporated a particle shifting algorithm to avoid particle clustering and enhance numerical stability. The method was applied for Reynolds numbers of 100, 400, 1000, and 3200, and the obtained results were consistent with the data from Ghia et al. Ghia et al. [67]. The main result was that the new boundary condition reduces errors in the wall force

calculations and improves the formation of vortices inside the cavity. A recent validation study from Sumesaraayi and Ihmsen [72] reproduced the case $Re = 100$, confirming that the total kinetic energy reaches a steady state after ≈ 10 s with a value of 0.034, with the calculated u and v profiles corresponding to the Ghia data within numerical tolerance.

The test case under consideration is the classical lid-driven cavity flow, consisting of a square cavity with an aspect ratio of 1, as shown in Figure 3.1, and described by the Navier-Stokes equations. The top wall (lid) moves with a constant velocity U_{lid}, while all others remain stationary. Table 3.1 summarizes the characteristic lengths for the lid-driven cavity flow case study.

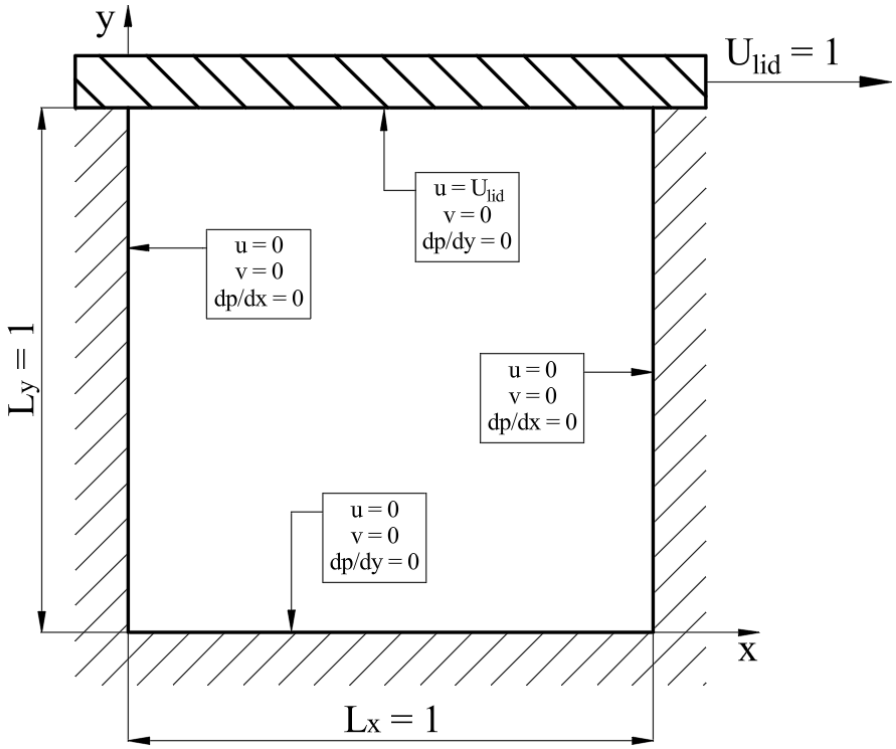

Figure 3.1: Sketch of the domain and boundary conditions for the lid-driven cavity flow case study.

Table 3.1: Scaling parameters used to non-dimensionalize the momentum equations and continuity equation, along with their primary dimensions, for the lid-driven cavity flow case study

Scaling Parameter	Associated Variable	Value
L_c	L_x	1
V_c	U_{lid}	1
f	U_{lid}/L_x	1
$p_0 - p_\infty$	U_{lid}^2	1

Furthermore, we applied a second-order Finite Volume method for spatial discretization along with the boundary conditions shown in Table 3.2. Note that due to the left-staggered grid arrangement

Table 3.2: Boundary conditions for the lid-driven cavity flow domain

Left-wall	Bottom-wall	Right-wall	Top-wall
$u^*_{y,2} = 0$	$u^*_{0,x} = -u^*_{1,x}$	$u^*_{y,N_x+1} = 0$	$u^*_{N_y+1,x} = 2U^*_{lid} - u^*_{N_y,x}$
$v^*_{y,1} = -v^*_{y,2}$	$v^*_{1,x} = 0$	$v^*_{y,N_x+1} = -v^*_{y,N_x}$	$v^*_{N_y+1,x} = 0$
$p^*_{y,1} = p^*_{y,2}$	$p^*_{0,x} = p^*_{1,x}$	$p^*_{y,N_x+1} = p^*_{y,N_x}$	$p^*_{N_y+1,x} = p^*_{N_y,x}$

(see Figure B.1 in Appendix B), for the same wall, a combination of Dirichlet and Neumann boundary conditions is applied.

Lastly, the IMEX schemes are evaluated at a Reynolds number of 100, a regime where the time step is restricted primarily by the viscous CFL condition. This setting provides a controlled, yet challenging environment for assessing the accuracy, stability, and efficiency of the proposed approach.

3.3 Cilia Case Study (3D)

The setup of the cilia-driven flow is established with two main steps. First, we take the wind-farm case already implemented in *SP-Wind* software and changed the boundary conditions on the upper and lower walls to the no-slip condition, maintaining the periodic inlet-outlet. Then, we disable the modules for the blades and nacelle, as well as the aero-elastic module applied to the wind turbine's tower. Lastly, we adapt the code to also include the correct equation for the drag coefficient, Equation (2.24), and the post-processing subroutine to accommodate these changes.

It is important to emphasize that, although the cilia operate in a low-Reynolds-number regime where the Stokes approximation is often valid, all simulations in this thesis solve the full incompressible Navier–Stokes equations, Equation (2.10).

Our domain is now characterized by two fixed parallel plates, with a fixed-to-the-ground cylinder (representing the cilia) in-between as seen in Figure 3.2.

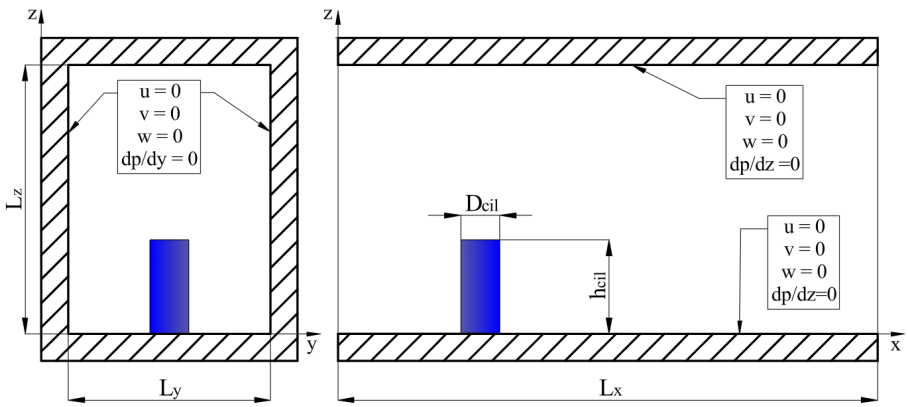

Figure 3.2: Sketch of the domain and boundary conditions for the cilia case.

The selected characteristic lengths are summarized in Table 3.3.

Table 3.3: Scaling parameters used to non-dimensionalize the momentum equations and continuity equation, along with their primary dimensions, for the cilia case study

Scaling Parameter	Associated Variable	Value
L_c	h	1
V_c	$\overline{u}_h = u(h)$	1
f	$\overline{u}_h / h$	1
$p_0 - p_\infty$	$\overline{u}_h^2$	1

In addition, the inlet velocity profile is set based on the analytical solution of the Navier-Stokes equations for a steady one-dimentional, laminar flow ((depend only on the vertical coordinate (z))) that hoes by,

$$u^*(z^*) = \frac{Re_{h^*} \cdot h^{*2}}{2} \left| \frac{\partial p^*}{\partial x^*} \right| \left[2z^* - z^{*2} \right] . \tag{3.9}$$

The reader is referred to the Appendix C for a detailed demonstration. Here, Re_{h^*} is the Reynolds number based on the channel half-width h^* and the mean inlet velocity $\overline{u}_h^* = u^*(h^*)$. The forcing term, $|\partial p^* / \partial x^*|$, is defined by the pressure gradient in the main flow direction x. By default, *SP-Wind* assumes $\rho = 1$ and all quantities are non-dimensional, such that $Re = 1/\nu$. The boundary conditions defined as no-slip everywhere, except for the inlet-outlet, that is of type, periodic.

Furthermore, the dimensions of the cilia, following the conventions in Section 1, the aspect ratio is bounded by:

$$40 \leq \frac{h_{cil}^*}{D_{cil}^*} \leq 60 . \tag{3.10}$$

In real conditions, the cilia operate in domains much larger than their size. The case studied here is therefore only representative, chosen to be simple enough to verify whether the IMEX scheme overcomes the viscous time-step restriction. We set the cilia height to half the channel half-width, $h_{cil}^* = 0.5$, which gives $D_{cil}^* = 0.01$, corresponding to the mid-value aspect ratio $\frac{h_{cil}^*}{D_{cil}^*} = 50$. In addition, when considering two cilia, the spacing (l_{cil}) is defined such that the applied mesh captures both cilias. Hence, a more correct approach would be to, based on Sedaghat et al. [30], define the cilia spacing as approximately three cilia diameters, i.e. $l_{cil} \approx 0.03$. This implies a further refinement of the grid, since in the current case $\Delta x^* = 0.222$.

Chapter 4

Results and Discussion

This chapter presents the computational results obtained from implementing and validating IMEX time integration schemes. The investigation progresses systematically from one-dimensional Burgers' equation, in Section 4.1, to two-dimensional lid-driven cavity flow, Section 4.2, culminating in a preliminary three-dimensional cilia flow study, presented in Section 4.3.

The numerical schemes are evaluated based on their accuracy, convergence behavior, and computational efficiency, in all sections. Spatial and temporal convergence analysis are conducted using Richardson extrapolation techniques, as seen in Section 4.2, with results benchmarked against established reference data. The performance of IMEX schemes is compared with classical Runge-Kutta methods to assess potential advantages in handling stiff problems.

We consider as validation metrics L^2-norm errors for Burgers' equation in Section, mean kinetic energy and velocity profiles for cavity flow in Section, and flow characteristics for cilia configurations. The cilia flow analysis examines both single cilium behavior and cilium-cilium interaction dynamics were we examine qualitatively the flow characteristics, providing sense for this type of flows.

4.1 Burgers' Equation (1D)

We implemented a computational code (see Appendix F) to perform the numerical integration of Burgers' equation. A test case defined by Javidi [65] was evaluated, with the discretization of 10 elements for an end simulation time of 1 second. The time-step is set equal to 10^{-4}, and the kinematic viscosity is equal to 1.

As shown in Figure 4.1, the IMEX solutions coincide with the RK4 solution, and both are in close agreement with the analytical solution given in Equation (3.5).

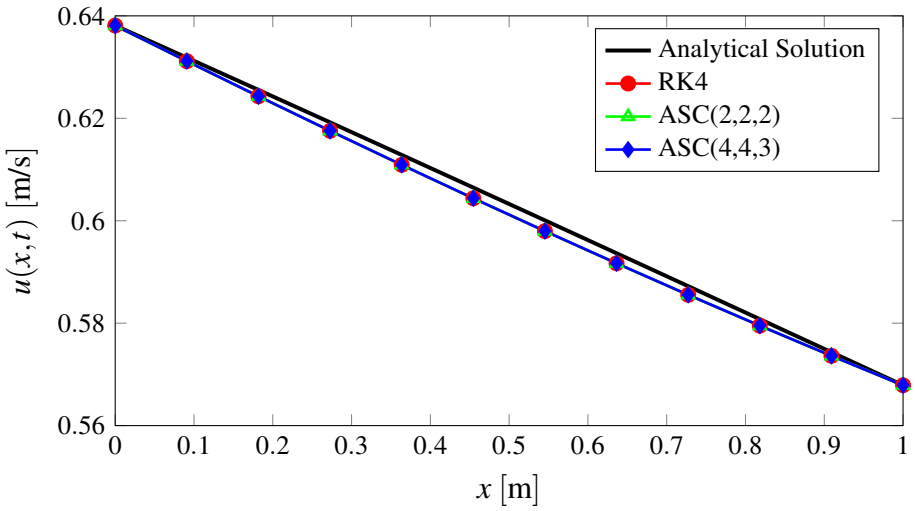

Figure 4.1: Comparison of numerical solutions obtained using classical Runge-Kutta (RK4) and IMEX schemes (ARS(2,2,2) and ARS(4,4,3)) against the analytical solution, for $t = 1$, $\nu = 1$ and $N_x = 10$, for the one-dimensional Burgers' equation case study.

Table 4.1 shows the L^2-norm velocity errors for each of the time integration schemes used compared to the exact solution. The results obtained are also compared with those reported by Javidi [65].

Table 4.1: L^2-norm velocity errors for the one-dimensional Burgers' equation case study using $\nu = 1$ $\left[\mathrm{m}^2/\mathrm{s}\right]$ and $N_x = 12$, for $\Delta t = 1 \times 10^{-4}$ [s]

t	RK4	ASC(2,2,2)	ASC(4,4,3)	Javidi
0.2	4.2425×10^{-3}	4.2430×10^{-3}	4.2413×10^{-3}	2.4244×10^{-11}
0.4	4.5292×10^{-3}	4.5297×10^{-3}	4.5280×10^{-3}	6.4101×10^{-11}
0.6	4.7337×10^{-3}	4.7343×10^{-3}	4.7323×10^{-3}	3.7221×10^{-10}
0.8	4.9284×10^{-3}	4.9291×10^{-3}	4.9268×10^{-3}	6.0105×10^{-10}
1.0	5.1125×10^{-3}	5.1133×10^{-3}	5.1108×10^{-3}	6.3600×10^{-10}

We conclude that the IMEx implementation as a proof of concept is successful.

4.2 Lid-Driven Cavity Flow (2D)

With the IMEX scheme encoded and validated in a one-dimensional case study, the next step is to analyze its behaviour in a two-dimensional case study. Using reference data from Ghia et al. [67] and Khorasanizade and Joao Sousa [71], we evaluated the velocity components u and v in the mid-planes (x and y directions) under steady-state conditions. In addition, an order-of-accuracy analysis based on the Richardson extrapolation method [73, 74] is carried out to study how the numerical error evolves with spatial and temporal refinement. These results are presented in Section 4.2.2 and Section 4.2.3, respectively.

Finally, in Section 2.2.5, we investigate whether admissible time-step enlargement in IMEX schemes can compensate for their additional complexity, thereby justifying their use.

4.2.1 Linear Solver Selection

As a first step, an appropriate linear solver was chosen for both the pressure Poisson equation, Equation (2.53), and the stage velocities equations, Equation (2.50). The selection process began with the RK4 time integration scheme. The underlying premise is that the most efficient and/or accurate pressure solver would remain optimal for the IMEX schemes, given identical mesh configurations and boundary conditions. Subsequently, with the pressure solver established, the optimal solver for the momentum equations was determined, within the ARS(4,4,3) time integration scheme.

Two linear solvers were evaluated: Conjugate Gradient (CG) and Algebraic Multigrid (AMG), see Section 2.2.7. The computational efficiency was quantified by the temporal cost per iteration ($C_{\Delta t}$), defined as the total wall time divided by the number of iterations. Figure 4.2 presents these results on a log-log scale, revealing the distinct scaling characteristics of each solver. The AMG solver demonstrates slightly faster convergence relative to the CG, particularly at higher resolutions, making it the preferred choice for pressure system solutions.

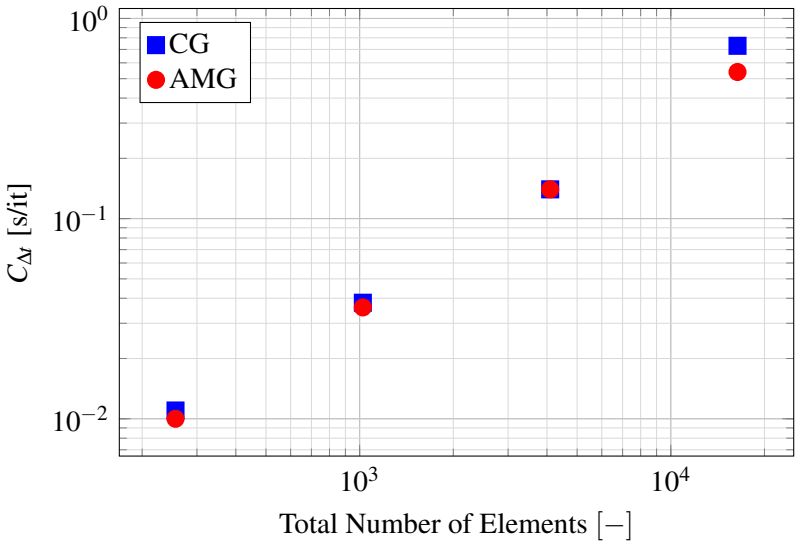

Figure 4.2: Comparative performance analysis of Conjugate Gradient (CG) and Algebraic Multigrid (AMG) linear solvers for the pressure Poisson equation using the RK4 time integration scheme for the two-dimensional lid-driven cavity flow case study.

Following the selection of AMG as the optimal linear solver for the pressure system, the same comparative approach was applied to solve the linear systems as part of the stage velocity calculation. The results again demonstrated that the numerical solution accuracy remains insensitive to the choice of linear solver. However, in contrast to the pressure system results, the CG method slightly outperformed AMG for the velocity calculations, as illustrated in Figure 4.3. Another important observation is the inherently higher computational cost associated with the IMEX scheme

compared to the RK4 method, when merging Figure 4.2 and 4.3. This was anticipated, as the IMEX formulation requires solving an additional linear system per iteration, thereby increasing the overall computational burden.

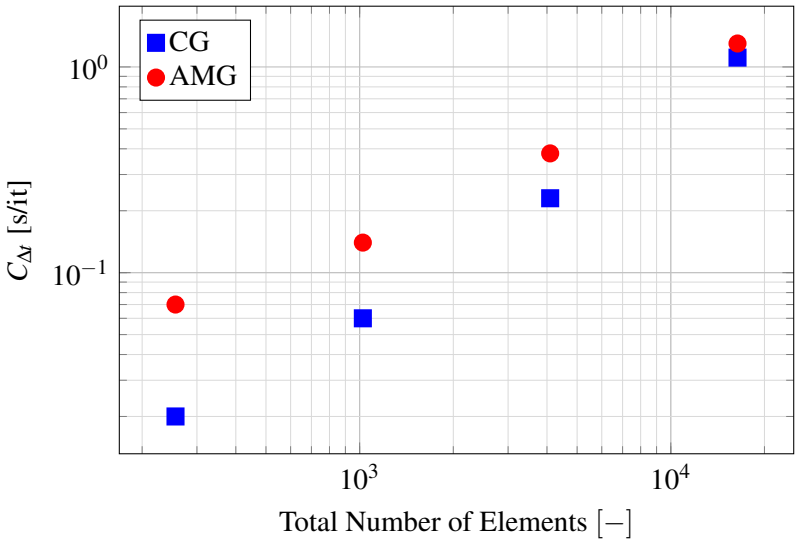

Figure 4.3: Comparative performance analysis of Conjugate Gradient (CG) and Algebraic Multigrid (AMG) linear solvers for the momentum equations using the IMEX ASC(4,4,3) time integration scheme for the two-dimensional lid-driven cavity flow case study.

The final choice will be to consider AMG as the pressure-Poisson solver and CG as the stage velocities solver, when used. The computational codes for the RK4 and IMEX time integration schemes can be found in Appendix G.

4.2.2 Spatial Convergence Analysis

Prior to presenting the spatial convergence results, it is essential to specify the physical quantity to be monitored during the convergence analysis. This quantity must be global in nature to avoid sensitivity to local variations between different grid resolutions. Following established practices in lid-driven cavity flow studies [67, 71, 73, 75], the mean kinetic energy of the cavity $\left(\overline{E}_k\right)$ was selected as the convergence metric defined as:

$$\overline{E}_k = \frac{1}{2\forall_T}\int_{\forall}|\mathbf{V}|^2\,d\forall \approx \frac{1}{2\forall_T}\sum_{i=1}^{N_{elm}}|\mathbf{V}|^2\,\forall_i = \frac{1}{2N_{elm}}\sum_{i=1}^{N_{elm}}|\mathbf{V}|^2\,, \tag{4.1}$$

where N_{elm} is the total number of mesh elements, given by $N_x \times N_y$, and $\forall_T$ denotes the total volume of fluid within the cavity, expressed as $l_x \times l_y \times 1$.

Proceeding with the spatial convergence analysis, four uniform computational grids $G_i\{N_i\}$ were used, each with N_i elements per direction and a constant refinement ratio of $r = N_{i+1}/N_i = 2$ between successive grids. In our studies, we have used $N_1 = 16$, $N_2 = 32$, $N_3 = 64$, and $N_4 = 128$ elements per direction, which corresponds to a total of 256, 1024, 4096, and 16,384 mesh

elements, respectively. For each simulation, we consider an end time of , which corresponds to the time at which the flow reaches steady state [72], requiring 10,000 iterations. The iteration count was determined by the CFL condition for G_4 at $Re = 100$, with a safety factor of ≈ 1.5 applied.

Table 4.2 summarizes the $\overline{E}_k$ and velocity results obtained with our code and those reported by Ghia et al. [67] and Khorasanizade and Joao Sousa [71]. An observed order of grid convergence of 1.469 and 2.115 was obtained for the sets S_1 and S_2, respectively (see also Figure 4.4). These results are consistent with the theoretical second-order accuracy of the spatial scheme. Both sets of meshes extrapolate the same continuum value, $\overline{E}^*_{k,h_0} = 0.0343$, indicating that the mean kinetic energy approaches this value as the grid spacing tends to zero. This result is in agreement with the reference data from Ghia et al. [67] and Khorasanizade and Joao Sousa [71]. According to NASA Glenn Research Center [74], the error order for Richardson-extrapolated values is one unit higher than the theoretical order, which implies that the final extrapolated results have a third order error.

Next, we evaluate the fractional error E_1 for each grid set, defined as

$$E_1 = \frac{\varepsilon}{r^{\hat{p}} - 1} = \frac{\overline{E}^*_{k,2} - \overline{E}^*_{k,1}}{\overline{E}^*_{k,1}\,(r^{\hat{p}} - 1)}\,, \tag{4.2}$$

where indices 1 and 2 correspond to the finest and medium grids of each set. This quantity serves as an ordered error estimator and provides a good approximation of the discretization error on the fine grid, if G_3 and G_4, for the respective sets S_1 and S_2, were computed with sufficient accuracy, i.e., $E_1 < 1$. As observed in Table 4.2, this condition is satisfied. Note that the relative error (ε) alone should not be used as an error estimator, since it does not account for the refinement ratio r or the observed order p, which may lead to underestimation or overestimation of the error [74].

Moreover, we compute the Grid Convergence Index (GCI) for the finest (GCI_{12}) and coarser (GCI_{23}) grid pairs in both sets S_1 and S_2. The GCI measures the percentage deviation of the computed solution from the asymptotic value, providing an error band that indicates how much the solution would change with further grid refinement [74]. To correctly assess this distance, we use the Asymptotic Rate of Convergence (ARC) metric, defined as

$$ARC = \frac{CGI_{23}}{r^{\hat{p}} \cdot CGI_{12}}\,. \tag{4.3}$$

Since both sets yield small ARC values, that is, close to unity, we conclude that the computations are within the asymptotic range.

Table 4.2: Richardson-extrapolation analysis for the mean kinetic energy of the two-dimensional lid-driven cavity flow case study. Maximum and minimum velocity components are also shown for comparison against Ghia et al. (1982) and Khorasanizade and Sousa (2014) results.

	Present				Ghia et al. [67]	Sousa [71]
$N_i = N_x = N_y$	16	32	64	128		
$\overline{E}_k^*$	0.02903	0.03265	0.03386	0.03421	0.034 [76]	0.0333
$\hat{p}$	-	-	1.469	2.115	-	-
$\overline{E}_{k,h0}^*$	-	-	0.0343	0.0343	-	-
$\left\|\overline{E}_k^* - \overline{E}_{k,h0}^*\right\| \times 10^3$	5.27	1.65	0.437	0.0814	-	-
E_1	-	-	-1.29%	-0.24%	-	-
GCI_{12}	-	-	3.868%	0.7186%	-	-
GCI_{23}	-	-	18.81%	3.21%	-	-
ARC	-	-	1.756	1.032	-	-
u_{min}	-0.1950	-0.2084	-0.2123	-0.2132	-0.2109	-0.2024
y_{min}	0.4688	0.4531	0.4609	0.4570	0.4531	-
v_{min}	-0.2377	-0.2476	-0.2515	0.2525	-0.2453	-0.2368
x_{min}	0.8438	0.8281	0.8047	0.8086	0.8047	-
v_{max}	0.1618	0.1742	0.1774	0.1782	0.1753	-
x_{max}	0.2188	0.2344	0.2422	0.2383	0.2344	0.1669

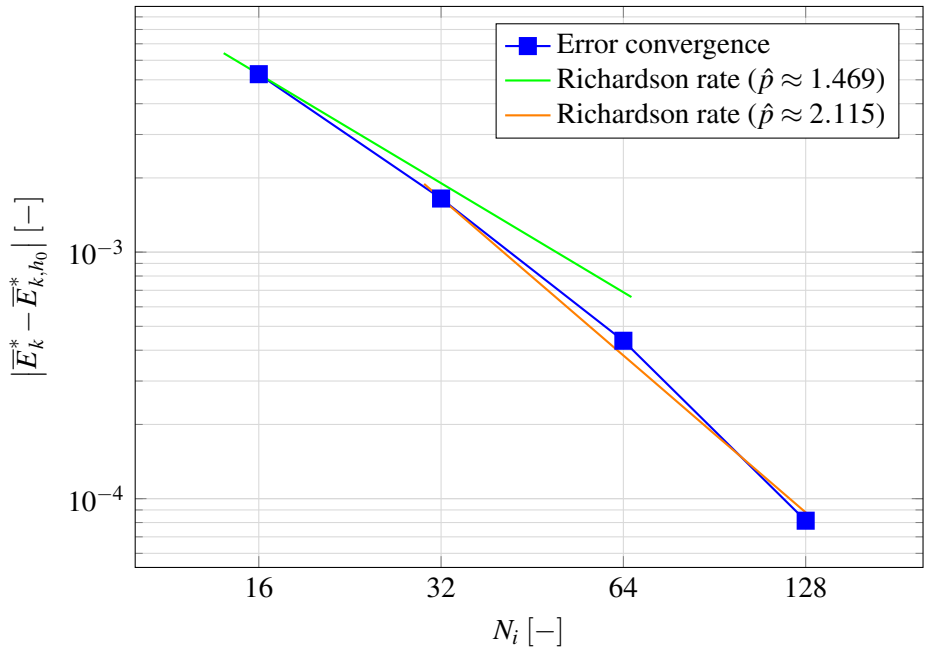

Figure 4.4: Order of convergence for the mean kinetic energy error $|\overline{E}^*_k - \overline{E}^*_{k,h_0}|$ with respect to grid resolution. The semi-log scale demonstrates the second-order convergence rate, as the error decreases approximately quadratically with increasing grid resolution.

Table 4.3 shows the relative errors of the mean kinetic energy compared to the reference data of Ghia et al. [67].

Table 4.3: Relative errors between $\overline{E}^*_k$ and data from Ghia et. al (1982) for G_1, G_2, G_3 and G_4

	$G_1\{16\}$	$G_2\{32\}$	$G_3\{64\}$	$G_4\{128\}$
ε^r_{ghia}	-14.62%	-3.97%	-0.41%	0.66%
ε^r_{ks}	-12.58%	-1.91%	1.65%	2.68%

When comparing the velocity profiles in the mid-planes, starting with the u^*-velocity along the x^*-centerline, the results show good agreement with the reference data from Ghia et al. [67], as illustrated in Figure 4.5, except for the coarsest grid $G_1\{16\}$. Table 4.4 shows the relative error between u^*_{min}, y^*_{min} and the data from [67] at $x^* = 0.5$.

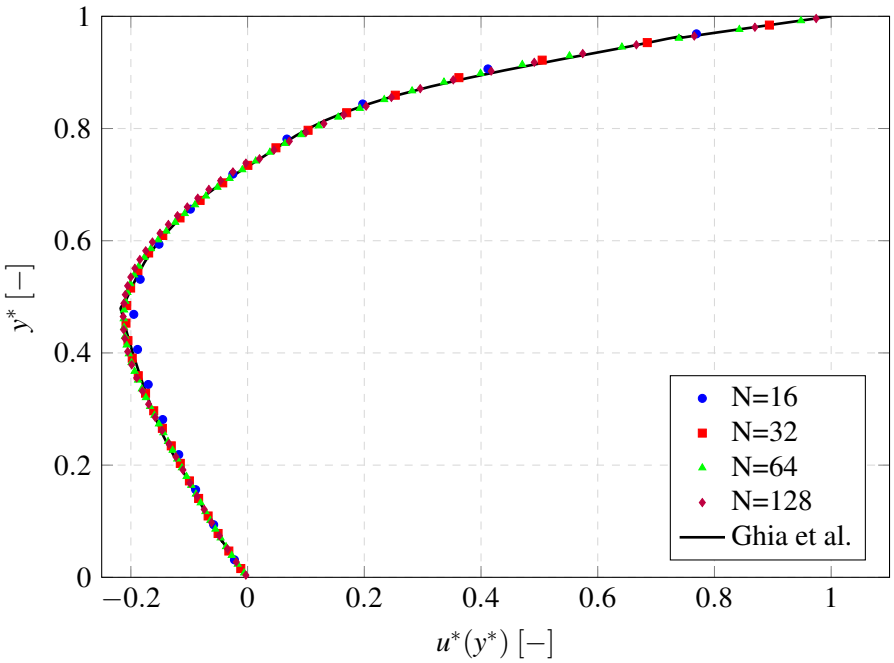

Figure 4.5: Vertical velocity profile ($u^*(y^*)$) along the cavity x^*-centerline plane for different grid resolutions. The results show convergence toward the reference data from Ghia et al. (1982) as grid resolution increases.

Table 4.4: Relative errors between u^*_{min} and its position y^*_{min} and the data from Ghia et al. (1982) at $x^* = 0.5$ plane

	$G_1\{16\}$	$G_2\{32\}$	$G_3\{64\}$	$G_4\{128\}$
$\varepsilon^r_{u^*_{min}}$	-7.55%	-1.20%	0.66%	1.09%
$\varepsilon^r_{y^*_{min}}$	3.45%	0.01%	1.73%	0.87%

For the v^*-velocity along the y^* centerline, similar conclusions hold, as shown in Figure 4.6. Table 4.5 reports the relative errors for v^*_{min}, v^*_{max}, and their respective positions.

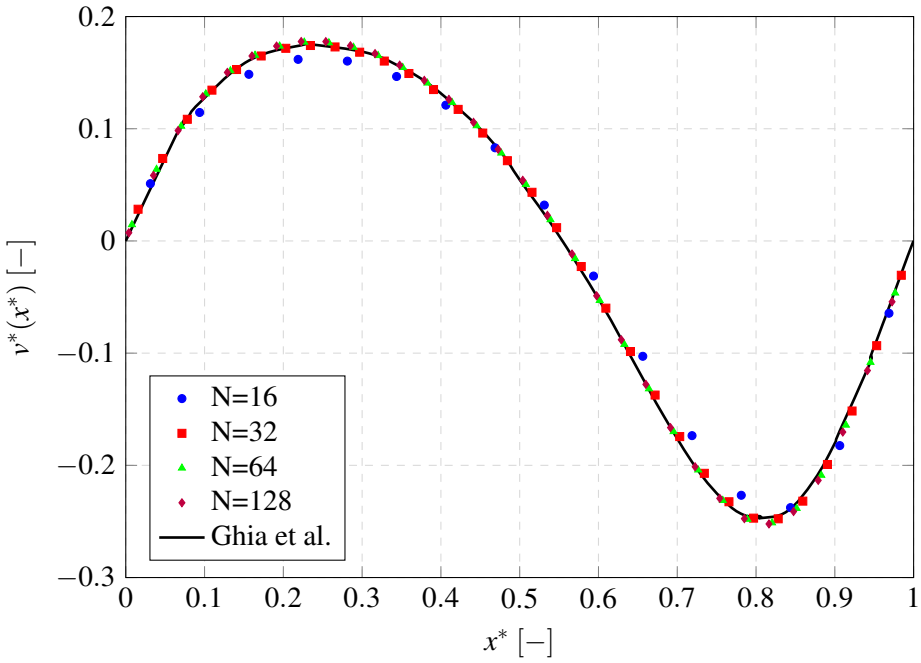

Figure 4.6: Horizontal velocity profile ($v^*(x^*)$) along the cavity y^*-centerline plane for different grid resolutions. The results show convergence toward the reference data from Ghia et al. (1982) as grid resolution increases.

Table 4.5: Relative errors for v^*_{min}, v^*_{max} and its respective position x^*_{min} and x^*_{max} with the data from Ghia et al. (1982) at $y^* = 0.5$

	$G_1\{16\}$	$G_2\{32\}$	$G_3\{64\}$	$G_4\{128\}$
$\varepsilon^r_{v^*_{min}}$	-3.13%	0.94%	2.53%	2.90%
$\varepsilon^r_{x^*_{min}}$	4.86%	2.91%	0.00%	0.48%
$\varepsilon^r_{v^*_{max}}$	-7.67%	-0.62%	1.20%	1.65%
$\varepsilon^r_{x^*_{max}}$	-6.66%	0.00%	3.33%	1.66%

As a final note, the non-monotonic trends in Tables 4.4 and 4.5 are due to the local nature of these velocity extrema. Their positions vary with grid refinement, so a direct comparison with the reference data (based on a 129×129 mesh) may not fully reflect the true convergence. The aim of these results is to confirm that our solutions are consistent with known reference data.

Lastly, in Figure 4.7, we show the contour plot for the velocity magnitude and the divergence of the velocity field at $Re = 100$ with mesh 128×128. Moreover, the stream-lines plot is also shown in Figure 4.8. The moving lid drives a large, central primary vortex that dominates most of the cavity interior. At this Reynolds number, small, weak secondary vortices are just beginning to form in the bottom corners of the cavity. Strong shear layers are visible near the moving top lid and the vertical side walls. The divergence of the velocity field in the domain is of the order

of 10^{-9}, demonstrating that the projection method implemented within the IMEX time integration scheme provides divergence-free solutions with physical meaning.

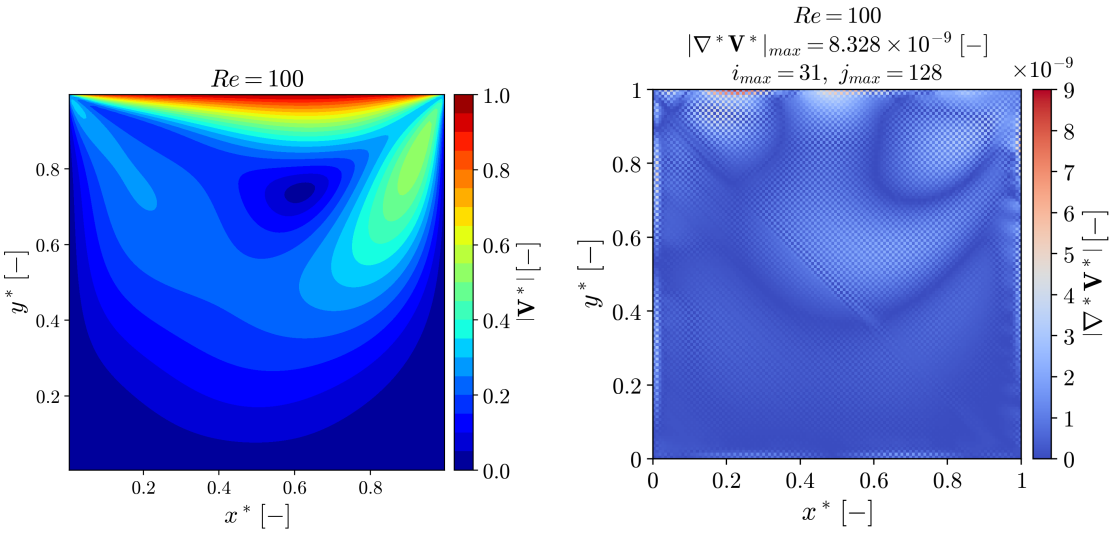

Figure 4.7: On the left, the velocity magnitude contour plot. On the right, the divergence of the velocity field. Results for a grid of 128×128 elements and $Re = 100$.

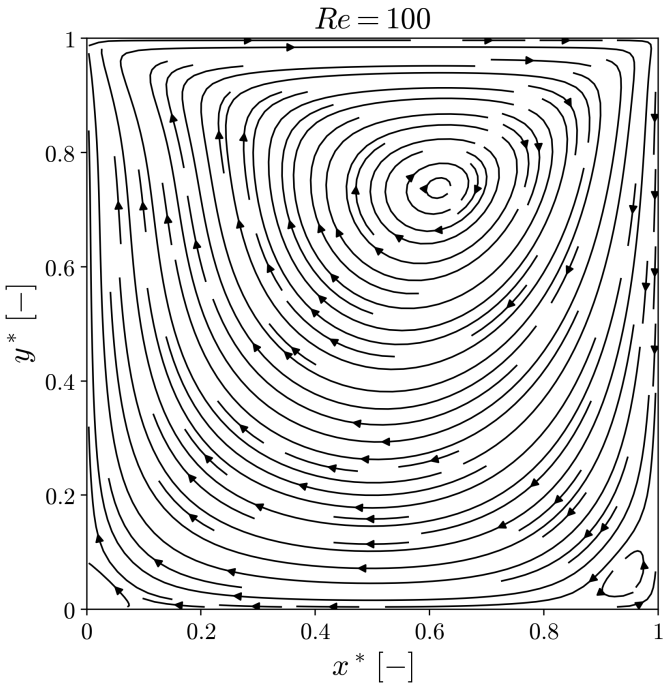

Figure 4.8: Streamline plot of the lid-driven cavity flow for a grid of 128×128 elements and $Re = 100$.

4.2.3 Temporal Convergence Analysis

We proceed into the temporal convergence analysis, considering the following uniform grids $G_1\{16\}$ and $G_2\{32\}$. The quantities of interest were evaluated using five successive time-step sizes Δt_i^* with a refinement ratio of 2, over a simulation period of 10 s. For G_1, the time-step sets were defined as

$$T_1^{G1} = \{0.1, 0.05, 0.025\}, \quad T_2^{G1} = \{0.05, 0.025, 0.0125\}, \quad T_3^{G1} = \{0.025, 0.0125, 0.00625\}.$$

Similarly, for G_2, we used,

$$T_1^{G2} = \{0.02, 0.01, 0.005\}, \quad T_2^{G2} = \{0.01, 0.005, 0.0025\}, \quad T_3^{G2} = \{0.005, 0.0025, 0.00125\}.$$

Figure 4.9 shows the results obtained for the order of convergence along simulation time for different groups of time step refinements using the RK4 time integration scheme. For the RK4 scheme on a 16×16 mesh (left panel), considering the first set of time steps T_1^{G1} (as defined in Section 4.2), the order decreases from ≈ 4.6 to 4.4, up to a simulation time of 1. For the second set, T_2^{G1}, the order decreases from roughly 4.3 to 4.2 up to $t \approx 0.7$, after which it exhibits minor oscillations around 4.2. For the 32×32 mesh (right panel), a similar trend is observed for all three temporal sets. The orders vary as follows: $T_1^{G2} \approx 4.50 - 4.30$, $T_2^{G2} \approx 4.23 - 4.14$, and $T_3^{G2} \approx 4.12 - 4.00$.

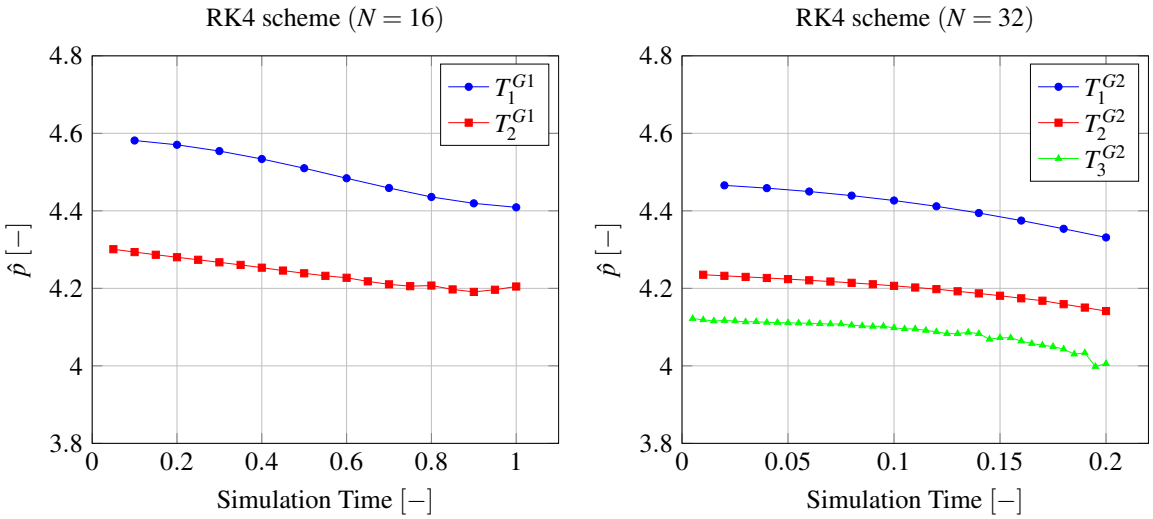

Figure 4.9: Temporal order $\hat{p}$ vs. simulation time for different mesh elements N with RK4 time integration scheme. The panel on the left is relative to the grid $G_1\,(16)$ and the temporal sets $T_1^{G1} = \{0.1, 0.05, 0.025\}$ and $T_2^{G1} = \{0.05, 0.025, 0.0125\}$. The panel on the right, is is relative to the grid $G_2\,(32)$ and the temporal sets $T_1^{G2} = \{0.02, 0.01, 0.005\}$, $T_2^{G2} = \{0.01, 0.005, 0.0025\}$ and $T_3^{G2} = \{0.005, 0.0025, 0.00125\}$.

The simulation time is limited to $t^* = 1$ for $G_1\{16\}$ and $t^* = 0.2$ for $G_2\{32\}$. Beyond these points, the solutions obtained from successive time-step refinements no longer differ, indicating that temporal convergence is achieved. This explains the absence of T_3^{G1} in the left panel: for

coarse grids, the high-order RK4 method reaches temporal convergence almost immediately, while finer grids, such as $G_2\{32\}$, require smaller time steps. Although all grids converge to the same steady state value, coarser grids converge faster in time but provide less accurate solutions, as illustrated in Figure 4.10.

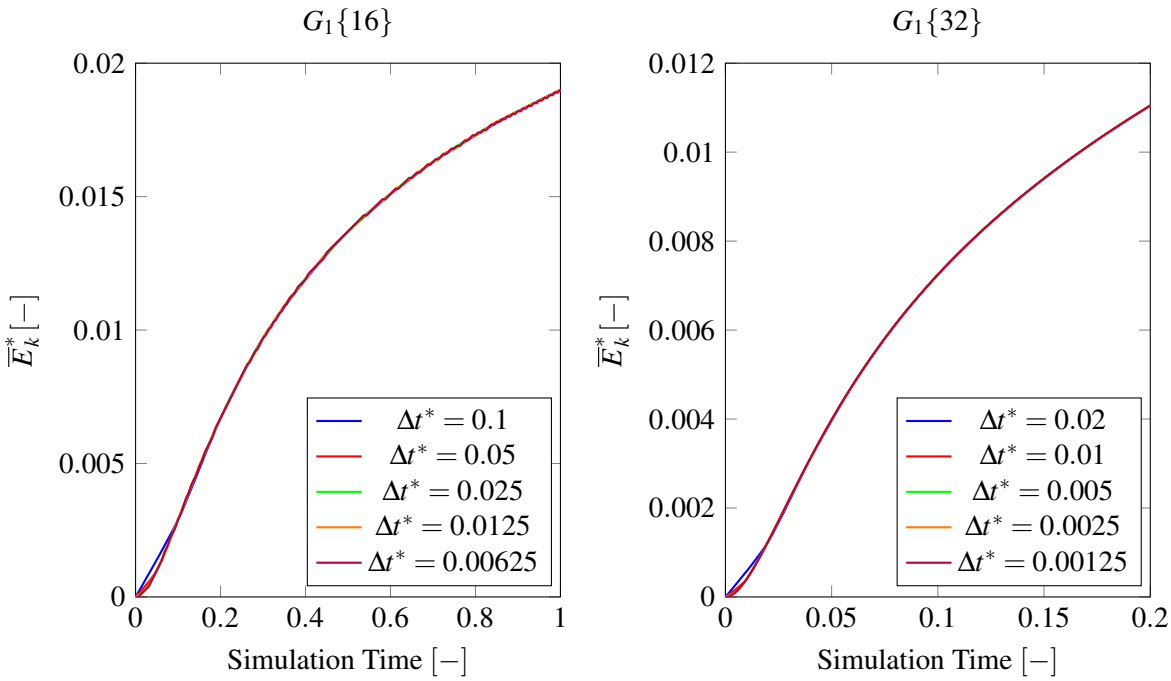

Figure 4.10: Mean non-dimensional kinetic energy $\left(\overline{E}_k^*\right)$ evolution over time for different time steps using the RK4 time integration scheme. The panel on the left uses $G_1\{16\}$, and the panel on the right uses $G_1\{32\}$. The curves show convergence behavior as Δt^* decreases.

Figure 4.11 shows the extrapolated value of the mean non-dimensional kinetic energy, $\overline{E}_{k,h0}^*$, over time for the RK4 time integration scheme.

A similar analysis for the IMEX time integration schemes, ASC(2,2,2) and ASC(4,4,3), was carried out as seen in Figures 4.12 and 4.13, respectively.

We conclude that ASC(2,2,2) and ASC(4,4,3) exhibit different temporal-order behaviors. In Figure 4.12, ASC (2,2,2) on the grid 16×16 shows a consistent decay in order across all temporal sets, with the maximum value occurring at the start of the simulation. Specifically, T_1^{G1}, T_2^{G1}, and T_3^{G1} start at approximately 2.13, 2.04, and 2.01, respectively. Both T_2^{G1} and T_3^{G1} converge to nearly the same value, close to 2. Although T_1^{G1} begins with the highest order, it decreases to the lowest observed value of about 1.99. On the 36×36 grid, a similar trend is observed, but with smaller discrepancies. In this case, T_1^{G1} and T_2^{G1} converge to 2, while T_3^{G1} stabilizes at a slightly higher value. Notably, T_3^{G1} displays an increase–decrease behavior, while the other sets follow a purely decreasing trend.

In contrast, ASC(4,4,3) achieves higher order values compared to ASC(2,2,2) as expected, since its theoretical order is three, while ASC(2,2,2) is only two. In addition, ASC (4,4,3) exhibits

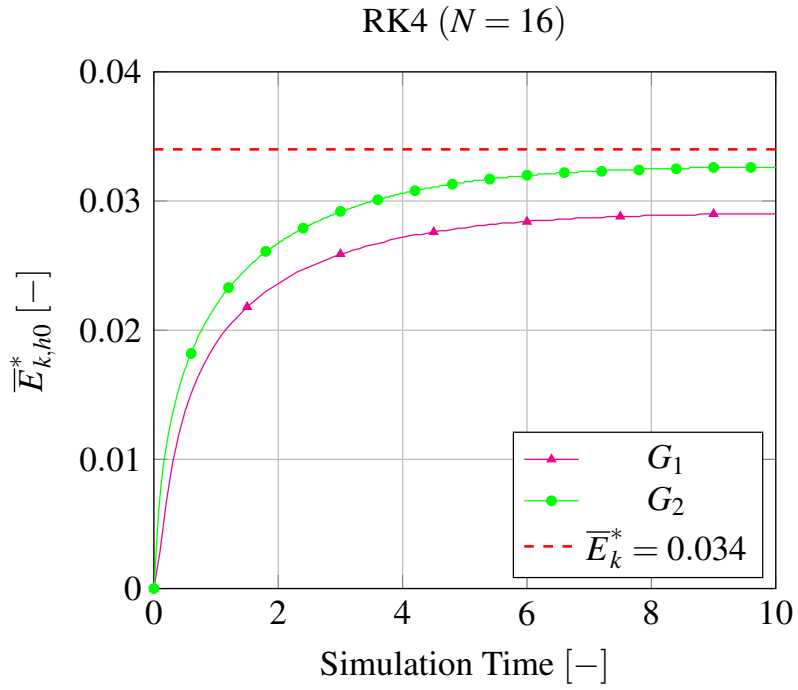

Figure 4.11: Evolution of $\overline{E}^*_{k,h0}$ over time for the RK4 time integration scheme.

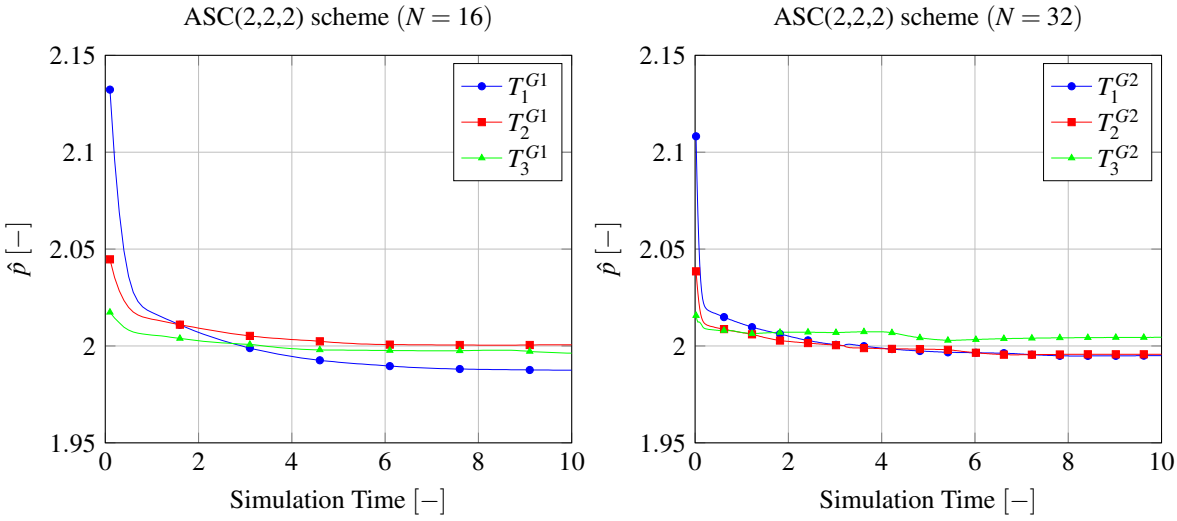

Figure 4.12: Temporal order $\hat{p}$ vs. simulation time for different mesh elements N with IMEX ASC(2,2,2) time integration scheme. The panel on the left is relative to the grid $G_1\,(16)$ and the temporal sets $T_1^{G1} = \{0.1, 0.05, 0.025\}$, $T_2^{G1} = \{0.05, 0.025, 0.0125\}$ and $T_3^{G1} = \{0.025, 0.0125, 0.00625\}$. The panel on the right is relative to the grid $G_2\,(32)$ and the temporal sets $T_1^{G2} = \{0.02, 0.01, 0.005\}$, $T_2^{G2} = \{0.01, 0.005, 0.0025\}$ and $T_3^{G2} = \{0.005, 0.0025, 0.00125\}$.

a more consistent behavior over time, without crossing among the curves of different temporal sets. The order of T_2^{G1} remains bounded between those of T_1^{G1} (upper) and T_3^{G1} (lower). However, we observe a significant reduction in the observed order compared to the theoretical value of ASC (4,4,3). In addition, the temporal order curve of ASC(4,4,3) is characterized by a minimum

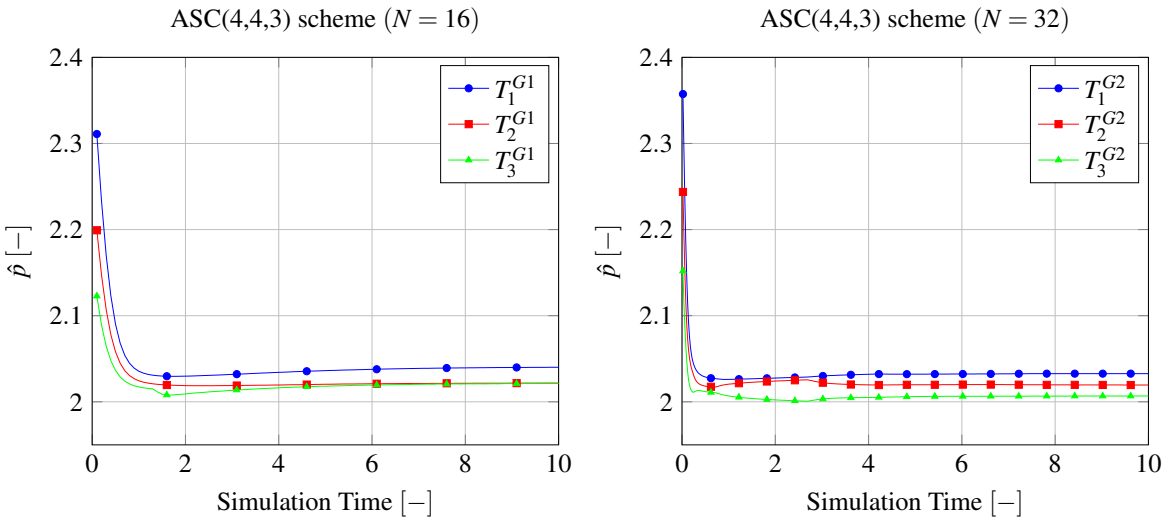

Figure 4.13: Temporal order $\hat{p}$ vs. simulation time for different mesh elements N with IMEX ASC(4,4,3) time integration scheme. The panel on the left is relative to the grid $G_1\,(16)$ and the temporal sets $T_1^{G1} = \{0.1, 0.05, 0.025\}$, $T_2^{G1} = \{0.05, 0.025, 0.0125\}$ and $T_3^{G1} = \{0.025, 0.0125, 0.00625\}$. The panel on the right is relative to the grid $G_2\,(32)$ and the temporal sets $T_1^{G2} = \{0.02, 0.01, 0.005\}$, $T_2^{G2} = \{0.01, 0.005, 0.0025\}$ and $T_3^{G2} = \{0.005, 0.0025, 0.00125\}$.

at approximately $t^* = [0.25, 1]$, after which it stabilizes (following a horizontal tendency) much faster than the other IMEX scheme.

Figures 4.14 and 4.15 show the evolution of the mean dimensionless kinetic energy with time for both IMEX schemes. Unlike the RK4, the curves remain separated for a longer time before approaching the same value, reflected by the lower-order character of these schemes.

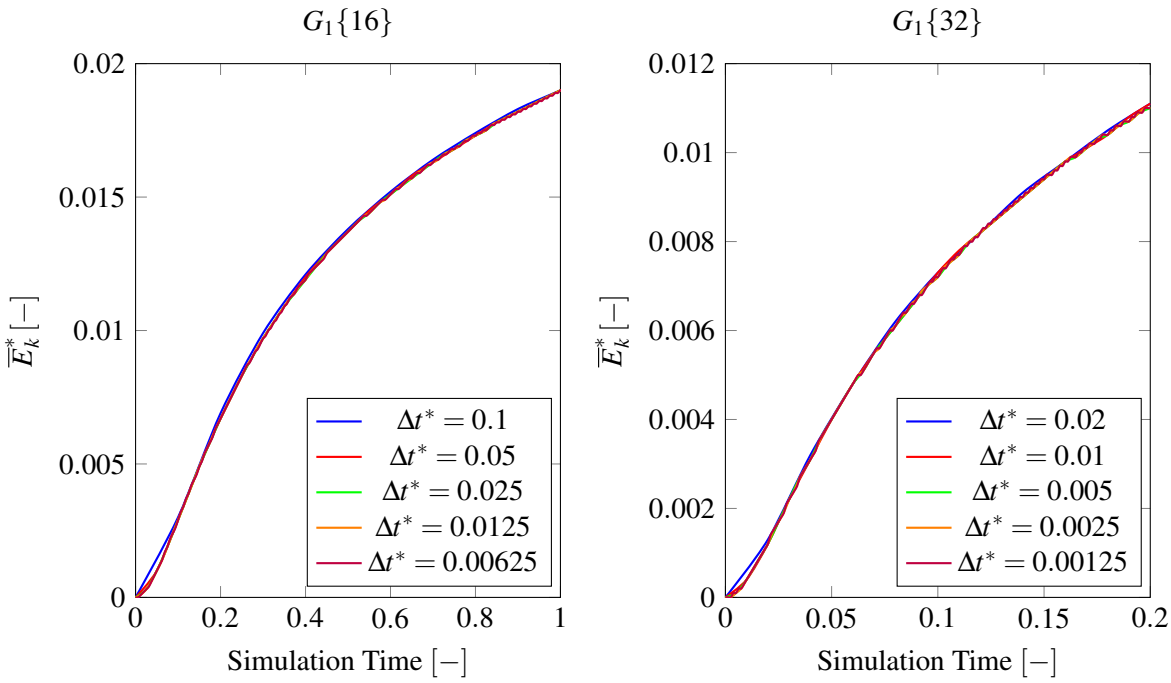

Figure 4.14: Mean non-dimensional kinetic energy $\left(\overline{E}_k^*\right)$ evolution over time for different time steps using the ASC(2,2,2) time integration scheme. The panel on the left uses $G_1\{16\}$, and the panel on the right uses $G_1\{32\}$. The curves show convergence behavior as Δt^* decreases.

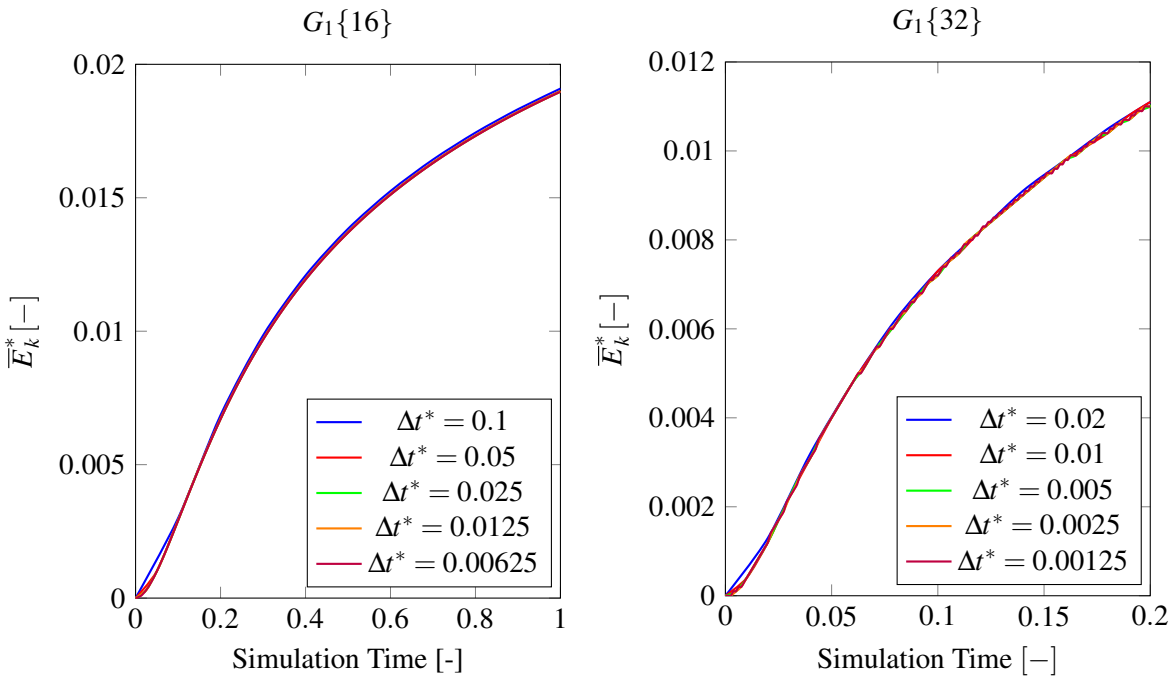

Figure 4.15: Mean non-dimensional kinetic energy $\left(\overline{E}_k^*\right)$ evolution over time for different time steps using ASC(4,4,3) time integration scheme. The panel on the left uses $G_1\{16\}$, and the panel on the right uses $G_1\{32\}$. The curves show convergence behavior as Δt^* decreases.

Figure 4.16 shows the extrapolated value of the mean non-dimensional kinetic energy error, $\overline{E}^*_{k,h0}$, over time for the IMEX time integration schemes.

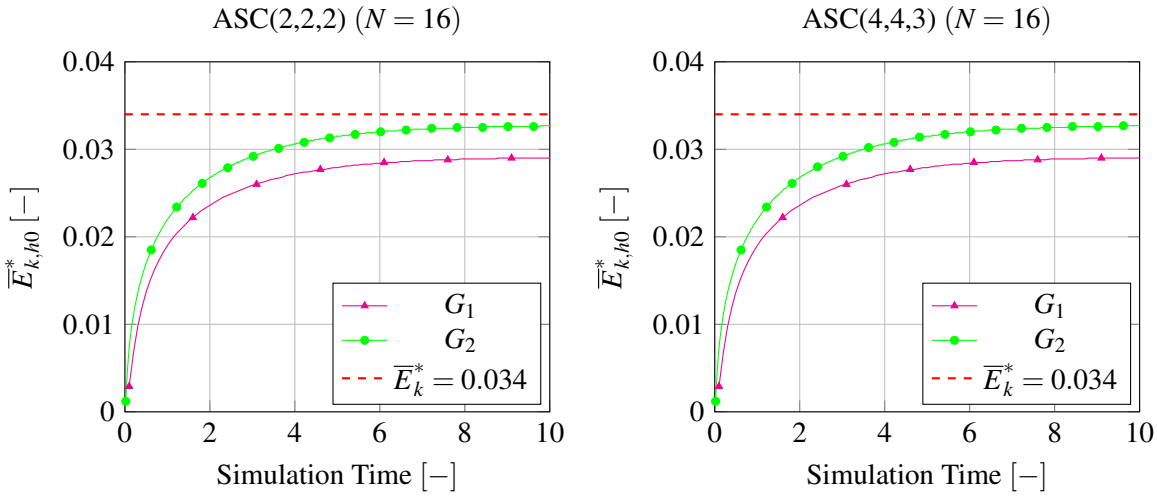

Figure 4.16: Evolution of $\overline{E}^*_{k,h0}$ over time for different IMEX time integration schemes: ASC(2,2,2) (left) and ASC(4,4,3) (right).

Figures 4.11 and 4.16 show that the time-interpolated solution converges to the steady spatial-interpolated solution.

4.2.4 Time-step Sensibility

To evaluate the temporal gains obtained by applying implicit-explicit (IMEX) time integration schemes, we measure the maximum admissible time step, Δt^*_{max}, which avoids numerical divergence of the simulations. Moreover, due to their stability properties (see Section 2.2.4), IMEX schemes are not constrained by viscous effects and are instead limited only by the convective time-step, which was estimated using a trial-and-error procedure, with an initial guess given by the classical CFL condition for advection,

$$\Delta t^*_\lambda \leq \frac{\Delta x}{U_{lid}}. \tag{4.4}$$

Although this approach is somewhat limited, the lack of a well-defined CFL condition for these schemes makes a direct assessment more difficult. We then run simulations on each grid, $G_1\{16\}$, $G_2\{32\}$, $G_3\{64\}$ and $G_4\{128\}$, using the RK4 method with its stable time step, Δt^*_λ , derived from the CFL condition defined by Saad and Karam [50].
In addition, for the RK4 time integration scheme, under the assumed $Re = 100$, the viscous CFL condition dominates over the convective condition for all tested grids. Saad and Karam [50] de-

fined a stable time step as

$$\Delta t_{\lambda}^{*} \leq \min \left(\frac{Re}{1.45 \left(\frac{1}{\Delta x^{*2}} + \frac{1}{\Delta y^{*2}} \right)}, \frac{2\sqrt{2}\Delta x^{*}}{U_{lid}^{*2}} \right). \tag{4.5}$$

The results of the stable time-step obtained using the RK4 for different grid resolutions are shown in Table 4.6. These results are used as a reference for comparison with those obtained with the IMEX time integration schemes shown in Table 4.7.

Table 4.6: Results for the stable time-step using the RK4 time integration scheme with different grid resolutions

	RK4			
$N_x = N_y$	16	32	64	128
Δt_{λ}^{*}	0.133	0.0337	0.0084	0.0021
t_{wall} [s]	1.21	9.61	156.17	2331.86
$\overline{E}_{k}^{*}$	0.02903	0.03265	0.03386	0.03421
u_{min}^{*}	−0.1950	−0.2084	−0.2115	−0.2132
v_{min}^{*}	−0.2377	−0.2476	−0.2515	−0.2525
v_{max}^{*}	0.1603	0.1742	0.1774	0.1781

Table 4.7: Results for the stable time-step using the IMEX time integration schemes with different grid resolutions

	ASC(2,2,2)				ASC(4,4,3)			
Nx=Ny	16	32	64	128	16	32	64	128
Δt_{max}^{*}	0.3125	0.1333	0.0076	0.00434	0.5	0.167	0.05	0.01
t_{wall} [s]	0.43	3.08	35.92	1003.84	0.48	3.93	47.56	1126.01
$\overline{E}_{k}^{*}$	0.02954	0.03323	0.03416	0.03425	0.03233	0.03463	0.03465	0.03424
u_{min}^{*}	−0.1883	−0.2082	-0.2122	−0.2131	−0.1970	−0.2086	−0.2123	−0.2132
v_{min}^{*}	−0.2369	−0.2469	−0.2512	−0.2522	−0.2434	−0.2486	−0.2519	−0.2487
v_{max}^{*}	0.1625	0.1742	0.1773	0.1782	0.1668	0.1756	0.1777	0.1781

We conclude that increasing the time step to that of the RK stability limit reduces the accuracy of the mean kinetic energy. This occurs because larger time steps decrease the temporal resolution of the simulation, causing small-scale effects (such as the corner vorticity structures) to be poorly captured. As shown in Figure 4.17, these regions are not properly resolved, leading to errors in the overall kinetic energy balance.

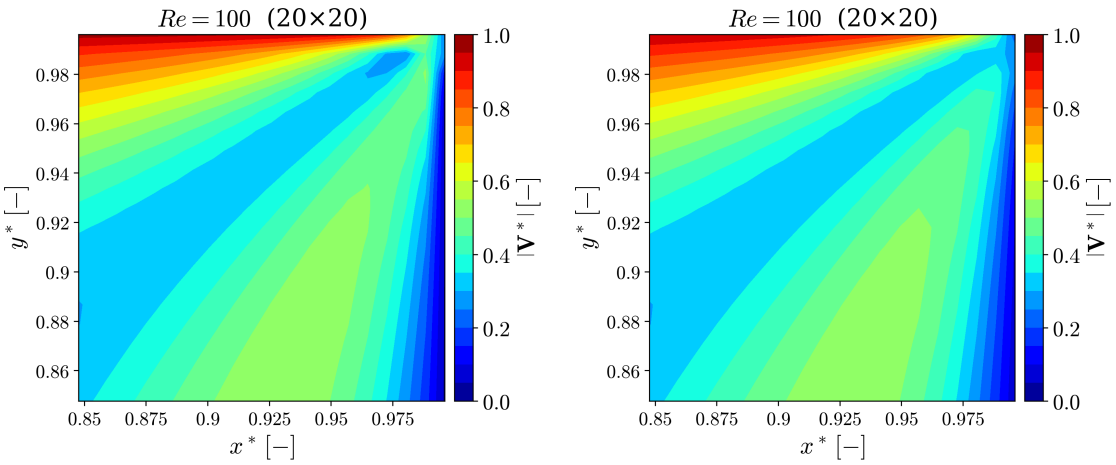

Figure 4.17: Comparison between the velocity magnitude in a zoomed region at the corner of the lid-driven cavity when using the IMEX(4,4,3) time integration scheme with $\Delta t^* = 0.01$ (left panel) and the RK4 time integration scheme with $\Delta t^* = 0.0021$ (right panel), for a 128×128 grid size at $t^* = 10$.

This effect can be mitigated with grid refinement. Finer grids, such as 256×256, allow for larger relative time-step increases (Δt^{*+}) when compared to coarser grids, without compromising accuracy. Table 4.8 shows the % increase in the time-step value and the respective accuracy level of the mean kinetic energy and velocity components extrema.

Table 4.8: Comparison of the % increase of the time-step value employed on different grid-sizes when using the IMEX time integration schemes. The percentage reduction of the computational wall time of the simulations is also calculated. The relative errors for the mean kinetic energy and velocity components extrema are also shown.

	ASC (2,2,2)				ASC(4,4,3)			
$N_x = N_y$	16	32	64	128	16	32	64	128
Δt^{*+}	135%	296%	417%	107%	276%	396%	495%	376%
t^-_{wall} [s]	64%	68%	77%	57%	60%	59%	70%	52%
$\varepsilon^r_{\overline{E}_k}$	1.76%	1.78%	0.89%	0.12%	11.37%	6.06%	2.33%	0.09%
$\varepsilon^r_{u_{min}}$	3.44%	0.10%	0.33%	0.047%	1.03%	0.10%	0.38%	0.00%
$\varepsilon^r_{v_{min}}$	0.34%	0.28%	0.12%	0.12%	2.40%	0.40%	0.16%	1.50%
$\varepsilon^r_{v_{max}}$	1.37%	0.00%	0.06%	0.06%	4.05%	0.80%	0.17%	0.00%

Note that we define accuracy as the relative error computed between the IMEX solution and the stable RK4 solution. Moreover, we confirm a good agreement between computation time reduction and accuracy for the IMEX time integration scheme. Hence, to further evaluate them, Figure 4.18 shows the accuracy loss per time unit saved.

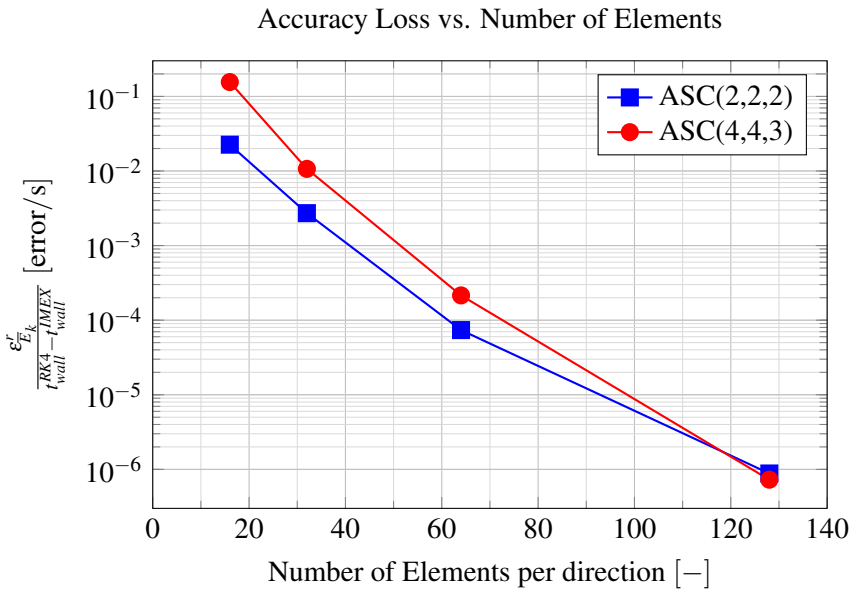

Figure 4.18: Accuracy loss per unit of time saved as a function of the number of elements. The plot uses a logarithmic scale on the y-axis to visualize the wide range of values across different element counts.

In conclusion, as we apply mesh refinement, the IMEX tends to be more viable. The values of this metric achieved by both IMEX schemes are closely comparable. Consequently, the determination of which scheme to use is largely dependent on the specific order of accuracy that is necessitated by the application at hand. For instance, since *SP-Wind* works by default with a temporal scheme of 4^{th} order, we select ASC(4,4,3), so as not to have greater order losses. Moreover, by further extrapolating the trend observed in these error lines, it can be deduced that as refinements become more substantial, the rate of accuracy degradation per unit of time saved associated with the IMEX(2,2,2) scheme will eventually surpass that of the IMEX(4,4,3) scheme. Nonetheless, the mean velocity profiles along the mid-planes remain accurate, with errors on the order of $\sim 1\%$. Overall, the stable time-step for IMEX schemes is approximately one order of magnitude larger than that of RK4.

4.3 Cilia Flow (3D)

Lastly, we present the results obtained for the three-dimensional cilia flow. The simulation was performed at $Re_{D^*} = 0.1$ with an end time of 4, sufficient to reach a steady state. The grid was set to $N_x = 36$, $N_y = 36$, and $N_z = 56$ elements, corresponding to a coarse resolution, simulated using 1 node of 36 cores. The resulting time step was approximately 10^{-5}, leading to a simulation wall time of approximately 12 hours. Notice that this time-step is calculated using the viscous CFL condition, Equation (2.72), with Courant number equal to 0.4. We verify that going further than this value leads to numerical divergence. For non-steady simulations, such as modeling ciliary

motion, this restrictive time step would render the simulation unfeasible, making it inefficient to expend high computational resources on such physically simple flows.

Nevertheless, the results indicate promising behavior. As shown in Figure 4.19, the cilium is represented in the domain at approximately $(x^*_{cil}, y^*_{cil}) \approx (2.5, 3)$. The slight discrepancy in position is due to the coarse grid, which may not include a grid point at the exact location of the cilium. Despite this, the interaction between the cilium and the flow is clearly visible. The velocity magnitude also exhibits symmetry along the plane $y = 3$, reflecting the ordered and symmetric nature of low-Reynolds number flows.

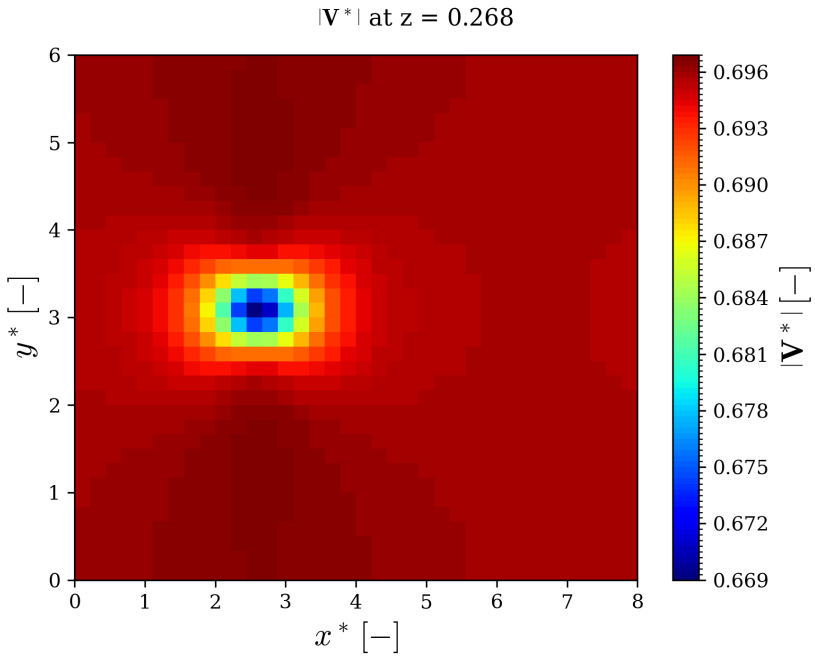

Figure 4.19: Contour of the velocity magnitude along the plane $z^* = 0.268$, with $Re_{D^*} = 0.1$, $N_x = 36$, $N_y = 36$, and $N_z = 56$, for the three-dimensional cilia flow using the RK4 time integration scheme.

Furthermore, in Figure 4.20, we confirm that the inlet velocity profile is being constructed correctly.

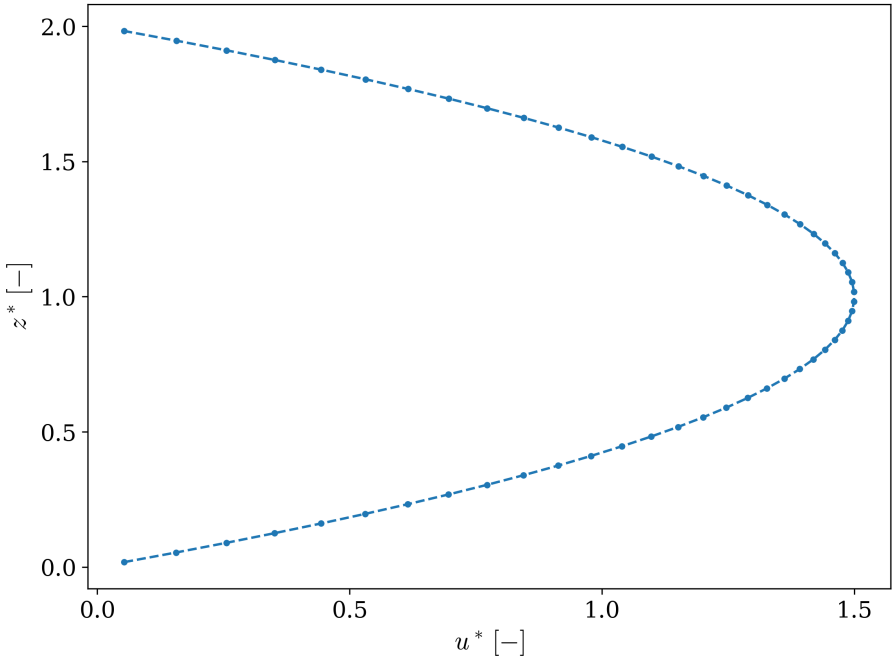

Figure 4.20: Inlet velocity profile at the latest time-step ($t^* = 4$), with $Re_{D^*} = 0.1$, $N_x = 36$, $N_y = 36$, and $N_z = 56$, for the three-dimensional cilia flow using the RK4 time integration scheme.

Next, we analyze the results obtained with the IMEX implementation for this cilia flow case study. To simplify the test, the body force was neglected in order to first verify whether the channel flow was correctly modeled. However, as shown in Figure 4.21, the velocity contour does not have a physical meaning.

Further examination revealed that the main issue is the incorrect enforcement of the imposed boundary condition near the top no-slip wall. Figure 4.22 shows the streamwise velocity profile, which confirms this discrepancy.

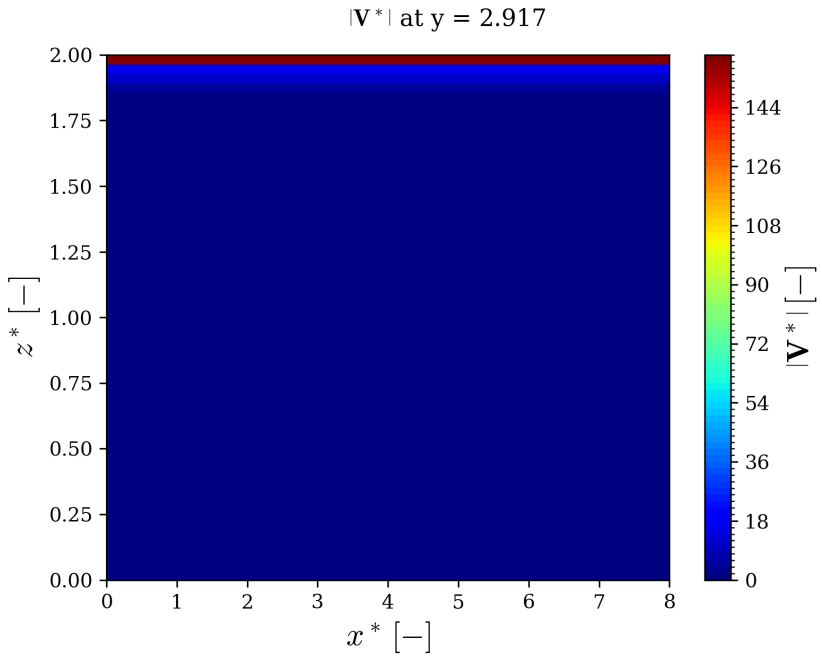

Figure 4.21: Contour of the velocity magnitude along the plane $y^* = 2.917$, with $Re_{D^*} = 0.1$, $N_x = 36$, $N_y = 36$, and $N_z = 56$, for the three-dimensional cilia flow using the IMEX(4,4,3) time integration scheme (after 3 iterations).

We hypothesize that the problem may be related either to the construction of the iteration matrix, Equation (2.51), or to halo communication within the linear solver for the stage velocities.

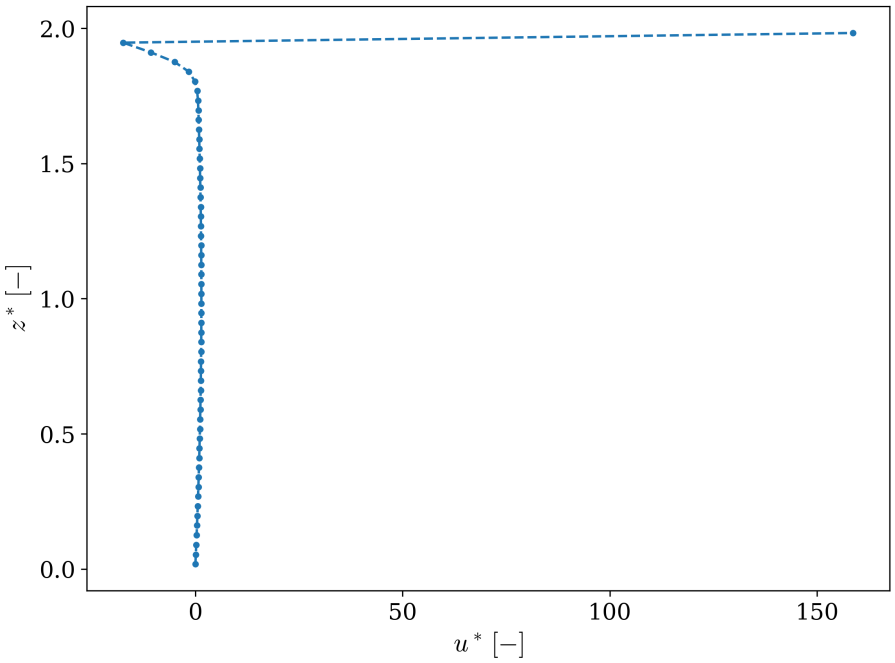

Figure 4.22: Inlet velocity profile, with $Re_{D^*} = 0.1$, $N_x = 36$, $N_y = 36$, and $N_z = 56$, for the three-dimensional cilia flow using the IMEX(4,4,3) time integration scheme (after 3 iterations).

The bottom no-slip boundary condition is correctly enforced. However, near the top wall an unexpected behavior is observed: the velocity profile crosses the y-axis, assumes negative values, and then overshoots at the last grid point to values greater than 150. This result is unphysical and indicates a numerical inconsistency in the solution. We conclude that the IMEX results do not preserve physical consistency, which poses a barrier to further studies using the current implementation.

Lastly, we evaluate the interaction between two cilia using the RK time integraton scheme on the same mesh. Figure 4.23 shows the velocity magnitude contour for the two cilia flow. We conclude that *SP-Wind* is able to interact the cilias.

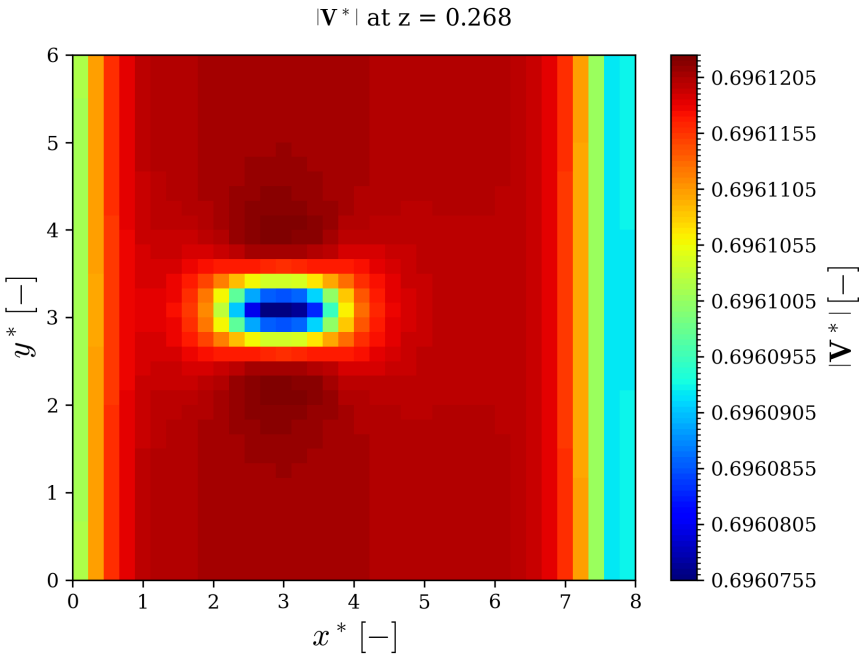

Figure 4.23: Contour of the velocity magnitude along the plane $z^* = 0.268$, with $Re_{D^*} = 0.1$, $N_x = 36$, $N_y = 36$, and $N_z = 56$, for the three-dimensional flow of two cilias using the RK4 time integration scheme.

Chapter 5

Conclusions and Future Work

The time-step restriction imposed by the viscous Courant–Friedrichs–Lewy condition renders the time steps used in the simulation of cilia-driven flows inadmissible for efficient computations. To perform viable time-dependent studies on cilia-driven flows, the time integration scheme must be one of implicit character.

Implicit-Explicit schemes constitute a good option, as they can eliminate these viscous restrictions, due to their unconditionally stable region along the negative real axis, without requiring the nonlinear advection term of the Navier-Stokes equations to be treated implicitly. However, unlike explicit temporal schemes, which allow the pressure correction step to be applied only at the advancing stage, IMEX schemes require it to be applied for all intermediate velocities and at the advancing step to avoid divergence. This, combined with the linear solver for the velocities, results in $2s+1$ linear systems to solve per time step.

Our implementation of IMEX time integration schemes was first verified with the one-dimensional Burgers' equation test case. We conclude that the IMEX schemes performed well for the advection-diffusion problem and respect the imposed periodic boundary conditions. The results obtained showed L^2 errors of the order of 10^{-3}, which can be considered acceptable.

Subsequently, the lid-driven cavity case study was used to test the IMEX implementation in a two-dimensional flow. The results obtained showed that physical behavior is maintained when using IMEX. The velocity profiles u and v along the respective x and y centerlines showed good agreement with the data from Ghia et al. [67], and the mean kinetic energy also aligns well with the results from Ghia et al. [67], and Khorasanizade and Joao Sousa [71]. Additionally, the order-of-convergence analysis performed for both spatial and temporal discretizations was found to be satisfactory and coherent. All three methods used (RK4, ASC (2,2,2), and ASC (4,4,3)) showed the same results in the grid convergence test, indicating that the nature of the steady-state solution is independent of the numerical time integration method. For the temporal order, all methods demonstrate an observable order close to the theoretical one and converge quickly at each time step. A temporal analysis showed that IMEX schemes allow larger time steps, making their implementation viable when compared to explicit methods, since the increase in admissible time step compensates for the additional computational complexity.

Although the final goal of implementing the IMEX time integration scheme in *SP-Wind* was not achieved, the main structure is in place and only the issues related to boundary conditions and/or halo communications remain to be solved.

This work has provided a deeper understanding of the discretized Navier-Stokes equations. Concepts that were unclear at the beginning, such as the pressure projection step, the finite-volume method, staggered grids, and order-of-convergence analysis, are now more familiar. Furthermore, I have gained confidence in working with more complex topics such as HPC environments, as well as *Fortran* and *Python* programming. As an engineer, I conclude this work with significantly more skills and confidence than when I started, which is, after all, the ultimate goal of this journey.

Future work should first address the boundary condition issues encountered with the IMEX implementation. Once resolved, the scheme can be tested for three-dimensional problems. If successful, the next step would be to simulate multiple cilia. A natural progression would be to start with two cilia aligned in the flow direction and spaced by a fixed distance, then extend to larger arrays with different spatial arrangements to study their interaction.

After validating static cilia interactions, the next stage would involve incorporating ciliary motion, following approaches such as those in Salman et al. [9], Hadi et al. [29] and Sedaghat et al. [30]. This would require modifying the inlet condition to a quiescent field and analyzing flow development from rest until reaching a statistically steady state. An initial case could involve a single cilium that undergoes a simple periodic back-and-forth motion in the x-direction. Once this is understood, the framework could be extended to multiple cilia and, ultimately, to more realistic beating patterns.

If the IMEX scheme proves inadequate for three-dimensional simulations, a fully implicit method may be required. Although this introduces challenges in handling the non-linear terms, established linearization strategies could provide a feasible path forward.

In summary, this dissertation set ambitious goals, some of which remain incomplete but are feasible for future exploration. With the current foundation, the path is open for advancing toward more complex and realistic studies of cilia-driven flows.

References

[1] B.R. Munson, D.F. Young, Ted Okiishi, and W.W. Huebsch. Fundamentals of fluid mechanics, sixth edition. *Differential Analysis of Fluid Flow, Ch. 6*, pages 166–173, 2009.

[2] C. B. Lindemann and K. A. Lesich. Flagellar and ciliary beating: the proven and the possible. *Journal of Cell Science*, 123(4):519–528, 2010. doi: 10.1242/jcs.051326.

[3] R. Luxmi and S. M. King. Cilia provide a platform for the generation, regulated secretion, and reception of peptidergic signals. *Cells*, 13(4), 2024. doi: 10.3390/cells13040303.

[4] Y. Imai T. Yamaguchi, T. Ishikawa. 5 - ciliary motion. In Takami Yamaguchi, Takuji Ishikawa, and Yohsuke Imai, editors, *Integrated Nano-Biomechanics*, Micro and Nano Technologies, pages 147–173. Elsevier, Boston, 2018. ISBN 978-0-323-38944-0.

[5] D. Chen, D. Norris, and Y. Ventikos. The active and passive ciliary motion in the embryo node: A computational fluid dynamics model. *Journal of Biomechanics*, 42(3):210–216, 2009. doi: https://doi.org/10.1016/j.jbiomech.2008.10.040.

[6] A. Decoene, S. Martin, and F. Vergnet. A continuum active structure model for the interaction of cilia with a viscous fluid. *Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik*, 2023. doi: 10.1002/zamm.202100534.

[7] S. Kim. Biorender. https://biorender.com, 2025. Image created with BioRender.

[8] H. Yoshida, S. Ishida, T. Yamamoto, T. Ishikawa, and Y. Imai. Effect of cilia-induced surface velocity on cerebrospinal fluid exchange in the lateral ventricles. *Journal of the Royal Society Interface*, 19(193), 2022. doi: 10.1098/rsif.2022.0321.

[9] H. E. Salman, N. Jurisch-Yaksi, and H. C. Yalcin. Computational modeling of motile cilia-driven cerebrospinal flow in the brain ventricles of zebrafish embryo. *Bioengineering*, 9(9), 2022. doi: 10.3390/bioengineering9090421.

[10] K. Wuttanachamsri and L. Schreyer. Effects of cilia movement on fluid velocity: Ii numerical solutions over a fixed domain. *Transport in Porous Media*, 134(2):471–489, 2020. doi: 10.1007/s11242-020-01455-4.

[11] V. Sahadevan, B. Panigrahi, and C. Chen. Microfluidic applications of artificial cilia: Recent progress, demonstration, and future perspectives. *Micromachines*, 13(5), 2022. doi: 10.3390/mi13050735.

[12] P. Satir and S. Christensen. Overview of structure and function of mammalian cilia. *Annual review of physiology*, 69:377–400, 2007. doi: 10.1146/annurev.physiol.69.040705.141236.

[13] J. Flaherty, Z. Feng, Z. Peng, Y. N. Young, and A. Resnick. Primary cilia have a length-dependent persistence length. *Biomechanics and Modeling in Mechanobiology*, 19(19): 901–911, 2020. doi: 10.1007/s10237-019-01220-7.

[14] M. Rana, A. Sachdev, and J. D'Souza. Appearing and disappearing acts of cilia. *Journal of Biosciences*, 48:8, 2023. doi: 10.1007/s12038-023-00326-6.

[15] E. Lauga and T. R. Powers. The hydrodynamics of swimming microorganisms. *Reports on Progress in Physics*, 72(9):096601, 2009. doi: 10.1088/0034-4885/72/9/096601.

[16] G. Orlando, P. F. Barbante, and L. Bonaventura. An efficient imex-dg solver for the compressible navier-stokes equations for non-ideal gases. *Journal of Computational Physics*, 471:111653, 2022. doi: https://doi.org/10.1016/j.jcp.2022.111653.

[17] M. Guesmi, M. Grotteschi, and J. Stiller. Assessment of high-order imex methods for incompressible flow. *International Journal for Numerical Methods in Fluids*, 01 2023. doi: 10.1002/fld.5177.

[18] F. Huang and J. Shen. A class of imex schemes and their error analysis for the navier–stokes cahn–hilliard system. *Annals of Mathematical Sciences and Applications*, 9:185–235, 01 2024. doi: 10.4310/AMSA.2024.v9.n1.a6.

[19] ENCCB. SP-WIND. https://www.enccb.be/index.php/SPWIND, 2024. Accessed: [Insert date of access].

[20] R.W.C.P. Verstappen and A.E.P. Veldman. Symmetry-preserving discretization of turbulent flow. *Journal of Computational Physics*, 187(1):343–368, 2003. doi: https://doi.org/10.1016/S0021-9991(03)00126-8.

[21] J.. Goit and J. Meyers. Optimal control of energy extraction in wind-farm boundary layers. *Journal of fluid mechanics*, 768:5–50, 2015.

[22] N. Janssens and J. Meyers. Towards real-time optimal control of wind farms using large-eddy simulations. *Wind Energy Science*, 9(1):65–95, 2024. doi: 10.5194/wes-9-65-2024.

[23] T. Bon and J. Meyers. Stable channel flow with spanwise heterogeneous surface temperature. *Journal of fluid mechanics*, 933, 2022. ISSN 0022-1120.

[24] T. Bon, D. Broos, R.B. Cal, and J. Meyers. Secondary flows induced by two-dimensional surface temperature heterogeneity in stably stratified channel flow. *Journal of fluid mechanics*, 970, 2023. ISSN 0022-1120.

[25] F.A. Morrison. *Understanding Rheology*. Raymond F. Boyer Library Collection. Oxford University Press, 2001. ISBN 9780195141665.

[26] Y.A. Çengel and J.M. Cimbala. *Fluid Mechanics: Fundamentals and Applications*. McGraw-Hill Education, 2018. ISBN 9781259921902.

[27] G. G. Stokes. On the effect of the internal friction of fluids on the motion of pendulums. *Transactions of the Cambridge Philosophical Society*, 9:8–106, 1851.

[28] F.M. White. *Viscous Fluid Flow 4e*. McGraw-Hill Education, 2021. ISBN 9780073529318.

[29] M. S. Hadi, S. Sadrizadeh, and O. Abouali. Three-dimensional simulation of mucociliary clearance under the ciliary abnormalities. *Journal of Non-Newtonian Fluid Mechanics*, 316: 105029, 2023. doi: https://doi.org/10.1016/j.jnnfm.2023.105029. URL https://www.sciencedirect.com/science/article/pii/S0377025723000411.

[30] M. Sedaghat, F. Stylianou, O. Abouali, P. Anagnostopoulou, and S Kassinos. Simulation of drug transport in airway surface liquid considering mucus flow and ciliary interaction. *Scientific Reports*, 15:31951, 08 2025. doi: 10.1038/s41598-025-16822-8.

[31] J. Meyers and C. Meneveau. Large eddy simulations of large wind-turbine arrays in the atmospheric boundary layer. 2010. ISBN 978-1-60086-959-4. doi: 10.2514/6.2010-827.

[32] R.L. Panton. *Incompressible Flow*. Developmental clinical psychology and psychiatry. Wiley, 1996. ISBN 9780471593584.

[33] D.J. Tritton. Experiments on the flow past a circular cylinder at low reynolds numbers. *Journal of Fluid Mechanics*, 6(4):547–567, 1959.

[34] C. Wieselsberger. New data on the laws of fluid resistance. NACA Technical Note 84, National Advisory Committee for Aeronautics, March 1922. University of North Texas Libraries, UNT Digital Library; accessed 8 September 2025.

[35] S Tomotika and T Aoi. The steady flow of viscous fluid past a sphere and circular cylinder. *Q. Jl Mech. Appl. Math*, 3:140–161, 1950.

[36] D. Sucker and H. Brauer. Investigation of the flow around transverse cylinders. *Wärme- und Stoffübertragung*, 8:149–158, 1975. doi: 10.1007/BF01681556.

[37] J.C. Butcher. A history of runge-kutta methods. *Applied Numerical Mathematics*, 20(3): 247–260, 1996. doi: https://doi.org/10.1016/0168-9274(95)00108-5.

[38] A. Iserles. *A First Course in the Numerical Analysis of Differential Equations*. Cambridge Texts in Applied Mathematics. Cambridge University Press, 1996. ISBN 9780521556552.

[39] J.C. Butcher. *Numerical Methods for Ordinary Differential Equations*. Wiley, 2008. ISBN 9780470753750.

[40] C. Kennedy and M. Carpenter. Diagonally implicit runge-kutta methods for ordinary differential equations. a review. page 163, 2016.

[41] C. Canuto, M.Y. Hussaini, A. Quarteroni, and J.Z. Thomas A. *Spectral Methods in Fluid Dynamics*. Scientific Computation. Springer Berlin Heidelberg, 2012. ISBN 9783642841088.

[42] G. E. Karniadakis, M. Israeli, and S. A. Orszag. High-order splitting methods for the incompressible navier-stokes equations. *Journal of Computational Physics*, 97(2):414–443, 1991. doi: https://doi.org/10.1016/0021-9991(91)90007-8.

[43] G. Orlando, P. Barbante, and L. Bonaventura. An efficient imex-dg solver for the compressible navier-stokes equations for non-ideal gases. *Journal of Computational Physics*, 471: 111653, 2022. doi: 10.1016/j.jcp.2022.111653.

[44] U. M. Ascher, S. J. Ruuth, and R. J. Spiteri. Implicit-explicit methods for time-dependent partial differential equations. *SIAM Journal on Numerical Analysis*, 32(3):797–823, 1995.

[45] U. M. Ascher, S. J. Ruuth, and R. J. Spiteri. Implicit-explicit runge-kutta methods for time-dependent partial differential equations. *Applied Numerical Mathematics*, 25(2):151–167, 1997. doi: https://doi.org/10.1016/S0168-9274(97)00056-1. Special Issue on Time Integration.

[46] M. Guesmi, M. Grotteschi, and J. Stiller. Assessment of high-order imex methods for incompressible flow. *International Journal for Numerical Methods in Fluids*, 95(6):954–978, 2023. doi: 10.1002/fld.5177.

[47] M. P. Calvo, J. de Frutos, and J. Novo. Linearly implicit runge–kutta methods for advection–reaction–diffusion equations. *Applied Numerical Mathematics*, 37(4):535–549, 2001. doi: https://doi.org/10.1016/S0168-9274(00)00061-1.

[48] H. M. von Wahl. Implicit-explicit time splitting schemes for incompressible navier-stokes flows. Masterarbeit, Georg-August-Universität Göttingen, Fakultät für Mathematik und Informatik, Institut für Numerische und Angewandte Mathematik, 2018.

[49] F. Moukalled, L. Mangani, and M. Darwish. *The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM® and Matlab*. Fluid Mechanics and Its Applications. Springer International Publishing, 2015. ISBN 9783319168739.

[50] T. Saad and M. Karam. Stable timestep formulas for high-order advection–diffusion and navier–stokes solvers. *Computers & Fluids*, 244:105564, 2022. doi: https://doi.org/10.1016/j.compfluid.2022.105564.

[51] H. Bhatia, G. Norgard, V. Pascucci, and P. Bremer. The helmholtz-hodge decomposition-a survey. *IEEE Transactions on Visualization and Computer Graphics*, 19:1386–, 2012. doi: 10.1109/TVCG.2012.316.

[52] C. Liu, H. Xu, X. Cai, and Y. Gao. *Liutex and Its Applications in Turbulence Research*. Elsevier Science, 2020. ISBN 9780128190234.

[53] F. Brezzi and M. Fortin. *Mixed and Hybrid Finite Element Methods*, volume 15. Springer-Verlag, 1991.

[54] J. Guzmán, A. J. Salgado, and F. Sayas. A note on the ladyzhenskaya-babuška-brezzi condition. *Submitted to Journal of Scientific Computing*, 2012. Preprint.

[55] M.D. Gunzburger. *Finite Element Methods for Viscous Incompressible Flows: A Guide to Theory, Practice, and Algorithms*. Computer science and scientific computing. Academic Press, 1989. ISBN 9780123073501.

[56] S. R. Reddy, J. Sebastian, S.M. Miyyadad, R. Banerjee, and N. Sivadasan. Single and two phase flow cfd solvers using gpu. In *2013 National Conference on Parallel Computing Technologies (PARCOMPTECH)*, pages 1–8, 2013.

[57] Y. Yamada, M. Sakai, S. Mizurani, S. Koshizuka, M. Oochi, and K. Murizono. Numerical simulation of three-dimensional free-surface flows with explicit moving particle simulation method. *Transactions of the Atomic Energy Society of Japan*, 10(3):185–193, 2011. doi: 10.3327/taesj.J10.033.

[58] SciPy Development Team. scipy.sparse.linalg.cg — conjugate gradient solver. https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.cg.html, 2023. SciPy version: 1.11.0. Accessed: 2023-11-15.

[59] J. Lottes. *Towards Robust Algebraic Multigrid Methods for Nonsymmetric Problems*. Springer Theses, Recognizing Outstanding Ph.D. Research. Springer International Publishing, Cham, 1st ed. 2017. edition, 2017. ISBN 3-319-56306-8.

[60] pyAMG Development Team. pyamg.classical — classical amg. https://pyamg.readthedocs.io/en/latest/generated/pyamg.classical.html, 2023. Version: latest. Accessed: 2023-11-15.

[61] M. T. Darvishi and M. Javidi. A numerical solution of burgers' equation by pseudospectral method and darvishi's preconditioning. *Applied Mathematics and Computation*, 173(1): 421–429, 2006. doi: 10.1016/j.amc.2005.04.079.

[62] M. A. Abdou and A. A. Soliman. Variational iteration method for solving burger's and coupled burger's equations. *Journal of Computational and Applied Mathematics*, 181(2): 245–251, 2005.

[63] V. Mukundan and A. Awasthi. Linearized implicit numerical method for burgers' equation. *Nonlinear Engineering*, 5, 2016. doi: 10.1515/nleng-2016-0031.

[64] J. Rottmann-Matthes. An imex-rk scheme for capturing similarity solutions in the multidimensional burgers' equation, 2016. URL https://arxiv.org/abs/1612.04127.

[65] M. Javidi. A numerical solution of burger's equation based on modified extended bdf scheme. *INTERNATIONAL MATHEMATICAL FORUM*, 2006.

[66] M. T. Darvishi and M. Javidi. Method of lines for solving system of time-dependent partial differential equations. *WSEAS Transactions on Mathematics*, 1:218–222, 2002.

[67] U. Ghia, K. N. Ghia, and C. T. Shin. High-re solutions for incompressible flow using the navier-stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3): 387–411, 1982. doi: https://doi.org/10.1016/0021-9991(82)90058-4.

[68] A.S. Benjamin and V.E. Denny. On the convergence of numerical solutions for 2-d flows in a cavity at large re. *Journal of Computational Physics*, 33(3):340–358, 1979. doi: https://doi.org/10.1016/0021-9991(79)90160-8.

[69] {R. K.} Agarwal. Third-order-accurate upwind scheme for navier-stokes solutions at high reynolds numbers. *AIAA Paper*, 1981. ISSN 0146-3705. AIAA Pap AIAA Aerosp Sci Meet, 19th ; Conference date: 12-01-1981 Through 15-01-1981.

[70] E. Erturk. Discussions on driven cavity flow. *International Journal for Numerical Methods in Fluids*, 60:275 – 294, 2009. doi: 10.1002/fld.1887.

[71] S. Khorasanizade and M. M. Joao Sousa. A detailed study of lid-driven cavity flow at moderate reynolds numbers using incompressible sph. *International Journal for Numerical Methods in Fluids*, 76(10):653–668, 2014. doi: 10.1002/fld.3949.

[72] S. G. Sumesaraayi and M. Ihmsen. Lid driven cavity 2d in preonlab. https://www.fifty2.eu/innovation/lid-driven-cavity-2d-in-preonlab, 2023. Accessed: 2025-08-19.

[73] A. Meana-Fernández, J. M. F. Oro, K. M. A Díaz, M. Galdo-Vega, and S. Velarde-Suárez. Application of richardson extrapolation method to the cfd simulation of vertical-axis wind turbines and analysis of the flow field. *Engineering Applications of Computational Fluid Mechanics*, 13(1):359–376, 2019. doi: 10.1080/19942060.2019.1596160.

[74] NASA Glenn Research Center. Examining spatial (grid) convergence tutorial. https://www.grc.nasa.gov/www/wind/valid/tutorial/spatconv.html, 2020. Accessed: 2025-08-19.

[75] F. Pimenta and M. A. Alves. *RheoTool: User Guide*, 2022. Version 6.0.

[76] M. Basa, N. Quinlan, and M. Lastiwka. Robustness and accuracy of sph formulations for viscous flow. *International Journal for Numerical Methods in Fluids*, 60:1127 – 1148, 2009. doi: 10.1002/fld.1927.

[77] Y.A. Çengel and M.A. Boles. *Thermodynamics: An Engineering Approach*. McGraw-Hill series in mechanical engineering. McGraw-Hill Education, 2018. ISBN 9781260092684.

[78] J. M.J. den Toonder and P. R. Onck. Microfluidic manipulation with artificial/bioinspired cilia. *Trends in Biotechnology*, 31(2):85–91, 2013. doi: https://doi.org/10.1016/j.tibtech.2012.11.005.

[79] J. Olmsted and G.M. Williams. *Chemistry: The Molecular Science*. Mosby, 1997. ISBN 9780815184508.

[80] Alvar M. Kabe and Brian H. Sako. Chapter 1 - structural dynamics. In A. M. Kabe and B. H. Sako, editors, *Structural Dynamics Fundamentals and Advanced Applications*, pages 1–45. Academic Press, 2020. ISBN 978-0-12-821614-9.

[81] O. D. Kellogg. *The Divergence Theorem*, pages 84–121. Springer Berlin Heidelberg, Berlin, Heidelberg, 1967. ISBN 978-3-642-86748-4.

[82] S. Lifson. Molecular forces. In R. Jaenicke and E. Helmreich, editors, *Protein-Protein Interactions*, pages 3–16, Berlin, Heidelberg, 1972. Springer Berlin Heidelberg. ISBN 978-3-642-65456-5.

[83] R.B. Bird, W.E. Stewart, and E.N. Lightfoot. *Transport Phenomena*. Number vol. 1 in Transport Phenomena. Wiley, 2006. ISBN 9780470115398.

[84] F. Irgens. *Rheology and Non-Newtonian Fluids*. Springer International Publishing, 2013. ISBN 9783319010533.

[85] S. Kaplun. Low reynolds number flow past a circular cylinder. *Journal of Mathematics and Mechanics*, 6(4):595–603, 1957.

[86] M. S. Swanson. *Classical Field Theory and the Stress–Energy Tensor (Second Edition)*. 2053-2563. IOP Publishing, 2022. ISBN 978-0-7503-3455-6.

[87] P.J. Pritchard. *Fox and McDonald's Introduction to Fluid Mechanics, 8th Edition*. John Wiley & Sons, 2010. ISBN 9781118139455.

[88] F.P. Incropera. *Fundamentals of Heat and Mass Transfer*. Wiley, 2007. ISBN 9780471761150.

[89] E. Süli and D.F. Mayers. *An Introduction to Numerical Analysis*. An Introduction to Numerical Analysis. Cambridge University Press, 2003. ISBN 9780521007948.

[90] M. W. Kutta. Beitrag zur näherungsweisen integration totaler differentialgleichungen. *Zeitschrift für Mathematik und Physik*, 46:435–453, 1901.

[91] Académie royale des sciences (France). *Mémoires de l'Académie (royale) des sciences de l'Institut (imperial) de France*, volume T.6(1827), page 441. Paris,, 1827. https://www.biodiversitylibrary.org/bibliography/4002.

[92] H. S. Hele-Shaw. Investigation of the nature of surface resistance of water and of streamline motion under certain experimental conditions. *Transactions of the Institution of Naval Architects*, page 26, 1898. Also known as the "Second Paper" on this topic.

[93] A.R. CURTIS. An eighth order runge-kutta process with eleven function evaluations per step. *Numerische Mathematik*, 16:268–277, 1970/71.

[94] S. Khashin. A symbolic-numeric approach to the solution of the butcher equations. *The Canadian Applied Mathematics Quarterly*, 17, 2009.

[95] P. Houwen and B. Sommeijer. Parallel iteration of high-order runge-kutta methods with stepsize control. *Journal of Computational and Applied Mathematics*, 29:111–127, 1990. doi: 10.1016/0377-0427(90)90200-J.

[96] H. Bhatia, G. Norgard, V. Pascucci, and P. Bremer. The helmholtz-hodge decomposition—a survey. *IEEE Transactions on Visualization and Computer Graphics*, 19(8):1386–1404, 2013. doi: 10.1109/TVCG.2012.316.

[97] Marcelo J.S. de Lemos. 10 - numerical modeling and algorithms. In Marcelo J.S. de Lemos, editor, *Turbulence in Porous Media (Second Edition)*, pages 143–198. Elsevier, Oxford, second edition edition, 2012. ISBN 978-0-08-098241-0.

[98] M. Asadzadeh and L. Beilina. A posteriori error estimates in a globally convergent fem for a hyperbolic coefficient inverse problem. *Inverse Problems*, 26:115007, 2010. doi: 10.1088/0266-5611/26/11/115007.

[99] K. Aghigh, M. Masjed-Jamei, and M. Dehghan. A survey on third and fourth kind of chebyshev polynomials and their applications. *Applied Mathematics and Computation*, 199(1):2–12, 2008. doi: https://doi.org/10.1016/j.amc.2007.09.018.

[100] Bruno Sudret. Global sensitivity analysis using polynomial chaos expansions. *Reliability Engineering & System Safety*, 93(7):964–979, 2008. doi: https://doi.org/10.1016/j.ress.2007.04.002. Bayesian Networks in Dependability.

[101] T. Zahtila, W. Lu, L. Chan, and A. Ooi. A systematic study of the grid requirements for a spectral element method solver. *Computers & Fluids*, 251:105745, 2023. doi: https://doi.org/10.1016/j.compfluid.2022.105745.

[102] A. P. S. Selvadurai. *Poisson's equation*, pages 503–647. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. ISBN 978-3-662-09205-7.

[103] M. El-Gamel, N. R. Nassar, and A. El-Shenawy. Fast and accurate poisson solver algorithm in 3d simply and double connected domains with a smooth complex geometry with applications in optics. *Optical and Quantum Electronics*, 2025. doi: 10.1007/s11082-025-08054-x.

[104] J. Liu. Poisson's Equation in Electrostatics. Technical report, Institute of Computational and Modeling Science, National Tsing Hua University, Hsinchu, Taiwan, 2007. Updated periodically (2010, 2011, 2012, 2017).

[105] A. A. Mohamad. *The Laplace, Poisson, and Biharmonic Equations*, pages 81–86. Springer London, London, 2019. ISBN 978-1-4471-7423-3.

[106] G.H. Golub and C.F. Van Loan. *Matrix Computations*. Johns Hopkins Studies in the Mathematical Sciences. Johns Hopkins University Press, 2013. ISBN 9781421407944.

[107] D.S. Watkins. *Fundamentals of Matrix Computations*. Pure and Applied Mathematics: A Wiley Series of Texts, Monographs and Tracts. Wiley, 2004. ISBN 9780471461678.

[108] E. Hairer, S. Norsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8. 1993. ISBN 978-3-540-56670-0.

[109] S. Chapra and R. Canale. *Numerical Methods for Engineers*. McGraw-Hill Education, 2009. ISBN 9780073401065.

[110] B. Knaepen and Y. Velizhanina. Solving partial differential equations (mooc). https://aquaulb.github.io/book_solving_pde_mooc/, 2022. Notebook: "Euler methods" (and "Runge–Kutta methods")—includes Python code for plotting stability regions.

[111] E. Erturk. Discussions on driven cavity flow. *International Journal for Numerical Methods in Fluids*, 60(3):275–294, 2008. doi: 10.1002/fld.1887.

[112] W. Hundsdorfer and J. Verwer. *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*. 2003. ISBN 978-3-642-05707-6.

[113] K. Van Canneyt and P. Verdonck. 10.02 - mechanics of biofluids in living body. In Anders Brahme, editor, *Comprehensive Biomedical Physics*, pages 39–53. Elsevier, Oxford, 2014. ISBN 978-0-444-53633-4.

[114] P. Pintado, P. Sampaio, B. Tavares, T. D. Montenegro-Johnson, D. J. Smith, and S. S. Lopes. Dynamics of cilia length in left–right development. *Royal Society Open Science*, 4:161102, 2017. doi: 10.1098/rsos.161102.

[115] M. V. Dyke, editor. *An Album of Fluid Motion*. The Parabolic Press, Stanford, CA, 1982. Photograph credited to D. H. Peregrine.

Appendix A

Stability of Linear Solvers

The use of an iterative solver requires that the iteration matrix exhibit two properties to ensure efficiency. $[\mathbf{M}]$ should be sparse — a characteristic commonly observed in pressure-Poisson equations — and diagonally dominant.

Definition A.0.1 (Sparse Matrix)**.** *A matrix* $[\mathbf{M}] \in \mathbb{R}^{N\times N}$ *is said to be* sparse *if the ratio between the number of zero elements, denoted* $n_0 \in \mathbb{N}$*, and the total number of elements,* $N_{\text{elm}} = N^2$*, is greater than* 0.5*. That is,*

$$\text{Sp} = \frac{n_0}{N_{\text{elm}}} > 0.5, \tag{A.1}$$

where Sp *is referred to as* sparsity *of the matrix.*

Definition A.0.2 (Diagonally Dominant Matrix)**.** *Let* $N \in \mathbb{N}$ *and* $m_{i,j} \in \mathbb{R}$*. A matrix* $[\mathbf{M}] \in \mathbb{R}^{N\times N}$ *is said to be* diagonally dominant *if, for all rows* $i = 1,\ldots,N$*, the magnitude of the diagonal element* $m_{i,i} \in \mathbb{R}$ *is greater than or equal to the sum of the magnitudes of all the non-diagonal elements in that row. Formally,*

$$|m_{i,i}| \geq \sum_{\substack{j=1 \\ j\neq i}}^{N} |m_{i,j}|, \quad \forall i \in \{1,\ldots,N\}. \tag{A.2}$$

In accordance with the spatial discretization scheme adopted, Equation (2.83) can be further expressed using a discrete Laplacian operator. Additionally, for convenience in notation, the right-hand side is denoted by $\mathbf{b}$, which yields the following finite volume formulation:

$$\frac{\tilde{p}_{i+1,j} + \tilde{p}_{i-1,j} - 2\tilde{p}_{i,j}}{\Delta x^2} + \frac{\tilde{p}_{i,j+1} + \tilde{p}_{i,j-1} - 2\tilde{p}_{i,j}}{\Delta y^2} = b_{i,j}. \tag{A.3}$$

Rearranging Equation (A.3) to isolate $\tilde{p}_{i,j}$ yields a form that is particularly well-suited for iterative solvers applied to the Pressure-Poisson equation. The general iterative update expression can be written as:

$$\tilde{p}_{i,j}^{(k+1)} = \frac{1}{2\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)} \left(\frac{\tilde{p}_{i+1,j}^{(C_1)} + \tilde{p}_{i-1,j}^{(C_2)}}{\Delta x^2} + \frac{\tilde{p}_{i,j+1}^{(C_3)} + \tilde{p}_{i,j-1}^{(C_4)}}{\Delta y^2} \right). \tag{A.4}$$

If $\Delta x = \Delta y = h_k$, Equation (A.4) further simplifies to,

$$\tilde{p}_{i,j}^{(k+1)} = \frac{1}{4}\left(\tilde{p}_{i+1,j}^{(C_1)} + \tilde{p}_{i-1,j}^{(C_2)} + \tilde{p}_{i,j+1}^{(C_3)} + \tilde{p}_{i,j-1}^{(C_4)} - h_k^2 b_{i,j}\right). \tag{A.5}$$

In these expressions, the superscripts indicate the iteration stage at which each neighboring pressure value is evaluated. The central node (i, j) is always updated at iteration $k+1$, while the values of C_1, C_2, C_3, and C_4, called iteration identifiers, determine the specific iterative method being employed:

- If $C_1 = C_2 = C_3 = C_4 = k$, the Jacobi method is used, as all neighboring values come from the current iteration stage;
- If $C_1 = C_3 = k$ and $C_2 = C_4 = k+1$, the scheme corresponds to the Gauss–Seidel method, where the right and top elements come from previous iterations and the left and bottom elements come from the new computed values.

All things considered, a more generalized and compact form of Equation (A.5) can be derived.

Definition A.0.3. *Consider the linear system*

$$[\mathbf{A}]\,\mathbf{x} = \mathbf{b}, \tag{A.6}$$

where $[\mathbf{A}] \in \mathbb{R}^{n\times n}$ *is a given matrix,* $\mathbf{x} \in \mathbb{R}^n$ *is the unknown vector, and* $\mathbf{b} \in \mathbb{R}^n$ *is the right-hand side vector. A matrix* $[\mathbf{P}] \in \mathbb{R}^{n\times n}$ *is called a preconditioner for* $[\mathbf{A}]$ *if the condition number of the preconditioned matrix* $[\mathbf{P}]^{-1}[\mathbf{A}]$ *is smaller than that of* $[\mathbf{A}]$*, i.e.,*

$$\kappa\{[\mathbf{P}]^{-1}[\mathbf{A}]\} < \kappa\{[\mathbf{A}]\}, \tag{A.7}$$

where $\kappa\{\cdot\}$ *denotes the condition number with respect to a chosen norm. Using the preconditioner, the original linear system can be rewritten as*

$$[\mathbf{P}]\,\mathbf{x} = ([\mathbf{P}] - [\mathbf{A}])\mathbf{x} + \mathbf{b}, \tag{A.8}$$

which naturally leads to the iterative scheme

$$\mathbf{x}^{(k+1)} = [\mathbf{W}]\,\mathbf{x}^{(k)} + [\mathbf{P}]^{-1}\,\mathbf{b}, \tag{A.9}$$

where the iteration matrix $[\mathbf{W}]$ *is defined by*

$$[\mathbf{W}] = [\mathbf{I}] - [\mathbf{P}]^{-1}[\mathbf{A}]. \tag{A.10}$$

The convergence of such methods is entirely determined by the spectral properties of $[\mathbf{W}]$. In particular, the efficiency of a good preconditioner is derived from its ability to reduce the spectral radius $\rho\{[\mathbf{W}]\}$, thus accelerating convergence. This observation links directly to the following fundamental result on stationary iterative methods [107]:

Theorem A.0.1 (Convergence Criterion for Stationary Iterative Methods)**.** *Let* $[\mathbf{A}] \in \mathbb{R}^{n\times n}$*, and consider the linear system*

$$[\mathbf{A}]\,\mathbf{x} = \mathbf{b}. \tag{A.11}$$

Suppose the application of a stationary iterative method of the form

$$\mathbf{x}^{(k+1)} = [\mathbf{W}]\,\mathbf{x}^{(k)} + \mathbf{c}, \tag{A.12}$$

where $[\mathbf{W}] \in \mathbb{R}^{n\times n}$ *is called the* iteration matrix*. Then the iterative method converges to the exact solution* $\mathbf{x}^*$ *from any initial guess* $\mathbf{x}^{(0)}$ *if and only if*

$$\rho\left\{[\mathbf{W}]\right\} < 1, \tag{A.13}$$

where $\rho\left\{[\mathbf{M}]\right\}$ *denotes the maximum eigenvalue of* $[\mathbf{W}]$*, commonly known as spectral radius of* $[\mathbf{W}]$*.*

Proof. Let the error at iteration k be defined by

$$\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*, \tag{A.14}$$

where $\mathbf{x}^*$ denotes the exact solution of the system, which satisfies

$$\mathbf{x}^* = [\mathbf{W}]\,\mathbf{x}^* + \mathbf{c}. \tag{A.15}$$

Subtracting (A.15) from the iterative scheme

$$\mathbf{x}^{(k+1)} = [\mathbf{W}]\,\mathbf{x}^{(k)} + \mathbf{c}, \tag{A.16}$$

yields

$$\mathbf{e}^{(k+1)} = [\mathbf{W}]\,\mathbf{e}^{(k)}. \tag{A.17}$$

By repeated substitution (induction), it follows that

$$\mathbf{e}^{(k)} = [\mathbf{W}]^k\,\mathbf{e}^{(0)}. \tag{A.18}$$

Convergence of the method requires that the error tends to zero for any initial error $\mathbf{e}^{(0)}$, i.e.,

$$\lim_{k\to\infty}\left\|\mathbf{e}^{(k)}\right\| = \lim_{k\to\infty}\left\|[\mathbf{W}]^k\,\mathbf{e}^{(0)}\right\| = 0. \tag{A.19}$$

From the spectral radius theorem (Gelfand's formula), for any induced matrix norm,

$$\lim_{k\to\infty}\left\|[\mathbf{W}]^k\right\|^{1/k} = \rho\left\{[\mathbf{W}]\right\}, \tag{A.20}$$

and asymptotically,

$$\left\|[\mathbf{W}]^k\,\mathbf{e}^{(0)}\right\| \approx \left(\rho\left\{[\mathbf{W}]\right\}\right)^k. \tag{A.21}$$

Thus, (A.19) holds if and only if

$$\rho\{[\mathbf{W}]\} < 1, \tag{A.22}$$

□

Lastly, there are two metrics that define every linear solver: time and space complexity. The first refers to how long an algorithm would take to complete a given input of size n_i and is measured as the number of floating-point operations in an algorithm. Typically, iterative solvers are much cheaper in this sense for sparse matrices, i.e., matrices that are mainly composed of zeros. The second simply refers to an algorithm's memory requirements.

Appendix B

Schematic Grid Representations

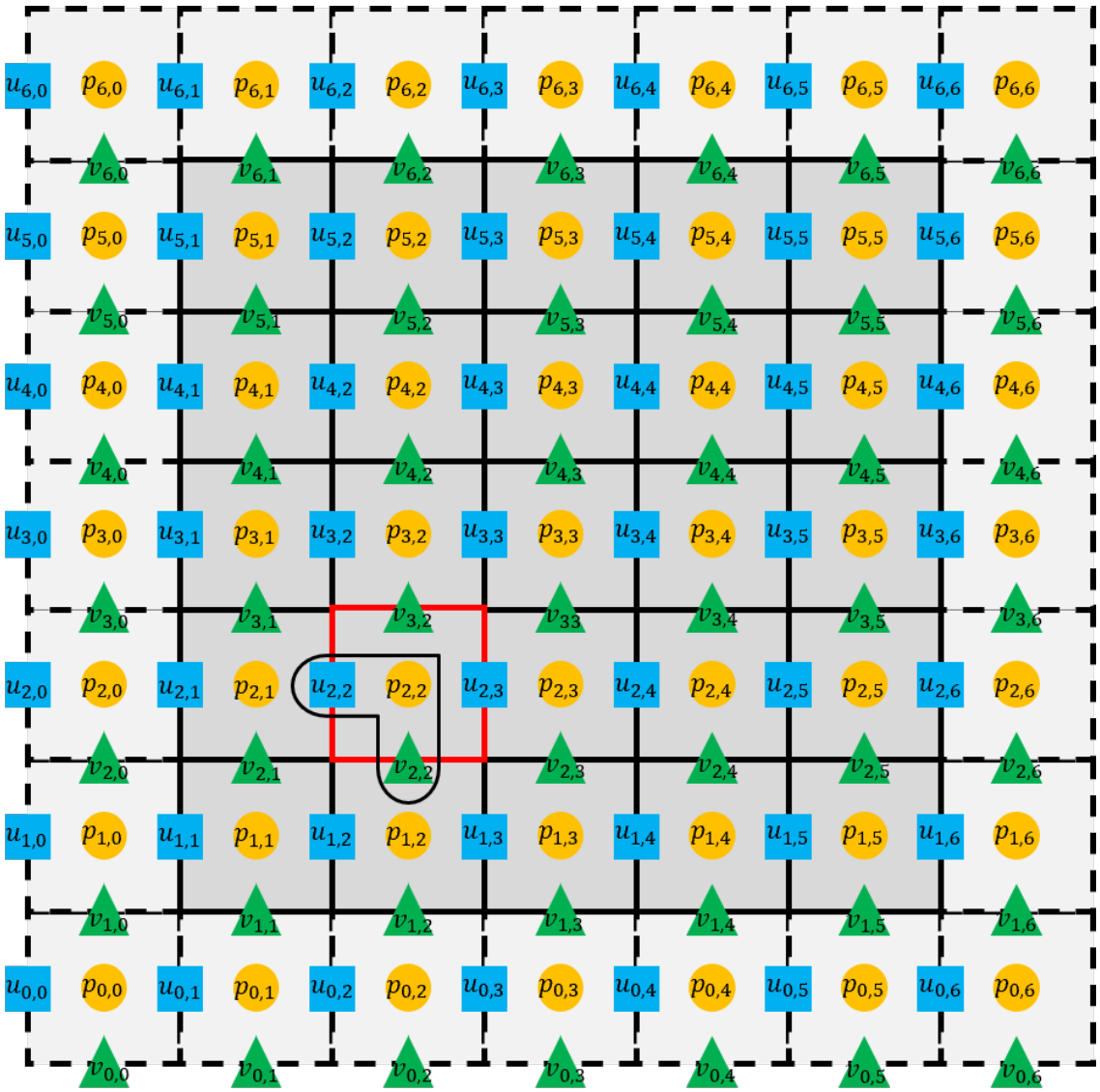

Figure B.1: Staggered grid scheme in Python notation.

Appendix C

Laminar Flow between two Horizontal Parallel Plates

Consider a steady and incompressible flow between two infinitely long parallel plates, oriented along the x-axis, with no velocity components in the $y-$ and $z-$ directions. The streamwise velocity component u varies only with the vertical coordinate z, Equation (C.1). The characteristic length and velocity scales for this configuration are defined as the half-distance between the plates, h, and the maximum velocity, u_{max}, which occurs at the mid-plane, $z = h$. In addition to the Neumann boundary condition, the problem is further defined by two Dirichlet boundary conditions corresponding to the no-slip condition on the upper and lower walls, Equations (C.2) and (C.3), respectively.

$$\frac{\partial u}{\partial x} = 0, \quad \forall x, z \in \Omega \tag{C.1}$$

$$u\{z = 0\} = 0\,, \tag{C.2}$$

$$u\{z = 2h\} = 0\,. \tag{C.3}$$

The momentum equations can be then written as follows for x, y and z, respectively:

$$0 = -\frac{\partial p}{\partial x} + \mu \frac{\partial^2 u}{\partial z^2}\,, \tag{C.4}$$

$$0 = \frac{\partial p}{\partial y}\,, \tag{C.5}$$

$$0 = -\frac{\partial p}{\partial z} - \rho g\,. \tag{C.6}$$

By merging Equations (C.5) and (C.6) and then integrating the result, yields:

$$p = -\rho g z + \chi_1\{x\}\,, \tag{C.7}$$

indicating that the pressure varies hydrostatically in the z direction. $\chi_1\{x\}$ expresses an integration function depending on x. Now, at the x-momentum, rewrite Equation (C.4) into:

$$\frac{\partial^2 u}{\partial z^2} = \frac{1}{\mu}\frac{\partial p}{\partial x}, \tag{C.8}$$

and subsequently integrate with respect to z, twice:

$$u\{z\} = \frac{1}{2\mu}\left(\frac{\partial p}{\partial x}\right) z^2 + c_1 z + c_2. \tag{C.9}$$

Finally, employ the boundary conditions previously defined to obtain:

$$c_1 = -\frac{h}{\mu}\left(\frac{\partial p}{\partial x}\right), \tag{C.10}$$

$$c_2 = 0. \tag{C.11}$$

Then substituting c_1 and c_2 back into Equation (C.9):

$$u\{z\} = \frac{z^2}{2\mu}\left(\frac{\partial p}{\partial x}\right) - \frac{hz}{\mu}\left(\frac{\partial p}{\partial x}\right). \tag{C.12}$$

Isolating the term $-\frac{1}{2\mu}\left(\frac{\partial p}{\partial x}\right)$ and simultaneously applying a mathematical identity of order h^2 to the right-hand-side of this last equation:

$$u\{z\} = -\frac{h^2}{2\mu}\left(\frac{\partial p}{\partial x}\right)\left[2\left(\frac{z}{h}\right) - \left(\frac{z}{h}\right)^2\right]. \tag{C.13}$$

Note that the pressure gradient is a negative quantity by default; therefore, when affected by a minus sign, its value coincides with its modulus, so that

$$u\{z\} = \frac{h^2}{2\mu}\left|\frac{\partial p}{\partial x}\right|\left[2\left(\frac{z}{h}\right) - \left(\frac{z}{h}\right)^2\right]. \tag{C.14}$$

In addition, an expression for the maximum velocity is now feasible to write:

$$u_{max} = u\{h\} = \frac{h^2}{2\mu}\left|\frac{\partial p}{\partial x}\right|. \tag{C.15}$$

The velocity at any point in the domain is equal to the maximum velocity scaled by a factor α, which is a function of the relative distance $\delta_z = z/h$.

$$u\{z\} = \alpha\{\delta_z\} \cdot u_{max}. \tag{C.16}$$

Furthermore, the volumetric flow rate per unit of height, $\dot{V}'$, passing between plates, is obtained from the following relationship:

$$\dot{V}' = \int_0^{2h} \frac{h^2}{2\mu} \left| \frac{\partial p}{\partial x} \right| \left[2\left(\frac{z}{h}\right) - \left(\frac{z}{h}\right)^2 \right] dz, \tag{C.17}$$

or,

$$\dot{V}' = \frac{2h^3}{3\mu} \left| \frac{\partial p}{\partial x} \right|, \tag{C.18}$$

meaning that $\dot{V}'$ is proportional to the pressure gradient, inversely proportional to the viscosity, and strongly dependent $\left(\sim h^3\right)$ on the gap width. In terms of the mean velocity, $\overline{U}$, is given by

$$\overline{U} = \frac{h^2}{3\mu} \left| \frac{\partial p}{\partial x} \right|. \tag{C.19}$$

With this, the mean velocity relates with the maximum velocity as follows:

$$\overline{U} = \frac{2}{3} u_{max}, \tag{C.20}$$

The final expression for the velocity profile $u\{z\}$, which will be applied in SP-Wind, is given by:

$$u\{z\} = u_{max} \left[2\left(\frac{z}{h}\right) - \left(\frac{z}{h}\right)^2 \right]. \tag{C.21}$$

Moreover, White [28] states that the flow will be laminar up to $Re = \frac{\rho \overline{U} h}{\mu} = 700$. Beyond this threshold, the flow transitions to a turbulent regime, rendering the previous laminar flow analysis invalid due to the emergence of complex, three-dimensional, and time-dependent flow characteristics. However, in the case of considering the value of the density equal to the unit, these relations still remain, with the kinematic viscosity replacing the dynamic one.

Considering the characteristic quantities assumed in the construction of the cilia case, i.e., half-channel width (h) and mean inlet $u-$velocity $(\overline{u})$ to be equal to the unit (see Table 3.3), the following relation stands:

$$Re_h = \frac{\overline{u} \cdot h}{\nu} = \frac{1}{\nu}. \tag{C.22}$$

Turning our attention to the cilia, the cilia's Reynolds number is defined as:

$$Re_D = \frac{\overline{U}_{cil} D_{cil}}{\nu}, \tag{C.23}$$

that simplifies into:

$$Re_D = Re_h \cdot \overline{U}_{cil} \cdot D_{cil}. \tag{C.24}$$

Moreover, by integrating the velocity profile, Equation (C.21), into the projected cilia area, we derive an expression of the mean velocity that heats the cilia $\left(\overline{U}_{cil}\right)$. Considering the discrete area

$dS = D_{cil}\,dz$ and the total projected area $A_p = D_{cil} \cdot h_{cil}$:

$$\overline{U}_{cil} = \frac{1}{h_{cil}} \int_0^{h_{cil}} u_{max} \left[2\left(\frac{h_{cil}}{h}\right) - \left(\frac{h_{cil}}{h}\right)^2 \right] dz\,. \tag{C.25}$$

By solving this primitive, we obtain:

$$\overline{U}_{cil} = 1.5\overline{u} \left[\left(\frac{h_{cil}}{h}\right) - \frac{1}{3}\left(\frac{h_{cil}}{h}\right)^2 \right]. \tag{C.26}$$

Substituting Equation (C.26) into (C.24), and considering Equation (C.20) and the assumed dimensions, Re is written as:

$$Re = \frac{Re_D}{1.5 D_{cil}\left(h_{cil} - \frac{1}{3}h_{cil}^2\right)}\,. \tag{C.27}$$

Finally, by manipulating Equation (C.15), we get:

$$\left|\frac{\partial p}{\partial x}\right| = \frac{3}{Re}\,. \tag{C.28}$$

Appendix D

Linear Solver's Code

```python
from scipy.sparse.linalg import bicgstab
import pyamg

def cg_solver(p, S, App, nx, ny):

    # Flatten the source term (excluding ghost cells)
    b = S[1:-1, 1:-1].ravel()

    # Initial guess (flattened, excluding ghost cells)
    p0 = p[1:-1, 1:-1].ravel()

    # Solve using PCG with diagonal preconditioner
    M = diags(1.0/App.diagonal())  # Jacobi preconditioner

    p_flat, info = cg(App, b, x0=p0, rtol=1e-06, M=M)

    if info != 0:
        print(f"PCG did not converge: info = {info}")
    # Compute residual norm
    residual = np.abs(App @ p_flat - b).reshape((ny,nx))
    #print(f'DimentionsResi_p = {np.shape(residual)}')

    # Reshape solution and maintain ghost cells
    p_new = p.copy()
    p_new[1:-1, 1:-1] = p_flat.reshape((ny, nx))

    return p_new, residual

```

```
29 def amg_solver(p, S, Ap, nx, ny, ml):
30     """
31     Solves the linear system Ap x = b using PyAMG as a solver.
32     """
33     # Flatten source term (excluding ghost cells)
34     b = S[1:-1, 1:-1].ravel()
35 
36     # Initial guess
37     x0 = p[1:-1, 1:-1].ravel()
38 
39     # Setup PyAMG multigrid solver
40     #ml = pyamg.smoothed_aggregation_solver(Ap)
41 
42     # Solve the system using AMG
43     x = ml.solve(b, x0=x0, tol=1e-6, cycle ='V')
44 
45 
46     # Compute residual norm
47     #residual = np.linalg.norm(Ap @ x - b)
48     residual = np.abs(Ap @ x - b).reshape((ny,nx))
49 
50     # Insert result back into pressure field
51     p_new = p.copy()
52     p_new[1:-1, 1:-1] = x.reshape((ny, nx))
53 
54     return p_new, residual
```

Appendix E

Stability Region Code

The stability regions were numerically computed by evaluating the stability functions $R(z)$ for each time integration method across the complex plane $z = \lambda \Delta t$, where λ represents the eigenvalues of the model problem and Δt is the time step size. The computation was adapted from reference implementations [110] with modifications for the IMEX methods. The implementation consists of three main components:

1. **Stability Functions**:
    - For implicit ARS methods, $R(z)$ is computed analytically via the functions `ARS443` and `ARS222`;
    - For explicit methods (Euler, RK4), $R(z)$ is calculated as polynomial expansions in z.
2. **Grid Evaluation**:
    - A 100×100 grid spans $z \in [-5, 1.5] \times [-5i, 5i]$;
    - Computes $|R(z)|$ at each grid point.
3. **Boundary Extraction**:
    - Contours where $|R(z)| = 1$ are identified using `matplotlib.contour`;
    - Contours are smoothed with spline interpolation;
    - Results saved as `.dat` files for LaTeX plotting.

```python
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.interpolate import splprep, splev
import json

def ARS443(X,Y):
    ReTerm1 = 48*(6-6*X+X**3)
    ReTerm2 = -144*(1-X)*Y**2
    ReTerm3 = 3*(X**2) * (Y**2) -7*Y**4

    ImTerm = 48*(6-6*X-Y**2)*Y + 29*Y*X**3 + 57*X*Y**3
    scale = 18*(X-2)**4
    R = (ReTerm1+ReTerm2+ReTerm3)/scale + 1j*(ImTerm/scale)
    return R

def ARS222(X,Y):
    gamma = 1 - (2**0.5)/2
    delta = 1 - 1/(2*gamma)
    ReTerm = 1 + (1-2*gamma)*X + gamma*(delta-1)*Y**2
    ImTerm = Y + gamma*(1-delta-gamma)*X*Y
    scale = (1-gamma*X)**2
    R = ReTerm/scale + 1j*(ImTerm/scale)
    return R

# Grid setup
nx, ny = 100, 100
x = np.linspace(-5, 1.5, nx)
y = np.linspace(-5, 5, ny)
X, Y = np.meshgrid(x, y)
Z = X + 1j*Y

# Stability functions
sigma1 = 1 + Z  # Euler
sigma4 = 1 + Z + Z**2/2 + Z**3/6 + Z**4/24  # RK4
sigma443 = ARS443(X,Y)  # ARS(4,4,3)
sigma222 = ARS222(X,Y)  # ARS(2,2,2)

def get_contour_data(X, Y, norm, method_name):
    fig, ax = plt.subplots()
```

```
    contour = ax.contour(X, Y, norm, levels=[1])
    plt.close(fig)
    contour_lines = contour.allsegs[0]
    all_points = []
    for i, segment in enumerate(contour_lines):
        if len(segment) > 10:
            tck, u = splprep(segment.T, s=0.0, per=0)
            u_new = np.linspace(u.min(), u.max(), 200)
            x_new, y_new = splev(u_new, tck, der=0)
            arr = np.column_stack((x_new, y_new))
            all_points.append(arr)
            np.savetxt(f"{method_name}_contour_{i}.dat", arr,
             ↪  fmt="%.6f")
    return all_points

# Generate and save contours
euler_contour = get_contour_data(X, Y, np.abs(sigma1)**2, 'euler')
rk4_contour = get_contour_data(X, Y, np.abs(sigma4)**2, 'rk4')
IM443_contour = get_contour_data(X, Y, np.abs(sigma443)**2,
 ↪  'ARS443')
IM222_contour = get_contour_data(X, Y, np.abs(sigma222)**2,
 ↪  'ARS222')

# Plotting
fig, ax = plt.subplots(figsize=(8, 8))
ax.contour(X, Y, np.abs(sigma1)**2, levels=[1], colors='r',
 ↪  linewidths=2)
ax.contour(X, Y, np.abs(sigma4)**2, levels=[1], colors='b',
 ↪  linewidths=2)
ax.contour(X, Y, np.abs(sigma443)**2, levels=[1], colors='g',
 ↪  linewidths=2)
ax.contour(X, Y, np.abs(sigma222)**2, levels=[1], colors='m',
 ↪  linewidths=2)

# Axis formatting
ax.spines[['right','top']].set_visible(False)
ax.spines[['left','bottom']].set_position('zero')
ax.set_xlabel(r'$\lambda_r \Delta t$', labelpad=10)
ax.set_ylabel(r'$\lambda_i \Delta t$', rotation=0, labelpad=10)
ax.set_aspect('equal')
```

```
plt.savefig('stability_regions.png', dpi=300, bbox_inches='tight')
plt.show()
```

The output files (`*_contour_*.dat`) contain the stability boundaries used to generate Figure 2.4. This numerical approach avoids symbolic computation while accurately capturing the stability limits.

Appendix F

Burgers Equation Code

```
import numpy as np
import matplotlib.pyplot as plt
import math
import os
import argparse
import time
import sys
from scipy.sparse import diags, csr_matrix, identity
from scipy.sparse.linalg import cg
# --- Add argument parsing ---
parser = argparse.ArgumentParser(description='Run Burgers equation simulation.')
parser.add_argument('--output_dir', type=str,
                    help='Directory to save output plots and data.',
                    default=None)
parser.add_argument('--nu', type=float, default=0.1,
                    help='Viscosity coefficient')
parser.add_argument('--n', type=int, default=10,
                    help='Number of internal points')
parser.add_argument('--dt', type=float, default=1e-5,
                    help='Time step size')
parser.add_argument('--method', type=str, default='all',
                    choices=['fe', 'rk4', 'mebdf', 'IMEX222', 'IMEX443', 'all'],
                    help='Integration method to use')
args = parser.parse_args()

# Determine the base directory for outputs
```

```
if args.output_dir:
    output_base_dir = args.output_dir
else:
    output_base_dir = os.path.dirname(os.path.abspath(__file__))

script_dir = output_base_dir
print(f"Output files will be saved to: {script_dir}")

#===Parameters from paper==========================================================================
nu = args.nu  # Viscosity coefficient
n = args.n    # Number of internal points (internal only; total grid points = n + 2)
dt = args.dt  # Time step
t_end = 1   # Final time
method = args.method

#===Domain Setup===================================================================================
lx = 1.0
x = np.linspace(0, lx, n+2)  # Including boundary points
h = x[1] - x[0]
print(f'x = {x}')
print(f'h = {h}')
print(f'nu = {nu}')
print(f'method = {method}')

print(f'dt_visc = {0.5*h**2/nu}')


#===Initial condition==============================================================================
# Exact solution function for Problem 1 (from paper)
def exact_solution(x_loc, t_loc, nu_loc):
    A = 0.05 / nu_loc * (x_loc - 0.5 + 4.95 * t_loc)
    B = 0.25 / nu_loc * (x_loc - 0.5 + 0.75 * t_loc)
    C = 0.5 / nu_loc * (x_loc - 0.5 - 0.375 * t_loc)
    numerator = 0.1 * np.exp(-A) + 0.5 * np.exp(-B) + np.exp(-C)
    denominator = np.exp(-A) + np.exp(-B) + np.exp(-C)
    return numerator / denominator
```

```
63
64  # Initial condition
65  u_initial = exact_solution(x, 0.0, nu)
66  print(f'u_0 = {u_initial}')
67  times = [0.0]
68
69  #===High-order finite difference schemes (from paper equations
    ↪  7-12)=================================================================
70  def compute_E(u, g1, g2, h):
71      """Compute E_i(t) = dU/dx(x_i,t) using high-order finite
        ↪  differences"""
72      n_internal = len(u) - 2  # Number of internal points
73      E = np.zeros(len(u))  # Including boundaries
74
75      # For x = x_1 (equation 7)
76      if n_internal >= 4:
77          E[1] = (-6*g1 - 20*u[1] + 36*u[2] - 12*u[3] + 2*u[4]) /
            ↪  (24*h)
78
79      # For internal points x_i, i=2,...,n-1 (equation 8)
80      for i in range(2, n_internal):
81          if i+2 < len(u):
82              E[i] = (2*u[i-2] - 16*u[i-1] + 16*u[i+1] - 2*u[i+2]) /
                ↪  (24*h)
83
84      # For x = x_n (equation 9)
85      if n_internal >= 4:
86          E[n_internal] = (-2*u[n_internal-3] + 12*u[n_internal-2] -
            ↪  36*u[n_internal-1] + 20*u[n_internal] + 6*g2) / (24*h)
87
88      return E
89
90  def compute_R(u, g1, g2, h):
91      """Compute R_i(t) = d²U/dx²(x_i,t) using high-order finite
        ↪  differences"""
92      n_internal = len(u) - 2  # Number of internal points
93      R = np.zeros(len(u))  # Including boundaries
94
95      # For x = x_1 (equation 10)
96      if n_internal >= 5:
```

```
97          R[1] = (20*g1 - 30*u[1] - 8*u[2] + 28*u[3] - 12*u[4] +
            ↪  2*u[5]) / (12*h**2)
98
99      # For internal points x_i, i=2,...,n-1 (equation 11)
100     for i in range(2, n_internal):
101         if i+2 < len(u):
102             R[i] = (-2*u[i-2] + 32*u[i-1] - 60*u[i] + 32*u[i+1] -
                ↪  2*u[i+2]) / (12*h**2)
103
104     # For x = x_n (equation 12)
105     if n_internal >= 5:
106         R[n_internal] = (2*u[n_internal-4] - 12*u[n_internal-3] +
            ↪  28*u[n_internal-2] - 8*u[n_internal-1] -
            ↪  30*u[n_internal] + 20*g2) / (12*h**2)
107
108     return R
109
110 #--- Nonlinear explicit term and R-term helpers (correctly use stage
    ↪  Y) ---
111 def N_term(u, t_loc):
112     """Nonlinear convective term N(u) = -u * (du/dx) computed on
        ↪  full grid (including BCs)"""
113     g1 = exact_solution(0.0, t_loc, nu)
114     g2 = exact_solution(lx, t_loc, nu)
115     E = compute_E(u, g1, g2, h)
116     v = np.zeros_like(u)
117     v[1:-1] = -u[1:-1] * E[1:-1]
118     return v
119
120 def R_term(u, t_loc):
121     """Return R(u) (second derivative) (no \nu factor) and BCs
        ↪  used"""
122     g1 = exact_solution(0.0, t_loc, nu)
123     g2 = exact_solution(lx, t_loc, nu)
124     R = compute_R(u, g1, g2, h)
125     v = np.zeros_like(u)
126     v[1:-1] = R[1:-1]
127     return v, g1, g2
128
129 def rhs(t_loc, u, nu_loc, h_loc):
```

```
130     """Right-hand side function for the ODE system (equation 15)"""
131     g1 = exact_solution(0.0, t_loc, nu_loc)  # Boundary condition at
        ↪ x=0
132     g2 = exact_solution(lx, t_loc, nu_loc)  # Boundary condition at
        ↪ x=1
133
134     E = compute_E(u, g1, g2, h_loc)
135     R = compute_R(u, g1, g2, h_loc)
136
137     # f_i(t,y(t)) = -y_i(t)E_i(t) + \nu R_i(t)
138     n_internal = len(u) - 2
139     rhs_val = np.zeros_like(u)
140     rhs_val[1:n_internal+1] = -u[1:n_internal+1] * E[1:n_internal+1]
        ↪ + nu_loc * R[1:n_internal+1]
141
142     return rhs_val
143
144 #=== Discrete 4th-order Laplacian and source (no \nu factored in)
    ↪ ============================================================
145 def generate_4th_order_laplacian(n_int, h_loc):
146     """Return (dense) matrix L (n_int x n_int) approximating R
        ↪ (without \nu) and identity"""
147     total_size = n_int
148     L = np.zeros((total_size, total_size))
149     factor = 1.0 / (12.0 * h_loc**2)
150
151     for i in range(total_size):
152         if i == 0:
153             L[0, 0] = -30 * factor
154             if total_size > 1: L[0, 1] = -8 * factor
155             if total_size > 2: L[0, 2] = 28 * factor
156             if total_size > 3: L[0, 3] = -12 * factor
157             if total_size > 4: L[0, 4] = 2 * factor
158         elif i == total_size - 1:
159             if total_size > 4: L[-1, -5] = 2 * factor
160             if total_size > 3: L[-1, -4] = -12 * factor
161             if total_size > 2: L[-1, -3] = 28 * factor
162             if total_size > 1: L[-1, -2] = -8 * factor
163             L[-1, -1] = -30 * factor
164         elif i == 1:
```

```
            L[1, 0] = 32 * factor
            L[1, 1] = -60 * factor
            if total_size > 2: L[1, 2] = 32 * factor
            if total_size > 3: L[1, 3] = -2 * factor
        elif i == total_size - 2:
            if total_size > 3: L[-2, -4] = -2 * factor
            L[-2, -3] = 32 * factor
            L[-2, -2] = -60 * factor
            L[-2, -1] = 32 * factor
        else:
            if i >= 2: L[i, i-2] = -2 * factor
            if i >= 1: L[i, i-1] = 32 * factor
            L[i, i] = -60 * factor
            if i < total_size - 1: L[i, i+1] = 32 * factor
            if i < total_size - 2: L[i, i+2] = -2 * factor

    return L

def generate_4th_order_ST(n_int, h_loc, g1_loc, g2_loc):
    """Boundary source vector S (no \nu factor) corresponding to
     ↪ discrete R"""
    b = np.zeros(n_int)
    factor = 1.0 / (12.0 * h_loc**2)
    if n_int >= 1:
        b[0] += 20 * factor * g1_loc
    if n_int >= 2:
        b[1] += -2 * factor * g1_loc
    if n_int >= 2:
        b[-2] += -2 * factor * g2_loc
    if n_int >= 1:
        b[-1] += 20 * factor * g2_loc
    return b

# Build discrete operators (sparse)
L_dense = generate_4th_order_laplacian(n, h)    # n x n (internal
 ↪ points)
L_mtx = csr_matrix(L_dense)
I_mtx = identity(n, format='csr')

```

```
#=== Generic IMEX stage solver (works for both IMEX222 and IMEX443 with appropriate tableaus) ================================
def imex_step_general(u, t_loc, dt_loc, nu_loc, h_loc, A, A_hat, b, b_hat, c, c_hat):
    """
    Generic IMEX RK stage solver.
    """
    s = A.shape[0]
    # Stage solutions Y_i (full-grid vectors)
    #Y = [u.copy() for _ in range(s)]
    # Stage evaluations
    #N_eval = [np.zeros_like(u) for _ in range(s)]
    #R_eval = [np.zeros_like(u) for _ in range(s)]
    #ST_eval = [np.zeros(n) for _ in range(s)]  # internal-only source vectors
    Khat_strg = []
    K_strg = []
    U_strg = []

    Y0 = np.zeros_like(u)
    Yi = np.zeros_like(u)

    Mi = I_mtx - (dt_loc * A[0,0] * nu_loc) * L_mtx

    t0 = t_loc + c[0] * dt_loc
    _, g10, g20 = R_term(u.copy(), t0)

    S0 = generate_4th_order_ST(n, h_loc, g10, g20)

    #First stage
    K_hat0 = N_term(u.copy(), t_loc + c_hat[0]*dt_loc)
    Khat_strg.append(K_hat0)

    rhs_int0 = u[1:-1].copy() + dt_loc * A_hat[1, 0]*K_hat0[1:-1] + dt_loc * A[0,0] * nu_loc * S0
    # Jacobi preconditioner
    Mp_diags = 1.0 / Mi.diagonal()
    Mprec = diags(Mp_diags)
    # solve Mi * Yi_int = rhs_int (use cg)
    Y0_int, info = cg(Mi, rhs_int0, x0=u[1:-1], rtol=1e-6, M=Mprec)
```

```
238     Y0[1:-1] = Y0_int
239     #Apply BC
240 
241     Y0[0] = exact_solution(0.0, t0, nu_loc)
242     Y0[-1] = exact_solution(lx, t0, nu_loc)
243 
244     U_strg.append(Y0)
245     K0, _, _ = R_term(Y0, t0)
246     K_strg.append(K0)
247 
248     for i in range (1,s):
249         K_hati = N_term(U_strg[i-1].copy(), t_loc + c_hat[i]*dt_loc)
250         Khat_strg.append(K_hati)
251         ti = t_loc + c[i] * dt_loc
252         _, g1i, g2i = R_term(u.copy(), ti)
253 
254         Si = generate_4th_order_ST(n, h_loc, g1i, g2i)
255 
256         rhs_inti = u[1:-1].copy() + dt_loc * A_hat[i+1,
             ↪  i]*K_hati[1:-1] + dt_loc * A[i,i] * Si
257         for j in range(i):
258             rhs_inti += dt * (A[i,j]*K_strg[j][1:-1] +
                 ↪  A_hat[i+1,j]*K_strg[j][1:-1])
259 
260         Yi_int, info = cg(Mi, rhs_inti, x0=u[1:-1], rtol=1e-6,
             ↪  M=Mprec)
261         Yi[1:-1] = Yi_int
262         #Apply BC
263         ti = t_loc + c[i] * dt_loc
264         Yi[0] = exact_solution(0.0, ti, nu_loc)
265         Yi[-1] = exact_solution(lx, ti, nu_loc)
266         U_strg.append(Yi)
267 
268         Ki, _, _ = R_term(Yi, ti)
269         K_strg.append(Ki)
270 
271     #Last K_hat
272     K_hats = N_term(U_strg[-1].copy(), t_loc + c_hat[-1]*dt_loc)
273     Khat_strg.append(K_hats)
274 
```

```
275     u_new = u.copy() + dt * b_hat[-1] * K_hats
276     for i in range(s):
277         u_new += dt_loc * (b[i] * K_strg[i] + b_hat[i]*Khat_strg[i])
278 
279     tf = t_loc + dt_loc
280     u_new[0] = exact_solution(0.0, tf, nu_loc)
281     u_new[-1] = exact_solution(lx, tf, nu_loc)
282 
283     return u_new
284 
285 #===Temporal Integration
    ↪ Methods=============================================================================
286 # IMEX(2,2,2) (ARS-style) coefficients:
287 def IMEX222_step(u, t_loc, dt_loc, nu_loc, h_loc):
288     gamma = (2-math.sqrt(2))/2
289     d222 = 1- (1/(2*gamma))
290     #print(f'gamma = {gamma}')
291     A = np.array([
292         [gamma,   0     ],
293         [1-gamma,   gamma]
294     ])
295     b = A[-1]
296     c = np.sum(A, axis=1)
297 
298     # Explicit (A_hat, b_hat)
299     A_hat = np.array([
300         [0,     0,     0],
301         [gamma,   0,     0],
302         [d222, 1-d222,  0]
303     ])
304     b_hat = A_hat[-1]
305     c_hat = np.sum(A_hat, axis=1)
306     return imex_step_general(u, t_loc, dt_loc, nu_loc, h_loc, A,
        ↪ A_hat, b, b_hat, c, c_hat)
307 
308 # IMEX(4,4,3) -- use the A-matrix you supplied (implicit) and a
    ↪ trimmed explicit tableau
309 def IMEX443_step(u, t_loc, dt_loc, nu_loc, h_loc):
310     A = np.array([
311         [0.5,   0,     0,     0],
```

```
312         [1/6,   0.5,   0,     0],
313         [-0.5,  0.5,   0.5,   0],
314         [1.5,  -1.5,   0.5,   0.5]
315     ])
316     b = A[-1]
317     c = np.sum(A, axis=1)
318 
319     # Explicit (A_hat, b_hat)
320     A_hat = np.array([
321         [0,     0,     0,     0, 0],
322         [0.5,   0,     0,     0, 0],
323         [11/18, 1/18,  0,     0, 0],
324         [5/6,  -5/6,   1/2,     0, 0],
325         [1/4, 7/4, 3/4, -7/4, 0]
326     ])
327     b_hat = A_hat[-1]
328     c_hat = np.sum(A_hat, axis=1)
329     return imex_step_general(u, t_loc, dt_loc, nu_loc, h_loc, A,
        ↪  A_hat, b, b_hat, c, c_hat)
330 
331 def IMEX_step(u, t_loc, dt_loc, nu_loc, h_loc, flag222=True):
332     """Wrapper to choose IMEX222 or IMEX443"""
333     if flag222:
334         return IMEX222_step(u, t_loc, dt_loc, nu_loc, h_loc)
335     else:
336         return IMEX443_step(u, t_loc, dt_loc, nu_loc, h_loc)
337 
338 def forward_euler_step(u, t_loc, dt_loc, nu_loc, h_loc):
339     """Forward Euler step"""
340     rhs_val = rhs(t_loc, u, nu_loc, h_loc)
341     u_new = u + dt_loc * rhs_val
342 
343     # Apply boundary conditions
344     u_new[0] = exact_solution(0.0, t_loc + dt_loc, nu_loc)
345     u_new[-1] = exact_solution(lx, t_loc + dt_loc, nu_loc)
346 
347     return u_new
348 
349 def rk4_step(u, t_loc, dt_loc, nu_loc, h_loc):
350     """RK4 step"""
```

```
351     # Stage 1
352     k1 = rhs(t_loc, u, nu_loc, h_loc)
353     u_temp = u + 0.5 * dt_loc * k1
354     u_temp[0] = exact_solution(0.0, t_loc + 0.5*dt_loc, nu_loc)
355     u_temp[-1] = exact_solution(lx, t_loc + 0.5*dt_loc, nu_loc)
356 
357     # Stage 2
358     k2 = rhs(t_loc + 0.5*dt_loc, u_temp, nu_loc, h_loc)
359     u_temp = u + 0.5 * dt_loc * k2
360     u_temp[0] = exact_solution(0.0, t_loc + 0.5*dt_loc, nu_loc)
361     u_temp[-1] = exact_solution(lx, t_loc + 0.5*dt_loc, nu_loc)
362 
363     # Stage 3
364     k3 = rhs(t_loc + 0.5*dt_loc, u_temp, nu_loc, h_loc)
365     u_temp = u + dt_loc * k3
366     u_temp[0] = exact_solution(0.0, t_loc + dt_loc, nu_loc)
367     u_temp[-1] = exact_solution(lx, t_loc + dt_loc, nu_loc)
368 
369     # Stage 4
370     k4 = rhs(t_loc + dt_loc, u_temp, nu_loc, h_loc)
371 
372     # Combine
373     u_new = u + dt_loc * (k1 + 2*k2 + 2*k3 + k4) / 6.0
374     u_new[0] = exact_solution(0.0, t_loc + dt_loc, nu_loc)
375     u_new[-1] = exact_solution(lx, t_loc + dt_loc, nu_loc)
376 
377     return u_new
378 
379 def mebdf_step(u, t_loc, dt_loc, nu_loc, h_loc):
380     """
381     Simplified MF-MEBDF step (backward Euler with fixed-point
        ↪ iteration)
382     """
383     # Boundary conditions at next time
384     g1 = exact_solution(0.0, t_loc + dt_loc, nu_loc)
385     g2 = exact_solution(lx, t_loc + dt_loc, nu_loc)
386 
387     # Initial guess
388     u_new = u.copy()
389     u_new[0] = g1
```

```
390         u_new[-1] = g2
391 
392         # Fixed-point iteration
393         max_iter = 20
394         tol = 1e-10
395 
396         for _ in range(max_iter):
397             f_new = rhs(t_loc + dt_loc, u_new, nu_loc, h_loc)
398             u_next = u + dt_loc * f_new
399 
400             # Apply boundary conditions
401             u_next[0] = g1
402             u_next[-1] = g2
403 
404             # Check convergence
405             if np.max(np.abs(u_next[1:-1] - u_new[1:-1])) < tol:
406                 u_new = u_next
407                 break
408 
409             u_new = u_next
410 
411         return u_new
412 
413 #===Simulation
    ↪ Function=========================================================================
414 def run_simulation(method_name, u_init, dt_loc, t_end_loc, nu_loc,
    ↪ h_loc):
415         """Run simulation with specified method and return results"""
416         print(f"\nRunning {method_name} simulation...")
417         start_time = time.time()
418 
419         u = u_init.copy()
420         solutions = [u_init.copy()]
421         times_out = [0.0]
422         errors_L2 = [calculate_errors(u_init, exact_solution(x, 0.0,
            ↪ nu_loc))[0]]
423 
424         # Calculate number of time steps
425         nit = int(np.ceil(t_end_loc / dt_loc))
426         store_interval = max(1, nit // 10)  # Store ~10 snapshots
```

```
427
428     for it in range(nit):
429         t_current = it * dt_loc
430
431         # Choose integration method
432         if method_name == 'fe':
433             u = forward_euler_step(u, t_current, dt_loc, nu_loc,
                ↪  h_loc)
434             #print(f'dt_conv = {h/max(u)}')
435             #print(f'Pe = {max(u)*h/nu}')
436         elif method_name == 'rk4':
437             u = rk4_step(u, t_current, dt_loc, nu_loc, h_loc)
438         elif method_name == 'mebdf':
439             u = mebdf_step(u, t_current, dt_loc, nu_loc, h_loc)
440         elif method_name == 'IMEX222':
441             u = IMEX_step(u, t_current, dt_loc, nu_loc, h_loc,
                ↪  flag222=True)
442         elif method_name == 'IMEX443':
443             u = IMEX_step(u, t_current, dt_loc, nu_loc, h_loc,
                ↪  flag222=False)
444         else:
445             raise ValueError(f"Unknown method {method_name}")
446
447         # Store solution and error at intervals
448         if (it + 1) % store_interval == 0:
449             solutions.append(u.copy())
450             times_out.append((it + 1) * dt_loc)
451
452             # compute error vs exact
453             u_ex = exact_solution(x, (it + 1) * dt_loc, nu_loc)
454             l2_err, _ = calculate_errors(u, u_ex)
455             errors_L2.append(l2_err)
456
457         # Print progress (guard against tiny nit)
458         if nit >= 10 and (it + 1) % max(1, nit // 10) == 0:
459             print(f"Progress: {(it+1)/nit*100:.1f}% complete")
460
461     end_time = time.time()
462     computation_time = end_time - start_time
```

```
463     print(f"{method_name} simulation completed in
        ↪ {computation_time:.2f} seconds")
464
465     return u, solutions, times_out, errors_L2, computation_time
466
467 #===Error
    ↪ Calculation=========================================================================
468 def calculate_errors(numerical_sol, exact_sol):
469     """Calculate L2 and L_infinity errors over internal points"""
470     diff = numerical_sol[1:-1] - exact_sol[1:-1]   # exclude
        ↪ boundary points
471     l2_error = np.sqrt(np.sum(diff**2))            # sum, not
        ↪ average
472     linf_error = np.max(np.abs(diff))
473     return l2_error, linf_error
474
475
476 #===Main
    ↪ Execution===========================================================================
477 # Determine which methods to run (fix typos)
478 methods_to_run = []
479 if method == 'all':
480     methods_to_run = ['fe', 'rk4', 'IMEX222', 'IMEX443']
481 else:
482     # allow user to input one of the names
483     methods_to_run = [method]
484
485 # Run simulations
486 results = {}
487 for method_name in methods_to_run:
488     final_u, solutions_list, times_list, errors_list, comp_time =
        ↪ run_simulation(
489         method_name, u_initial, dt, t_end, nu, h
490     )
491     results[method_name] = {
492         'final_u': final_u,
493         'solutions': solutions_list,
494         'times': times_list,
495         'errors_L2': errors_list,
496         'comp_time': comp_time
```

```
497     }
498
499 # Calculate exact solution at final time
500 u_exact = exact_solution(x, t_end, nu)
501
502 # Calculate and display errors
503 print("\n" + "="*60)
504 print("PERFORMANCE COMPARISON")
505 print("="*60)
506
507 for method_name in methods_to_run:
508     l2_error, linf_error =
        ↪ calculate_errors(results[method_name]['final_u'], u_exact)
509     print(f"{method_name.upper():6s} - L2 Error: {l2_error:.6e},
        ↪ L_inf Error: {linf_error:.6e}, Time:
        ↪ {results[method_name]['comp_time']:.3f}s")
510
511 #===Visualization=====================================================================
512 def plot_comparison(x_arr, results_dict, u_exact_arr, nu_loc):
513     """Plot comparison of all methods"""
514     plt.figure(figsize=(12, 8))
515
516     # Colors for different methods (include IMEX)
517     colors = {'fe': 'blue', 'rk4': 'green', 'mebdf': 'orange',
        ↪ 'IMEX222': 'purple', 'IMEX443': 'cyan'}
518     linestyles = {'fe': '-', 'rk4': '--', 'mebdf': '-.', 'IMEX222':
        ↪ '-', 'IMEX443': '--'}
519
520     # Plot each method
521     for method_name in results_dict.keys():
522         plt.plot(x_arr, results_dict[method_name]['final_u'],
523                  color=colors.get(method_name, 'black'),
524                  linestyle=linestyles.get(method_name, '-'),
525                  linewidth=2,
526                  label=f'{method_name.upper()}')
527
528     # Plot exact solution
529     plt.plot(x_arr, u_exact_arr, 'r--', linewidth=3, label='Exact')
530
531     plt.xlabel('x')
```

```
532     plt.ylabel('u(x,t)')
533     plt.title(f'Burgers Equation Solutions at t={t_end}
        ↪  (\nu={nu_loc}, dt={dt})')
534     plt.legend()
535     plt.grid(True, alpha=0.3)
536 
537     # Save the plot
538     output_path = os.path.join(script_dir,
        ↪  f'burgers_comparison_nu{nu_loc}_dt{dt}.png')
539     plt.savefig(output_path, dpi=300, bbox_inches='tight')
540     print(f"Comparison plot saved to: {output_path}")
541     plt.close()
542 
543 def plot_individual_solutions(x_arr, results_dict, u_exact_arr,
    ↪  nu_loc):
544     """Plot individual solutions for each method"""
545     for method_name in results_dict.keys():
546         plt.figure(figsize=(10, 6))
547 
548         # Plot numerical solution at different times
549         n_plots = min(5,
            ↪  len(results_dict[method_name]['solutions']))
550         time_indices = np.linspace(0,
            ↪  len(results_dict[method_name]['solutions'])-1, n_plots,
            ↪  dtype=int)
551 
552         colors_plot = plt.cm.viridis(np.linspace(0, 1, n_plots))
553 
554         for i, idx in enumerate(time_indices):
555             t_val = results_dict[method_name]['times'][idx]
556             plt.plot(x_arr,
                ↪  results_dict[method_name]['solutions'][idx],
557                      color=colors_plot[i], linewidth=2,
558                      label=f't = {t_val:.3f}')
559 
560         # Plot exact solution at final time
561         plt.plot(x_arr, u_exact_arr, 'r--', linewidth=3,
            ↪  label=f'Exact (t={t_end})')
562 
563         plt.xlabel('x')
```

```
564             plt.ylabel('u(x,t)')
565             plt.title(f'Burgers Equation - {method_name.upper()} Scheme
                ↪  (\nu={nu_loc})')
566             plt.legend()
567             plt.grid(True, alpha=0.3)
568
569             # Save the plot
570             output_path = os.path.join(script_dir,
                ↪  f'burgers_{method_name}_nu{nu_loc}.png')
571             plt.savefig(output_path, dpi=300, bbox_inches='tight')
572             print(f"{method_name.upper()} plot saved to: {output_path}")
573             plt.close()
574
575 # Create plots
576 if len(results) > 1:
577     plot_comparison(x, results, u_exact, nu)
578 plot_individual_solutions(x, results, u_exact, nu)
579
580 def plot_errors(results_dict):
581     plt.figure(figsize=(10,6))
582     for method_name, data in results_dict.items():
583         plt.semilogy(data['times'], data['errors_L2'],
            ↪  label=method_name.upper(), linewidth=2)
584     plt.xlabel('Time')
585     plt.ylabel('L2 Error')
586     plt.title('Error evolution over time')
587     plt.legend()
588     plt.grid(True, which="both", alpha=0.3)
589     output_path = os.path.join(script_dir, f'burgers_errors.png')
590     plt.savefig(output_path, dpi=300, bbox_inches='tight')
591     print(f"Error plot saved to: {output_path}")
592     plt.close()
593
594 # After simulations:
595 plot_errors(results)
596
597 #===Save Results (Space-separated
    ↪  format)==========================================================================
598 def save_results(x_arr, results_dict, u_exact_arr, script_dir_loc,
    ↪  nu_loc):
```

```
599     """Save numerical results and errors to file in space-separated
        ↪ format"""
600     for method_name in results_dict.keys():
601         # Save final solution only (no exact solution column)
602         data = np.column_stack((x_arr,
            ↪ results_dict[method_name]['final_u']))
603         header = f"# x {method_name.upper()}"
604         output_file = os.path.join(script_dir_loc,
            ↪ f'burgers_{method_name}_nu{nu_loc}.dat')
605         np.savetxt(output_file, data, header=header, fmt='%.8f',
            ↪ delimiter=' ', comments='')
606         print(f"{method_name.upper()} results saved to:
            ↪ {output_file}")
607
608         # Save error evolution over time - space-separated format
609         error_data =
            ↪ np.column_stack((results_dict[method_name]['times'],
610
                                        ↪ results_dict[method_name]['errors_L2']))
611         error_header = f"# time L2_error"
612         error_file = os.path.join(script_dir_loc,
            ↪ f'burgers_errors_{method_name}_nu{nu_loc}.txt')
613         np.savetxt(error_file, error_data, header=error_header,
            ↪ fmt='%.8f', delimiter=' ', comments='')
614         print(f"{method_name.upper()} error evolution saved to:
            ↪ {error_file}")
615
616         # Additional: Save comparison data for all methods in one
            ↪ file
617         if len(results_dict) > 1:
618             comp_data = np.column_stack((x_arr,
                ↪ results_dict[method_name]['final_u']))
619             comp_file = os.path.join(script_dir_loc,
                ↪ f'burgers_comparison_{method_name}_nu{nu_loc}.dat')
620             np.savetxt(comp_file, comp_data, fmt='%.8f', delimiter='
                ↪ ',
621                        header=f'# x {method_name.upper()}',
                            ↪ comments='')
622
623     # Save exact solution separately
```

```
624     exact_data = np.column_stack((x_arr, u_exact_arr))
625     exact_file = os.path.join(script_dir_loc,
        ↪  f'burgers_exact_nu{nu_loc}.dat')
626     np.savetxt(exact_file, exact_data, fmt='%.8f', delimiter=' ',
627                header='# x Exact', comments='')
628     print(f"Exact solution saved to: {exact_file}")
629
630
631 save_results(x, results, u_exact, script_dir, nu)
632
633 print("\nSimulation completed successfully! Data in space-separated
    ↪  format.")
```

Appendix G

Lid-Driven Cavity Code

RK4

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from mpl_toolkits.axes_grid1 import make_axes_locatable
from matplotlib.ticker import ScalarFormatter
import math
import json
import sys
import argparse
import os
import glob
import time
from scipy.sparse import diags, kron, eye, lil_matrix, linalg,
    ↪ block_diag
from scipy.sparse import identity
from scipy.sparse.linalg import spsolve
from scipy.sparse import lil_matrix, csr_matrix
from scipy.sparse.linalg import LinearOperator, cg # Corrected
    ↪ import for cg
from scipy.sparse.linalg import bicgstab
from scipy.sparse.linalg import spilu, LinearOperator
import pyamg

def generate_Ap(nx, ny, dx, dy):
    total_size = ny * nx

    # ========================
```

```
26      # Generate Apx (x-direction)
27      # ========================
28      inv_dx2 = 1.0 / (dx ** 2)
29
30      # Main diagonal
31      md_Apx = np.full(total_size, -2.0 * inv_dx2)
32      md_Apx[::nx] = -1.0 * inv_dx2       # First element of each
        ↪  x-block
33      md_Apx[nx-1::nx] = -1.0 * inv_dx2   # Last element of each
        ↪  x-block
34
35      # Upper/lower diagonals
36      upper_x = np.ones(total_size - 1) * inv_dx2
37      upper_x[nx-1::nx] = 0.0
38      lower_x = upper_x.copy()
39
40      Apx = diags([lower_x, md_Apx, upper_x], [-1, 0, 1],
        ↪  format='csr')
41
42      # ========================
43      # Generate Apy (y-direction)
44      # ========================
45      inv_dy2 = 1.0 / (dy ** 2)
46
47      # Main diagonal
48      md_Apy = np.zeros(total_size)
49
50      # Fill interior rows (-2/dy²)
51      md_Apy[nx:-nx] = -2.0 * inv_dy2
52
53      # Fill first and last ny rows (-1/dy²)
54      md_Apy[:nx] = -1.0 * inv_dy2      # First y-block
55      md_Apy[-nx:] = -1.0 * inv_dy2     # Last y-block
56
57      # Off-diagonals (+1/dy² spaced nx apart)
58      upper_y = np.ones(total_size - nx) * inv_dy2
59      lower_y = upper_y.copy()
60
61      Apy = diags([lower_y, md_Apy, upper_y], [-nx, 0, nx],
        ↪  format='csr')
```

```
62
63      # ========================
64      # Combine matrices
65      # ========================
66      Ap = Apx + Apy
67      #ml = pyamg.smoothed_aggregation_solver(Ap)
68      ml = pyamg.ruge_stuben_solver(Ap)
69
70      return Ap, ml
71
72  def amg_solver(p, S, Ap, nx, ny, ml):
73      """
74      Solves the linear system Ap x = b using PyAMG as a solver.
75      """
76      # Flatten source term (excluding ghost cells)
77      b = S[1:-1, 1:-1].ravel()
78
79      # Initial guess
80      x0 = p[1:-1, 1:-1].ravel()
81
82      # Setup PyAMG multigrid solver
83      #ml = pyamg.smoothed_aggregation_solver(Ap)
84
85      # Solve the system using AMG
86      x = ml.solve(b, x0=x0, tol=1e-6, cycle ='V')
87
88
89      # Compute residual norm
90      #residual = np.linalg.norm(Ap @ x - b)
91      residual = np.abs(Ap @ x - b).reshape((ny,nx))
92
93      # Insert result back into pressure field
94      p_new = p.copy()
95      p_new[1:-1, 1:-1] = x.reshape((ny, nx))
96
97      return p_new, residual
98
99
100 def apply_BC(u, v, Utop):
101     # BC on the u velocity
```

```
102     u[:,1] = 0.0 #Left Wall Impermiable
103     u[:,-1] = 0.0 #Right Wall Impermiable
104     u[0,:] =  -1.0 * u[1,:]   #Bottom Wall BC, Including Ghost Cells
105     u[-1,:] = 2.0 * Utop - u[-2,:] #Top Wall BC, including Ghost
        ↪ Cell
106
107     # BC on the v velocity
108     v[-1,:] = 0.0 #Top Wall Impermiable
109     v[1,:] = 0.0 #Bottom Wall Impermiable
110     v[:,0] =  -1.0 * v[:,1] #Left Wall BC, including Ghost Cell
111     v[:,-1] =  -1.0 * v[:,-2] #Right Wall
112     return u, v
113
114 def compute_rhs(u, v, nu, dx, dy):
115     ut = np.zeros_like(u)
116     vt = np.zeros_like(v)
117
118     # Compute u-component RHS
119     for i in range(2, nx+1):
120         for j in range(1, ny+1):
121             ue = 0.5 * (u[j,i+1] + u[j,i])
122             uw = 0.5 * (u[j,i] + u[j,i-1])
123             un = 0.5 * (u[j+1,i] + u[j,i])
124             us = 0.5 * (u[j,i] + u[j-1,i])
125
126             vn = 0.5 * (v[j+1, i-1] + v[j+1, i])
127             vs = 0.5 * (v[j, i-1] + v[j, i])
128
129             convection = -(ue*ue - uw*uw)/dx - (un*vn - us*vs)/dy
130             diffusion = nu * ((u[j,i-1] - 2.0 * u[j,i] +
                ↪ u[j,i+1])/dx/dx +
131                               (u[j-1,i] - 2.0 * u[j,i] +
                                  ↪ u[j+1,i])/dy/dy)
132             ut[j,i] = convection + diffusion
133
134     # Compute v-component RHS
135     for i in range(1, nx+1):
136         for j in range(2, ny+1):
137             ve = 0.5 * (v[j,i+1] + v[j,i])
138             vw = 0.5 * (v[j,i] + v[j,i-1])
```

```
139             ue = 0.5 * (u[j,i+1] + u[j-1,i+1])
140             uw = 0.5 * (u[j,i] + u[j-1,i])
141 
142             vn = 0.5 * (v[j+1, i] + v[j, i])
143             vs = 0.5 * (v[j, i] + v[j-1, i])
144 
145             convection = -(ue*ve - uw*vw)/dx - (vn*vn - vs*vs)/dy
146             diffusion = nu * ((v[j,i+1] - 2.0 * v[j,i] +
                ↪  v[j,i-1])/dx/dx +
147                               (v[j+1,i] - 2.0 * v[j,i] +
                                  ↪  v[j-1,i])/dy/dy)
148             vt[j,i] = convection + diffusion
149 
150     return ut, vt
151 
152 def NS_operator(u, v, nu, dx, dy, dpdx, dpdy):
153     diff_u = np.zeros_like(u)
154     conv_u = np.zeros_like(u)
155 
156     diff_v = np.zeros_like(v)
157     conv_v = np.zeros_like(v)
158 
159 
160     RHS_u = np.zeros_like(u)
161     RHS_v = np.zeros_like(v)
162 
163     NS_u = np.zeros_like(u)
164     NS_v = np.zeros_like(v)
165 
166 
167     # Compute u-component RHS
168     for i in range(2, nx+1):
169         for j in range(1, ny+1):
170             ue = 0.5 * (u[j,i+1] + u[j,i])
171             uw = 0.5 * (u[j,i] + u[j,i-1])
172             un = 0.5 * (u[j+1,i] + u[j,i])
173             us = 0.5 * (u[j,i] + u[j-1,i])
174 
175             vn = 0.5 * (v[j+1, i-1] + v[j+1, i])
176             vs = 0.5 * (v[j, i-1] + v[j, i])
```

```
177
178             conv_u[j,i] = -(ue*ue - uw*uw)/dx - (un*vn - us*vs)/dy
179             diff_u[j,i] = nu * ((u[j,i-1] - 2.0 * u[j,i] +
                ↪  u[j,i+1])/dx/dx +
180                                 (u[j-1,i] - 2.0 * u[j,i] +
                                    ↪  u[j+1,i])/dy/dy)
181
182             RHS_u = conv_u + diff_u
183
184     # Compute v-component RHS
185     for i in range(1, nx+1):
186         for j in range(2, ny+1):
187             ve = 0.5 * (v[j,i+1] + v[j,i])
188             vw = 0.5 * (v[j,i] + v[j,i-1])
189             ue = 0.5 * (u[j,i+1] + u[j-1,i+1])
190             uw = 0.5 * (u[j,i] + u[j-1,i])
191
192             vn = 0.5 * (v[j+1, i] + v[j, i])
193             vs = 0.5 * (v[j, i] + v[j-1, i])
194
195             conv_v[j,i] = -(ue*ve - uw*vw)/dx - (vn*vn - vs*vs)/dy
196             diff_v[j,i] = nu * ((v[j,i+1] - 2.0 * v[j,i] +
                ↪  v[j,i-1])/dx/dx +
197                                 (v[j+1,i] - 2.0 * v[j,i] +
                                    ↪  v[j-1,i])/dy/dy)
198
199             RHS_v = conv_v + diff_v
200
201     NS_u[1:-1, 2:-1] = RHS_u[1:-1, 2:-1] - dpdx
202     NS_v[2:-1, 1:-1] = RHS_v[2:-1, 1:-1] - dpdy
203
204     return NS_u, NS_v, RHS_u, RHS_v
205
206 def pressure_correction(u, v, Ap, p, dt, dx, dy, ml):
207     div = np.zeros_like(p)
208     div[1:-1, 1:-1] = (u[1:-1, 2:] - u[1:-1, 1:-1])/dx + (v[2:,
        ↪  1:-1] - v[1:-1, 1:-1])/dy
209
210     prhs = div / dt
211
```

```
212     # Use PCG instead of SOR     p, S, Ap, nx, ny
213     p_new, residual = amg_solver(p.copy(), prhs, Ap, nx, ny, ml)
214 
215     # Apply pressure correction to velocities
216     u_corr = u.copy()
217     v_corr = v.copy()
218 
219     dpdx = (p_new[1:-1, 2:-1] - p_new[1:-1, 1:-2])/dx
220     dpdy = (p_new[2:-1, 1:-1] - p_new[1:-2, 1:-1])/dy
221 
222     u_corr[1:-1, 2:-1] = u[1:-1, 2:-1] - dt * dpdx
223     v_corr[2:-1, 1:-1] = v[2:-1, 1:-1] - dt * dpdy
224 
225     u_corr, v_corr = apply_BC(u_corr, v_corr, Utop)
226 
227     return u_corr, v_corr, p_new, dpdx, dpdy, residual
228 
229 
230 #===Check Point
    ↪ Setup=============================================================================
231 checkpoint_interval = 5000  # Save checkpoint every N steps
232 checkpoint_dir = os.path.join(script_dir, "checkpoints")
233 latest_checkpoint = None
234 
235 #===Domain
    ↪ Setup=============================================================================
236 lx, ly = 1.0, 1.0
237 nx, ny = 128, 128
238 nelm = nx * ny
239 print(f'Nx = {nx}')
240 x = np.linspace(0, lx, nx)
241 y = np.linspace(0, ly, ny)
242 dx, dy = lx/nx, ly/ny
243 print(f' dx= {dx}')
244 xx, yy = np.meshgrid(x, y)
245 
246 #===BC for
    ↪ Pressure==========================================================================
247 App, ml = generate_Ap(nx, ny, dx, dy)
248 print(f'ml = {ml}')
```

```
249
250 #===Flow
    ↪ parameters=========================================================================
251 Utop = 1.0
252 Ubot = 0.0
253 Vleft = 0.0
254 Vright = 0.0
255 Re = 100            # Reynolds number
256 nu = lx * Utop / Re  # Kinematic viscosity = 1 / Re, in practise
257
258 #===Time
    ↪ parameters=========================================================================
259 t = 0.0
260 t_end = 10
261 nit = int(10000)
262
263 dt = np.float64(t_end / nit)
264 print(dt)
265
266 dt1 = 0.5/nu /(1.0/dx/dx + 1.0/dy/dy)
267 dt2 = 2.0 * nu / (Utop**2)
268
269 # Initialize arrays
270 p = np.zeros([ny+2, nx+2])
271 u = np.zeros([ny+2, nx+2])
272 v = np.zeros([ny+2, nx+2])
273 u_old = np.zeros_like(u)
274 v_old = np.zeros_like(v)
275
276 u[-1,:] = Utop
277
278 p_int = np.zeros([ny, nx])
279 u_int = np.zeros([ny, nx-1])
280 v_int = np.zeros([ny-1, nx])
281
282
283 Ek0 = 1e-6
284 dV = dx * dy
285 Ek_store = [Ek0]
286 t_sim = [0.0]
```

```
287
288 residuals_u = []
289 residuals_v = []
290 residuals_p = []
291 residuals_Ek = []
292 residuals_omega = []
293
294 resNSu = []
295 resNSv = []
296 resNSp = []
297
298 Uwallt = [0.0]
299 Uwall_mean_history = []  # To store mean Uwall over time
300 residuals_Uwall = []     # To store residuals of Uwall
301 counter = 0
302
303 #nsave = 5 #Number of time steps to save
304 dt_i = dt
305
306 if os.path.exists(checkpoint_dir):
307     checkpoint_files = sorted(glob.glob(os.path.join(checkpoint_dir,
      ↪  "checkpoint_Re*.npz")))
308     if checkpoint_files:
309         latest_checkpoint = checkpoint_files[-1]
310
311 if latest_checkpoint:
312     print(f"Found checkpoint file: {latest_checkpoint}")
313     result = load_checkpoint(latest_checkpoint)
314     if result:
315         (t, u, v, p, Ek_store, t_sim, residuals_u, residuals_v,
          ↪  residuals_p,
316          residuals_omega, residuals_Ek, Uwall_mean_history,
317          Re, nx, ny, dt, nu, Utop) = result
318
319         # Recreate grid parameters since they weren't saved
320         x = np.linspace(0, lx, nx)
321         y = np.linspace(0, ly, ny)
322         dx, dy = lx/nx, ly/ny
323         xx, yy = np.meshgrid(x, y)
324
```

```
325             # Reinitialize Ap, Ae, Aw, An, As arrays
326             Ap = np.zeros([ny+2, nx+2])
327             Ae = 1.0/dx/dx * np.ones([ny+2, nx+2])
328             Aw = 1.0/dx/dx * np.ones([ny+2, nx+2])
329             An = 1.0/dy/dy * np.ones([ny+2, nx+2])
330             As = 1.0/dy/dy * np.ones([ny+2, nx+2])
331 
332             # Boundary conditions for pressure
333             Aw[1:-1, 1] = 0.0
334             Ae[1:-1, -2] = 0.0
335             An[-2, 1:-1] = 0.0
336             As[1, 1:-1] = 0.0
337             Ap = - (Ae + Aw + An + As)
338 
339             # Set old variables for residual calculation
340             u_old, v_old, p_old = u.copy(), v.copy(), p.copy()
341             omega = np.zeros((ny, nx))  # Will be recalculated in first
                ↪  iteration
342             omega_old = omega.copy()
343             Uwall_old = abs(u[-1,1:] + u[-2,1:]) / 2
344             Ek0 = Ek_store[-1]
345 
346             print(f"Restarted simulation from t={t:.4f}")
347         else:
348             print("Failed to load checkpoint, starting from scratch")
349             # Initialize as before
350 else:
351     print("No checkpoint found, starting new simulation")
352     # Initialize as before
353 
354 #=== RK4 AMG
    ↪  ================================================================================
355 start_time = time.time()
356 cont_it = int(0)
357 
358 for cont_it in range(1, nit+1):
359     p0 = p.copy()
360     u, v = apply_BC(u, v, Utop)
361 
362     # Stage 1: Compute explicit RHS (advection + diffusion only)
```

```
363     k1u, k1v = compute_rhs(u, v, nu, dx, dy)
364     u1 = u + 0.5*dt * k1u
365     v1 = v + 0.5*dt * k1v
366     u1, v1 = apply_BC(u1, v1, Utop)
367     u1, v1, p1, _, _, _ = pressure_correction(u1, v1, App, p0,
        ↪  0.5*dt, dx, dy, ml)  # Project here
368
369     # Stage 2: Repeat with u1, v1
370     k2u, k2v = compute_rhs(u1, v1, nu, dx, dy)
371     u2 = u + 0.5*dt * k2u
372     v2 = v + 0.5*dt * k2v
373     u2, v2 = apply_BC(u2, v2, Utop)
374     u2, v2, p2, _, _, _ = pressure_correction(u2, v2, App, p0,
        ↪  0.5*dt, dx, dy, ml)  # Project again
375
376     # Stage 3: Full step
377     k3u, k3v = compute_rhs(u2, v2, nu, dx, dy)
378     u3 = u + dt * k3u
379     v3 = v + dt * k3v
380     u3, v3 = apply_BC(u3, v3, Utop)
381     u3, v3, p3, _, _, _ = pressure_correction(u3, v3, App, p0, dt,
        ↪  dx, dy, ml)  # Project again
382
383     # Stage 4: Final explicit RHS
384     k4u, k4v = compute_rhs(u3, v3, nu, dx, dy)
385
386     # RK4 weighted update (without pressure yet)
387     ut = u + (dt/6.0) * (k1u + 2*k2u + 2*k3u + k4u)
388     vt = v + (dt/6.0) * (k1v + 2*k2v + 2*k3v + k4v)
389     ut, vt = apply_BC(ut, vt, Utop)
390
391     # FINAL pressure correction (optional, but helps stability)
392     ut, vt, p, _, _, Rp = pressure_correction(ut, vt, App, p0, dt,
        ↪  dx, dy, ml)
393
394     # Update variables
395     u, v = ut.copy(), vt.copy()
396     u, v = apply_BC(u,v, Utop)
397
398     #NS_u, NS_v, _, _ = NS_operator(u, v, nu, dx, dy, dpdxn, dpdyn)
```

```
399
400         Ru = np.zeros([ny, nx+1])
401         Rv = np.zeros([ny+1, nx])
402
403         #Ru = (u[1:-1, 2:-1]-u_old[1:-1, 2:-1]) / dt - NS_u[1:-1, 2:-1]
404         #Rv = (v[2:-1, 1:-1]-v_old[2:-1, 1:-1]) / dt - NS_v[2:-1, 1:-1]
405         #NS_p = pressure_residual(p, u, v, dx, dy, dt, nx, ny, Ap, Ae,
            ↪  Aw, An, As)
406
407         Ru = abs(u[1:-1, 2:-1]-u_old[1:-1, 2:-1]) / np.max(u_old[1:-1,
            ↪  2:-1])
408         Rv = abs(v[2:-1, 1:-1]-v_old[2:-1, 1:-1]) / np.max(u_old[1:-1,
            ↪  2:-1])
409
410
411         Uwalli = abs(u[-1,1:] + u[-2,1:]) / 2
412         Uwallt.append(Uwalli)
413
414         # Calculate and store mean Uwall
415         Uwall_mean = np.mean(Uwalli)
416         Uwall_mean_history.append(Uwall_mean)
417
418         #Vorticity Calculations
419         dvdx = np.zeros((ny, nx))
420         dudy = np.zeros((ny, nx))
421         omega = np.zeros((ny, nx))
422         # Calculate dvdx for all j and i at once
423         dvdx = (v[1:-1, 2:] - v[1:-1, :-2]) / (2 * dx)
424
425         # Calculate dudy for all j and i at once
426         dudy = (u[2:, 1:-1] - u[:-2, 1:-1]) / (2 * dy)
427
428         # Compute vorticity
429         omega = dvdx - dudy
430
431         ui_centers = 0.5*(u[1:-1,1:-1] + u[1:-1,2:])  # Average u to
            ↪  cell centers
432         vi_centers = 0.5*(v[1:-1,1:-1] + v[2:,1:-1])
433
434         Eki = 0.5 * np.sum(ui_centers**2 + vi_centers**2) / nelm
```

```
435     #tol = abs(Ek0 - Eki) / Ek0
436     Ek_store.append(Eki)
437 
438     if len(Ek_store) > 1:
439         du = np.linalg.norm(u - u_old) / np.linalg.norm(u_old)  if
            ↪  'u_old' in locals() else 1.0
440         dv = np.linalg.norm(v - v_old) / np.linalg.norm(v_old) if
            ↪  'v_old' in locals() else 1.0
441         dp = np.linalg.norm(p - p_old) / np.linalg.norm(p_old) if
            ↪  'p_old' in locals() else 1.0
442         domega = np.linalg.norm(omega - omega_old) /
            ↪  np.linalg.norm(omega_old) if 'omega_old' in locals()
            ↪  else 1.0
443         dEk = abs(Ek0 - Eki) / Ek0  if len(Ek_store) > 1 else 1.0
444 
445 
446         residuals_u.append(du)
447         residuals_v.append(dv)
448         residuals_p.append(dp)
449         residuals_omega.append(domega)
450         residuals_Ek.append(dEk)
451 
452 
453         dur = np.max(np.abs(Ru))  # Maximum absolute residual for
            ↪  u-momentum
454         dvr = np.max(np.abs(Rv))  # Maximum absolute residual for
            ↪  v-momentum
455         dpr = np.max(np.abs(Rp))  # Maximum absolute residual for p
456         #dur = np.linalg.norm(NS_u)
457         #dvr = np.linalg.norm(NS_v)
458 
459         resNSu.append(dur)
460         resNSv.append(dvr)
461         resNSp.append(dpr)
462 
463     Ek0 = Eki
464     u_old, v_old, p_old = u.copy(), v.copy(), p.copy()
465     omega_old = omega.copy()
466     Uwall_old = Uwalli.copy()
467     counter += 1
```

```
468
469         t = np.float64(cont_it * dt)
470         t_sim.append(t)
471         dt_i = min(dt, t_end - t)
472
473 end_time = time.time()
474 elapsed_time = end_time - start_time
475
476 simtime_filename = f"simulation_time_Re{Re}_{nx}_{ny}_{dt:.4f}.txt"
477 simtime_path = os.path.join(script_dir, simtime_filename)
478
479 # Write the elapsed time to this new file
480 with open(simtime_path, 'w') as f:
481     f.write(f"Simulation elapsed time (seconds):
        ↪  {elapsed_time:.2f}\n")
482     f.write(f"Re = {Re}, nx = {nx}, ny = {ny}, dt = {dt:.18f}\n")
483
484 print(f"Simulation time saved to {simtime_path}")
485 print(f"Total simulation time: {elapsed_time:.2f} seconds")
486
487 #Perform th final Post Processing
488 final_message = postProcess(u, v, dx, dy, nx, ny, lx, ly, x, y, xx,
    ↪  yy, t,  Re, nu, Utop, p, omega, Ek_store, residuals_u,
    ↪  residuals_v, residuals_p, residuals_Ek, t_sim,
    ↪  Uwall_mean_history, script_dir, resNSu,resNSv, resNSp, Ru, Rv,
    ↪  Rp)
489
```

IMEX

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from mpl_toolkits.axes_grid1 import make_axes_locatable
from matplotlib.ticker import ScalarFormatter
import math
import json
import sys
import argparse
import os
import glob
import time
from scipy.sparse import diags, kron, eye, lil_matrix, linalg, block_diag
from scipy.sparse import identity
from scipy.sparse.linalg import spsolve
from scipy.sparse import lil_matrix, csr_matrix
from scipy.sparse.linalg import LinearOperator, cg # Corrected import for cg
from scipy.sparse.linalg import bicgstab
from scipy.sparse.linalg import spilu, LinearOperator
import pyamg

def generate_Ap(nx, ny, dx, dy):
    total_size = ny * nx

    # ==========================
    # Generate Apx (x-direction)
    # ==========================
    inv_dx2 = 1.0 / (dx ** 2)

    # Main diagonal
    md_Apx = np.full(total_size, -2.0 * inv_dx2)
    md_Apx[::nx] = -1.0 * inv_dx2       # First element of each x-block
    md_Apx[nx-1::nx] = -1.0 * inv_dx2   # Last element of each x-block

    # Upper/lower diagonals
```

```
36      upper_x = np.ones(total_size - 1) * inv_dx2
37      upper_x[nx-1::nx] = 0.0
38      lower_x = upper_x.copy()
39
40      Apx = diags([lower_x, md_Apx, upper_x], [-1, 0, 1],
        ↪  format='csr')
41
42      # =========================
43      # Generate Apy (y-direction)
44      # =========================
45      inv_dy2 = 1.0 / (dy ** 2)
46
47      # Main diagonal
48      md_Apy = np.zeros(total_size)
49
50      # Fill interior rows (-2/dy²)
51      md_Apy[nx:-nx] = -2.0 * inv_dy2
52
53      # Fill first and last ny rows (-1/dy²)
54      md_Apy[:nx] = -1.0 * inv_dy2       # First y-block
55      md_Apy[-nx:] = -1.0 * inv_dy2      # Last y-block
56
57      # Off-diagonals (+1/dy² spaced nx apart)
58      upper_y = np.ones(total_size - nx) * inv_dy2
59      lower_y = upper_y.copy()
60
61      Apy = diags([lower_y, md_Apy, upper_y], [-nx, 0, nx],
        ↪  format='csr')
62
63      # =========================
64      # Combine matrices
65      # =========================
66      Ap = Apx + Apy
67      #ml = pyamg.smoothed_aggregation_solver(Ap)
68      ml = pyamg.ruge_stuben_solver(Ap)
69
70      return Ap, ml
71
72  def cg_solver(p, S, App, nx, ny):
73
```

```
74      # Flatten the source term (excluding ghost cells)
75      b = S[1:-1, 1:-1].ravel()
76
77      # Initial guess (flattened, excluding ghost cells)
78      p0 = p[1:-1, 1:-1].ravel()
79
80      # Solve using PCG with diagonal preconditioner
81      M = diags(1.0/App.diagonal())  # Jacobi preconditioner
82
83      p_flat, info = cg(App, b, x0=p0, rtol=1e-06, M=M)
84
85      if info != 0:
86          print(f"PCG did not converge: info = {info}")
87      # Compute residual norm
88      residual = np.abs(App @ p_flat - b).reshape((ny,nx))
89      #print(f'DimentionsResi_p = {np.shape(residual)}')
90
91      # Reshape solution and maintain ghost cells
92      p_new = p.copy()
93      p_new[1:-1, 1:-1] = p_flat.reshape((ny, nx))
94
95      return p_new, residual
96
97  def amg_solver(p, S, Ap, nx, ny, ml):
98      """
99      Solves the linear system Ap x = b using PyAMG as a solver.
100     """
101     # Flatten source term (excluding ghost cells)
102     b = S[1:-1, 1:-1].ravel()
103
104     # Initial guess
105     x0 = p[1:-1, 1:-1].ravel()
106
107     # Setup PyAMG multigrid solver
108     #ml = pyamg.smoothed_aggregation_solver(Ap)
109
110     # Solve the system using AMG
111     x = ml.solve(b, x0=x0, tol=1e-6, cycle ='V')
112
113
```

```
114     # Compute residual norm
115     #residual = np.linalg.norm(Ap @ x - b)
116     residual = np.abs(Ap @ x - b).reshape((ny,nx))
117 
118     # Insert result back into pressure field
119     p_new = p.copy()
120     p_new[1:-1, 1:-1] = x.reshape((ny, nx))
121 
122     return p_new, residual
123 
124 
125 
126 def pressure_correction(u, v, Ap, p, dt, dx, dy, ml):
127     div = np.zeros_like(p)
128     div[1:-1, 1:-1] = (u[1:-1, 2:] - u[1:-1, 1:-1])/dx + (v[2:,
        ↪  1:-1] - v[1:-1, 1:-1])/dy
129 
130     prhs = div / dt
131 
132     # Use AMG instead of SOR     p, S, Ap, nx, ny
133     p_new, residual = amg_solver(p.copy(), prhs, Ap, nx, ny, ml)
134 
135     # Apply pressure correction to velocities
136     u_corr = u.copy()
137     v_corr = v.copy()
138 
139     dpdx = (p_new[1:-1, 2:-1] - p_new[1:-1, 1:-2])/dx
140     dpdy = (p_new[2:-1, 1:-1] - p_new[1:-2, 1:-1])/dy
141 
142     u_corr[1:-1, 2:-1] = u[1:-1, 2:-1] - dt * dpdx
143     v_corr[2:-1, 1:-1] = v[2:-1, 1:-1] - dt * dpdy
144 
145     u_corr, v_corr = apply_BC(u_corr, v_corr, Utop)
146 
147     return u_corr, v_corr, p_new, dpdx, dpdy, residual
148 
149 
150 def apply_BC(u, v, Utop):
151     # BC on the u velocity
152 
```

```
153     u[:,1] = 0.0 #Left Wall Impermiable
154     u[:,-1] = 0.0 #Right Wall Impermiable
155     u[0,:] =  -1.0 * u[1,:]   #Bottom Wall BC, Including Ghost Cells
156     u[-1,:] = 2.0 * Utop - u[-2,:] #Top Wall BC, including Ghost
        ↪  Cell
157
158     # BC on the v velocity
159     v[-1,:] = 0.0 #Top Wall Impermiable
160     v[1,:] = 0.0 #Bottom Wall Impermiable
161     v[:,0] =  -1.0 * v[:,1] #Left Wall BC, including Ghost Cell
162     v[:,-1] =  -1.0 * v[:,-2] #Right Wall
163     return u, v
164
165 def compute_explicit(u, v, dx, dy):
166     RHSu_hat = np.zeros_like(u)
167     RHSv_hat = np.zeros_like(v)
168
169     # Compute u-component RHS
170     for i in range(2, nx+1):
171         for j in range(1, ny+1):
172             ue = 0.5 * (u[j,i+1] + u[j,i])
173             uw = 0.5 * (u[j,i] + u[j,i-1])
174             un = 0.5 * (u[j+1,i] + u[j,i])
175             us = 0.5 * (u[j,i] + u[j-1,i])
176
177             vn = 0.5 * (v[j+1, i-1] + v[j+1, i])
178             vs = 0.5 * (v[j, i-1] + v[j, i])
179
180             RHSu_hat[j,i] = -(ue*ue - uw*uw)/dx - (un*vn - us*vs)/dy
181
182     # Compute v-component RHS
183     for i in range(1, nx+1):
184         for j in range(2, ny+1):
185             ve = 0.5 * (v[j,i+1] + v[j,i])
186             vw = 0.5 * (v[j,i] + v[j,i-1])
187             ue = 0.5 * (u[j,i+1] + u[j-1,i+1])
188             uw = 0.5 * (u[j,i] + u[j-1,i])
189
190             vn = 0.5 * (v[j+1, i] + v[j, i])
191             vs = 0.5 * (v[j, i] + v[j-1, i])
```

```
192
193             RHSv_hat[j,i] = -(ue*ve - uw*vw)/dx - (vn*vn - vs*vs)/dy
194
195     return RHSu_hat, RHSv_hat
196
197 def compute_implicit(u, v, nu, dx, dy):
198     RHSu = np.zeros_like(u)
199     RHSv = np.zeros_like(v)
200
201     # Compute u-component RHS
202     for i in range(2, nx+1):
203         for j in range(1, ny+1):
204             RHSu[j,i] = nu * ((u[j,i-1] - 2.0 * u[j,i] +
                ↪  u[j,i+1])/dx/dx +
205                               (u[j-1,i] - 2.0 * u[j,i] +
                                  ↪  u[j+1,i])/dy/dy)
206
207     # Compute v-component RHS
208     for i in range(1, nx+1):
209         for j in range(2, ny+1):
210             RHSv[j,i] = nu * ((v[j,i+1] - 2.0 * v[j,i] +
                ↪  v[j,i-1])/dx/dx +
211                               (v[j+1,i] - 2.0 * v[j,i] +
                                  ↪  v[j-1,i])/dy/dy)
212
213     return RHSu, RHSv
214
215 def NS_operator(u, v, nu, dx, dy, dpdx, dpdy):
216     diff_u = np.zeros_like(u)
217     conv_u = np.zeros_like(u)
218
219     diff_v = np.zeros_like(v)
220     conv_v = np.zeros_like(v)
221
222
223     RHS_u = np.zeros_like(u)
224     RHS_v = np.zeros_like(v)
225
226     NS_u = np.zeros_like(u)
227     NS_v = np.zeros_like(v)
```

```
228     NS_p = np.zeros_like(p)
229 
230 
231     # Compute u-component RHS
232     for i in range(2, nx+1):
233         for j in range(1, ny+1):
234             ue = 0.5 * (u[j,i+1] + u[j,i])
235             uw = 0.5 * (u[j,i] + u[j,i-1])
236             un = 0.5 * (u[j+1,i] + u[j,i])
237             us = 0.5 * (u[j,i] + u[j-1,i])
238 
239             vn = 0.5 * (v[j+1, i-1] + v[j+1, i])
240             vs = 0.5 * (v[j, i-1] + v[j, i])
241 
242             conv_u[j,i] = -(ue*ue - uw*uw)/dx - (un*vn - us*vs)/dy
243             diff_u[j,i] = nu * ((u[j,i-1] - 2.0 * u[j,i] +
                 ↪  u[j,i+1])/dx/dx +
244                                 (u[j-1,i] - 2.0 * u[j,i] +
                                     ↪  u[j+1,i])/dy/dy)
245 
246             RHS_u = conv_u + diff_u
247 
248     # Compute v-component RHS
249     for i in range(1, nx+1):
250         for j in range(2, ny+1):
251             ve = 0.5 * (v[j,i+1] + v[j,i])
252             vw = 0.5 * (v[j,i] + v[j,i-1])
253             ue = 0.5 * (u[j,i+1] + u[j-1,i+1])
254             uw = 0.5 * (u[j,i] + u[j-1,i])
255 
256             vn = 0.5 * (v[j+1, i] + v[j, i])
257             vs = 0.5 * (v[j, i] + v[j-1, i])
258 
259             conv_v[j,i] = -(ue*ve - uw*vw)/dx - (vn*vn - vs*vs)/dy
260             diff_v[j,i] = nu * ((v[j,i+1] - 2.0 * v[j,i] +
                 ↪  v[j,i-1])/dx/dx +
261                                 (v[j+1,i] - 2.0 * v[j,i] +
                                     ↪  v[j-1,i])/dy/dy)
262 
263             RHS_v = conv_v + diff_v
```

```
264
265     NS_u[1:-1, 2:-1] = RHS_u[1:-1, 2:-1] - dpdx
266     NS_v[2:-1, 1:-1] = RHS_v[2:-1, 1:-1] - dpdy
267     #print(f'NS_u(y,x)={np.shape(NS_u)}')
268     return NS_u, NS_v
269
270
271
272 def generate_Lu(nx, ny, dx, dy, Utop):
273     """
274     Generate Laplacian operators for a staggered grid of size ny × (nx-1).
275
276     Parameters:
277         nx (int): Number of x grid points (including boundaries)
278         ny (int): Number of y grid points (including boundaries)
279         dx (float): Grid spacing in x-direction
280         dy (float): Grid spacing in y-direction
281
282     Returns:
283         Lu (csr_matrix): Combined Laplacian operator (1/dx² Lux + 1/dy² Luy)
284         Lux (csr_matrix): x-direction component (unscaled)
285         Luy (csr_matrix): y-direction component (unscaled)
286     """
287     nux = nx - 1
288     N = ny * nux  # total number of u points
289
290     # --- x-direction Laplacian block (1D) ---
291     main_diag_x = -2 * np.ones(nux)
292     off_diag_x = np.ones(nux - 1)
293     Lux1D = diags([off_diag_x, main_diag_x, off_diag_x], offsets=[-1, 0, 1], format='csr')
294
295     # 2D x-direction Laplacian using Kronecker product
296     Iy = identity(ny, format='csr')
297     Lux = kron(Iy, Lux1D, format='csr')
298
299     # --- y-direction Laplacian block (1D) with boundary handling ---
```

```
300     main_diag_y = -2 * np.ones(ny)
301     main_diag_y[0] = -3
302     main_diag_y[-1] = -3
303     off_diag_y = np.ones(ny - 1)
304     Luy1D = diags([off_diag_y, main_diag_y, off_diag_y],
        ↪  offsets=[-1, 0, 1], format='csr')
305
306     # 2D y-direction Laplacian using Kronecker product
307     Ix = identity(nux, format='csr')
308     Luy = kron(Luy1D, Ix, format='csr')
309
310     # --- Combine the Laplacians ---
311     Lu = (1 / dx**2) * Lux + (1 / dy**2) * Luy
312
313     Su = np.zeros(N)
314     Su[(N-nux):] = 2*Utop /dy/dy
315     return Lu, Su, Lux, Luy
316
317 def generate_Lv(nx, ny, dx, dy):
318     """
319     Generate Laplacian operators for a staggered grid of size (ny-1)
        ↪  × nx (for v).
320
321     Parameters:
322         nx (int): Number of x grid points (including boundaries)
323         ny (int): Number of y grid points (including boundaries)
324         dx (float): Grid spacing in x-direction
325         dy (float): Grid spacing in y-direction
326
327     Returns:
328         Lv (csr_matrix): Combined Laplacian operator (1/dx² Lvx +
            ↪  1/dy² Lvy)
329         Lvx (csr_matrix): x-direction component (unscaled)
330         Lvy (csr_matrix): y-direction component (unscaled)
331     """
332     nvx = nx
333     nvy = ny - 1
334     Nv = nvx * nvy  # total number of v points
335
```

```
336     # --- x-direction Laplacian block (1D) with boundary treatment ---
337     main_diag_x = -2 * np.ones(nvx)
338     main_diag_x[0] = -3
339     main_diag_x[-1] = -3
340     off_diag_x = np.ones(nvx - 1)
341     Lvx1D = diags([off_diag_x, main_diag_x, off_diag_x], offsets=[-1, 0, 1], format='csr')
342 
343     # 2D x-direction Laplacian using Kronecker product
344     Iy = identity(nvy, format='csr')
345     Lvx = kron(Iy, Lvx1D, format='csr')
346 
347     # --- y-direction Laplacian block (1D) ---
348     main_diag_y = -2 * np.ones(nvy)
349     off_diag_y = np.ones(nvy - 1)
350     Lvy1D = diags([off_diag_y, main_diag_y, off_diag_y], offsets=[-1, 0, 1], format='csr')
351 
352     # 2D y-direction Laplacian using Kronecker product
353     Ix = identity(nvx, format='csr')
354     Lvy = kron(Lvy1D, Ix, format='csr')
355 
356     # --- Combine the Laplacians ---
357     Lv = (1 / dx**2) * Lvx + (1 / dy**2) * Lvy
358 
359     return Lv, Lvx, Lvy
360 
361 
362 def do_1st_RK_Step(u, v, dx, dy, dt, nu, A, A_hat, Iu, Iv, Lu, Lv, Su, p, Utop, App):
363     #Variable Initialization
364     u0_int = u.copy()
365     v0_int = v.copy()
366     Hu0 = np.zeros_like(u0_int)
367     Hv0 = np.zeros_like(v0_int)
368     Ku_hat_int = np.zeros_like(u0_int)
369     Kv_hat_int = np.zeros_like(v0_int)
370 
371     Uut = np.zeros_like(u)
```

```
372     Uvt = np.zeros_like(v)
373
374     p0 = p.copy()
375     Ku_hat = np.zeros_like(u) #[ny+2, nx+2]
376     Kv_hat = np.zeros_like(v) #[ny+2, nx+2]
377
378     u0_int = u[1:-1, 2:-1]
379     v0_int = v[2:-1, 1:-1]
380
381     Ku_hat, Kv_hat = compute_explicit(u, v, dx, dy)
382
383     Ku_hat_int = Ku_hat[1:-1, 2:-1] #[ny, nx-1]
384     Kv_hat_int = Kv_hat[2:-1, 1:-1] #[ny-1, nx]
385
386     Hu0 = u0_int + dt * A_hat[1,0] * Ku_hat_int #Only the interior
        ↪ points of u and Ku_hat [ny, nx-1]
387     Hv0 = v0_int + dt * A_hat[1,0] * Kv_hat_int #Only the interior
        ↪ points of v and  Kv_hat [ny-1, nx]
388
389     Mu = Iu - nu * dt * A[0,0] * Lu #[ny * (nx-1), ny * (nx-1)]
390     Mv = Iv - nu * dt * A[0,0] * Lv #[(ny-1) * nx, (ny-1) * nx]
391
392     b0_int = Hu0.ravel() + nu * dt * A[0,0] * Su
393
394     Mpu = diags(1.0/Mu.diagonal())  # Jacobi preconditioner
395     Uu_vec, info = cg(Mu, b0_int, x0=u0_int.ravel(), rtol=1e-06,
        ↪ M=Mpu)
396
397     Mpv = diags(1.0/Mv.diagonal())  # Jacobi preconditioner
398     Uv_vec, info = cg(Mv, Hv0.ravel(), x0=v0_int.ravel(),
        ↪ rtol=1e-06, M=Mpv)
399
400     Uut_int = Uu_vec.reshape(u0_int.shape) #[ny, nx-1]
401     Uvt_int = Uv_vec.reshape(v0_int.shape) #[ny-1, nx]
402
403     Uut[1:-1, 2:-1] = Uut_int
404     Uvt[2:-1, 1:-1] = Uvt_int
405
406     Uut, Uvt = apply_BC(Uut, Uvt, Utop)
407
```

```
408     #dt0 = (A[0,0]) * dt
409
410     Uu, Uv, _, dpdx0, dpdy0, _ = pressure_correction(Uut, Uvt, App,
        ↪  p0, dt, dx, dy, ml)
411     #print(f'dpdx0len = {np.shape(dpdx0)}')
412     Uu, Uv = apply_BC(Uu, Uv, Utop)
413
414     Ku, Kv = compute_implicit(Uu, Uv, nu, dx, dy)
415
416     #Ku[1:-1, 2:-1] = Ku[1:-1, 2:-1] - dpdx0
417     #Kv[2:-1, 1:-1] = Kv[2:-1, 1:-1] - dpdy0
418
419     return Ku, Kv, Uu, Uv, Ku_hat, Kv_hat
420
421 #===Check Point
    ↪  setup=============================================================================
422 checkpoint_interval = 1000  # Save checkpoint every N steps
423 checkpoint_dir = os.path.join(script_dir, "checkpoints")
424 latest_checkpoint = None
425
426 #===Domain
    ↪  Setup=============================================================================
427 lx, ly = 1.0, 1.0
428 nx, ny = 128,128
429 nelm = nx * ny
430 x = np.linspace(0, lx, nx)
431 y = np.linspace(0, ly, ny)
432 dx, dy = lx/nx, ly/ny
433 xx, yy = np.meshgrid(x, y)
434
435 print(f'dx = {dx}')
436
437 #===BC for
    ↪  Pressure==========================================================================
438 App, ml = generate_Ap(nx, ny, dx, dy)
439 print(f'ml = {ml}')
440
441 #===IMEX
    ↪  Paramethers=======================================================================
442 A = np.array([
```

```
    [0.5,   0,     0,     0],
    [1/6,   0.5,   0,     0],
    [-0.5,  0.5,   0.5,   0],
    [1.5,  -1.5,   0.5,   0.5]
])
b = A[-1]
c = np.sum(A, axis=1)

# Explicit (A_hat, b_hat)
A_hat = np.array([
    [0,     0,     0,     0, 0],
    [0.5,   0,     0,     0, 0],
    [11/18, 1/18,  0,     0, 0],
    [5/6,  -5/6,   1/2,     0, 0],
    [1/4, 7/4, 3/4, -7/4, 0]
])
b_hat = A_hat[-1]
c_hat = np.sum(A_hat, axis=1)


#===Flow parameters==========================================================================
Utop = 1.0
Ubot = 0.0
Vleft = 0.0
Vright = 0.0
Re = 100            # Reynolds number
nu = lx * Utop / Re  # Kinematic viscosity = 1 / Re, in practise

#===Time parameters==========================================================================
t = 0.0
t_end = 10
nit = int(500)

dt = np.float64(t_end / nit)
print(f'dt = {dt}')
print(f'Nx=Ny = {nx}')

dt1 = 0.5/nu /(1.0/dx/dx + 1.0/dy/dy)
```

```
dt2 = 2.0 * nu / (Utop**2)

Lu, Su, _, _ = generate_Lu(nx, ny, dx, dy, Utop)
print(f'Lu = {Lu}')
Lv, _, _ = generate_Lv(nx, ny, dx, dy)
Iu = eye(ny * (nx-1))
Iv = eye((ny-1) * nx)
#print(f'Lu={Lu}')

# Initialize arrays
p = np.zeros([ny+2, nx+2])
u = np.zeros([ny+2, nx+2])
v = np.zeros([ny+2, nx+2])
u_old = np.zeros_like(u)
v_old = np.zeros_like(v)

u[-1,:] = Utop

p_int = np.zeros([ny, nx])
u_int = np.zeros([ny, nx-1])
v_int = np.zeros([ny-1, nx])


Ek0 = 1e-6
dV = dx * dy
Ek_store = [Ek0]
t_sim = [0.0]
s = A.shape[0]

residuals_u = []
residuals_v = []
residuals_p = []
residuals_Ek = []
residuals_omega = []

resNSu = []
resNSv = []
resNSp = []

Uwallt = [0.0]
```

```
521 Uwall_mean_history = []  # To store mean Uwall over time
522 residuals_Uwall = []     # To store residuals of Uwall
523 counter = 0
524
525 nsave = 5 #Number of time steps to save
526
527 if os.path.exists(checkpoint_dir):
528     checkpoint_files = sorted(glob.glob(os.path.join(checkpoint_dir,
      ↪  "checkpoint_Re*.npz")))
529     if checkpoint_files:
530         latest_checkpoint = checkpoint_files[-1]
531
532 if latest_checkpoint:
533     print(f"Found checkpoint file: {latest_checkpoint}")
534     result = load_checkpoint(latest_checkpoint)
535     if result:
536         (t, u, v, p, Ek_store, t_sim, residuals_u, residuals_v,
          ↪  residuals_p,
537          residuals_omega, residuals_Ek, Uwall_mean_history,
538          Re, nx, ny, dt, nu, Utop) = result
539
540         # Recreate grid parameters since they weren't saved
541         x = np.linspace(0, lx, nx)
542         y = np.linspace(0, ly, ny)
543         dx, dy = lx/nx, ly/ny
544         xx, yy = np.meshgrid(x, y)
545
546         # Reinitialize Ap, Ae, Aw, An, As arrays
547         Ap = np.zeros([ny+2, nx+2])
548         Ae = 1.0/dx/dx * np.ones([ny+2, nx+2])
549         Aw = 1.0/dx/dx * np.ones([ny+2, nx+2])
550         An = 1.0/dy/dy * np.ones([ny+2, nx+2])
551         As = 1.0/dy/dy * np.ones([ny+2, nx+2])
552
553         # Boundary conditions for pressure
554         Aw[1:-1, 1] = 0.0
555         Ae[1:-1, -2] = 0.0
556         An[-2, 1:-1] = 0.0
557         As[1, 1:-1] = 0.0
558         Ap = - (Ae + Aw + An + As)
```

```
559
560         # Set old variables for residual calculation
561         u_old, v_old, p_old = u.copy(), v.copy(), p.copy()
562         omega = np.zeros((ny, nx))  # Will be recalculated in first
            ↪ iteration
563         omega_old = omega.copy()
564         Uwall_old = abs(u[-1,1:] + u[-2,1:]) / 2
565         Ek0 = Ek_store[-1]
566
567         print(f"Restarted simulation from t={t:.4f}")
568     else:
569         print("Failed to load checkpoint, starting from scratch")
570         # Initialize as before
571 else:
572     print("No checkpoint found, starting new simulation")
573     # Initialize as before
574
575
576 #---IMEX-----------------------------------------------------------------------
577 wall_times = [0.0]
578 cont_it = int(0)
579
580 start_time = time.time()
581 for cont_it in range(1, nit+1):
582
583     u, v = apply_BC(u, v, Utop)
584
585     ui_int = u[1:-1, 2:-1]
586     vi_int = v[2:-1, 1:-1]
587
588     K_hat = []
589     U = []
590     K = []
591
592     p0 = p.copy()
593
594     Ku0, Kv0, Uu0, Uv0, Ku_hat0, Kv_hat0 = do_1st_RK_Step(u, v, dx,
        ↪ dy, dt, nu, A, A_hat, Iu, Iv, Lu, Lv, Su, p0, Utop, App)
595
596     K_hat.append((Ku_hat0, Kv_hat0))
```

```
597     U.append((Uu0, Uv0))
598     K.append((Ku0, Kv0))
599 
600     for i in range(1, s):
601         pi = p.copy()
602         Mui = Iu - nu * dt * A[i,i] * Lu
603         Mvi = Iv - nu * dt * A[i,i] * Lv
604 
605         Kui_hat, Kvi_hat = compute_explicit(U[i-1][0], U[i-1][1],
            ↪ dx, dy)
606         K_hat.append((Kui_hat, Kvi_hat))
607 
608         Zui = u + dt * A_hat[i+1, i] * K_hat[i][0]
609         Zvi = v + dt * A_hat[i+1, i] * K_hat[i][1]
610 
611         for j in range(i):
612             Zui += dt * (A[i,j] * K[j][0] + A_hat[i+1,j] *
                ↪ K_hat[j][0])
613             Zvi += dt * (A[i,j] * K[j][1] + A_hat[i+1,j] *
                ↪ K_hat[j][1])
614 
615         Zui_int = np.zeros([ny, nx-1])
616         Zvi_int = np.zeros([ny-1, nx])
617         Zui_int = Zui[1:-1, 2:-1]
618         Zvi_int = Zvi[2:-1, 1:-1]
619 
620         bi_int = Zui_int.ravel() + nu * dt * A[i,i] * Su
621 
622         # Use CG for both systems (consistent with first step)
623         Mpui = diags(1.0/Mui.diagonal())  # Jacobi preconditioner
624         Uui_vec, info_u = cg(Mui, bi_int, x0=ui_int.ravel(),
            ↪ rtol=1e-06, M=Mpui)
625 
626         Mpvi = diags(1.0/Mvi.diagonal())  # Jacobi preconditioner
627         Uvi_vec, info_v = cg(Mvi, Zvi_int.ravel(),
            ↪ x0=vi_int.ravel(), rtol=1e-06, M=Mpvi)
628 
629         Uuit_int = Uui_vec.reshape(Zui_int.shape) #[ny, nx-1]
630         Uvit_int = Uvi_vec.reshape(Zvi_int.shape) #[ny-1, nx]
631 
```

```
632         Uuit = np.zeros_like(u)
633         Uvit = np.zeros_like(v)
634         Uuit[1:-1, 2:-1] = Uuit_int
635         Uvit[2:-1, 1:-1] = Uvit_int
636         Uuit, Uvit = apply_BC(Uuit, Uvit, Utop)
637 
638         #dti = (A[i,i]) * dt
639 
640         Uui, Uvi, _, dpdxi, dpdyi, _ = pressure_correction(Uuit,
            ↪  Uvit, App, pi, dt, dx, dy, ml)
641 
642         Uui, Uvi = apply_BC(Uui, Uvi, Utop)
643         U.append((Uui, Uvi))
644 
645         Kui, Kvi = compute_implicit(Uui, Uvi, nu, dx, dy)
646 
647         #Kui[1:-1, 2:-1] = Kui[1:-1, 2:-1] - dpdxi
648         #Kvi[2:-1, 1:-1] = Kvi[2:-1, 1:-1] - dpdyi
649 
650         K.append((Kui, Kvi))
651 
652     #Compute the last K_hat
653     Kus_hat, Kvs_hat = compute_explicit(U[-1][0], U[-1][1], dx, dy)
654     K_hat.append((Kus_hat, Kvs_hat))
655 
656     uhast = u + dt * b_hat[-1] * K_hat[-1][0]
657     vhast = v + dt * b_hat[-1] * K_hat[-1][1]
658 
659     for i in range(s):
660         uhast += dt * (b[i] * K[i][0] + b_hat[i] * K_hat[i][0])
661         vhast += dt * (b[i] * K[i][1] + b_hat[i] * K_hat[i][1])
662 
663     # uhast_int = np.zeros([ny, nx-1])
664     # vhast_int = np.zeros([ny-1, nx])
665     # uhast_int = uhast[1:-1, 2:-1]
666     # vhast_int = vhast[2:-1, 1:-1]
667 
668     u, v = apply_BC(u, v, Utop)
669     u, v, p, dpdxn, dpdyn, Rp = pressure_correction(uhast, vhast,
        ↪  App, p.copy(), dt, dx, dy, ml)
```

```
670
671     u, v = apply_BC(u, v, Utop)
672
673     NS_u, NS_v = NS_operator(u_old, v_old, nu, dx, dy, dpdxn, dpdyn)
674
675     #NS_u, NS_v = compute_implicit(u, v, nu, dx, dy)
676     #NS_u[1:-1, 2:-1] = NS_u[1:-1, 2:-1] - dpdxn
677     #NS_v[2:-1, 1:-1] = NS_v[2:-1, 1:-1] - dpdyn
678     Ru = np.zeros([ny, nx+1])
679     Ru = np.zeros([ny+1, nx])
680
681     Ru = abs((u[1:-1, 2:-1]-u_old[1:-1, 2:-1])) / np.max(u_old[1:-1,
        ↪  2:-1])
682     Rv = abs((v[2:-1, 1:-1]-v_old[2:-1, 1:-1])) / np.max(v_old[2:-1,
        ↪  1:-1])
683
684     Uwalli = abs(u[-1,1:] + u[-2,1:]) / 2
685     Uwallt.append(Uwalli)
686
687     # Calculate and store mean Uwall
688     Uwall_mean = np.mean(Uwalli)
689     Uwall_mean_history.append(Uwall_mean)
690
691     #Vorticity Calculations
692     dvdx = np.zeros((ny, nx))
693     dudy = np.zeros((ny, nx))
694     omega = np.zeros((ny, nx))
695     # Calculate dvdx for all j and i at once
696     dvdx = (v[1:-1, 2:] - v[1:-1, :-2]) / (2 * dx)
697
698     # Calculate dudy for all j and i at once
699     dudy = (u[2:, 1:-1] - u[:-2, 1:-1]) / (2 * dy)
700
701     # Compute vorticity
702     omega = dvdx - dudy
703
704     ui_centers = 0.5*(u[1:-1,1:-1] + u[1:-1,2:])  # Average u to
        ↪  cell centers
705     vi_centers = 0.5*(v[1:-1,1:-1] + v[2:,1:-1])
706
```

```
707         Eki = 0.5 * np.sum(ui_centers**2 + vi_centers**2) / nelm
708         #tol = abs(Ek0 - Eki) / Ek0
709         Ek_store.append(Eki)
710 
711         if len(Ek_store) > 1:
712             du = np.linalg.norm(u - u_old) / np.linalg.norm(u_old)  if
                ↪  'u_old' in locals() else 1.0
713             dv = np.linalg.norm(v - v_old) / np.linalg.norm(v_old) if
                ↪  'v_old' in locals() else 1.0
714             dp = np.linalg.norm(p - p_old) / np.linalg.norm(p_old) if
                ↪  'p_old' in locals() else 1.0
715             domega = np.linalg.norm(omega - omega_old) /
                ↪  np.linalg.norm(omega_old) if 'omega_old' in locals()
                ↪  else 1.0
716             dEk = abs(Ek0 - Eki) / Ek0  if len(Ek_store) > 1 else 1.0
717 
718 
719             residuals_u.append(du)
720             residuals_v.append(dv)
721             residuals_p.append(dp)
722             residuals_omega.append(domega)
723             residuals_Ek.append(dEk)
724 
725 
726             dur = np.max(np.abs(Ru))  # Maximum absolute residual for
                ↪  u-momentum
727             dvr = np.max(np.abs(Rv))  # Maximum absolute residual for
                ↪  v-momentum
728             dpr = np.max(np.abs(Rp))  # Maximum absolute residual for p
729             #dur = np.linalg.norm(NS_u)
730             #dvr = np.linalg.norm(NS_v)
731 
732             resNSu.append(dur)
733             resNSv.append(dvr)
734             resNSp.append(dpr)
735 
736         Ek0 = Eki
737         u_old, v_old, p_old = u.copy(), v.copy(), p.copy()
738         omega_old = omega.copy()
739         Uwall_old = Uwalli.copy()
```

```
740         counter += 1
741 
742         current_wall_time = time.time() - start_time
743         wall_times.append(current_wall_time)
744 
745         t = np.float64(cont_it * dt)
746         t_sim.append(t)
747     #Perform th final Post Processing
748 
749     end_time = time.time()
750     elapsed_time = end_time - start_time
751 
752     simtime_filename = f"simulation_time_Re{Re}_{nx}_{ny}_{dt:.4f}.txt"
753     simtime_path = os.path.join(script_dir, simtime_filename)
754 
755     # Write the elapsed time to this new file
756     with open(simtime_path, 'w') as f:
757         f.write(f"Simulation elapsed time (seconds):
            ↪  {elapsed_time:.2f}\n")
758         f.write(f"Re = {Re}, nx = {nx}, ny = {ny}, dt = {dt:.18f}\n")
759 
760     print(f"Simulation time saved to {simtime_path}")
761     print(f"Total simulation time: {elapsed_time:.2f} seconds")
762 
763     final_message = postProcess(u, v, dx, dy, nx, ny, lx, ly, x, y, xx,
        ↪  yy, t,  Re, nu, Utop, p, omega, Ek_store, residuals_u,
        ↪  residuals_v, residuals_p, residuals_Ek, t_sim,
        ↪  Uwall_mean_history, script_dir, resNSu,resNSv, resNSp, Ru, Rv,
        ↪  Rp)
764 
765     plt.figure()
766     plt.plot(t_sim, wall_times, label="Wall time vs Simulation time")
767     plt.xlabel("Simulation time [s]")
768     plt.ylabel("Wall time [s]")
769     plt.title("Simulation Time vs Wall Clock Time")
770     plt.grid(True)
771     plt.legend()
772     plt.tight_layout()
773     plt.savefig(os.path.join(script_dir,
        ↪  f"wall_time_vs_sim_time_Re{Re}_{nx}_{ny}.png"), dpi=300)
```

```
774
775
776 # Save simulation time vs wall time vs Ek
777 sim_vs_wall_file = f"sim_vs_wall_Re{Re}_{nx}_{ny}_dt{dt:.4f}.txt"
778 sim_vs_wall_path = os.path.join(script_dir, sim_vs_wall_file)
779
780 with open(sim_vs_wall_path, 'w') as f:
781     f.write("SimulationTime,WallTime,Ek\n")
782     for sim_t, wall_t, ek in zip(t_sim, wall_times, Ek_store):
783         f.write(f"{sim_t},{wall_t},{ek}\n")
784
785 print(f"Simulation time vs wall time and Ek saved to
    ↪ {sim_vs_wall_path}")
```